



CompTIA PenTest+

(PT0-003)

Study Guide

Version 1.0

Table of Contents

Section 1. Introduction.....	6
Introduction.....	6
Purpose of the PenTest+ Certification.....	6
Expected Background and Prerequisites.....	6
PenTest+ Exam Domains Overview.....	6
Course Structure and Objective Mapping.....	6
Exam Question Style and Difficulty.....	6
Exam Logistics and Scoring.....	6
Study Tips and Student Resources.....	6
Exam Tips.....	6
Understanding How Exam Questions Are Written.....	6
Identifying Distractors and Keywords.....	6
Answering from the CompTIA Perspective.....	6
Choosing the Best Answer.....	6
Avoiding the Urge to Fight the Exam.....	6
Recognizing Concepts and Tools.....	6
100% Pass Guarantee.....	6
Course Components and Learning Structure.....	6
Checkpoint Quizzes and Progress Requirements.....	6
Practice Exams as a Learning Tool.....	6
Using Practice Exams Effectively.....	6
Completion, Support, and Certification.....	6
Section 2. PenTesting Overview.....	6
PenTesting Overview.....	6
Ethical and Professional Use of Penetration Testing Skills.....	6
What Penetration Testing Is and Why It Matters.....	7
Purpose of This Section.....	7
Planning the Engagement.....	7
Information Gathering.....	7

Attacks and Exploits.....	7
Post-Exploitation.....	8
Reporting.....	8
Planning the Engagement.....	8
Importance of the Planning Phase.....	8
Defining the Scope.....	8
Legal and Regulatory Considerations.....	8
Rules of Engagement.....	9
Non-Disclosure Agreements.....	9
Statement of Work.....	9
Communication and Escalation Paths.....	9
Risk Tolerance and Business Impact.....	9
Authorization to Proceed.....	9
Information Gathering.....	10
Passive Reconnaissance.....	10
Active Reconnaissance.....	11
Documentation and Stealth Considerations.....	11
Attacks and Exploits.....	12
Purpose of the Attacks and Exploits Phase.....	12
Target Prioritization.....	12
Using Exploitation Frameworks.....	12
Password Attacks.....	12
Phishing Attacks.....	12
Web Application Attacks.....	13
Documentation and Evidence Collection.....	13
Post-Exploitation.....	13
Purpose of the Post-Exploitation Phase.....	13
Establishing Persistence.....	14
Privilege Escalation.....	14
Lateral Movement.....	14

Pivoting to Other Network Segments.....	14
Actions on Objectives and Data Exfiltration.....	15
Cleanup and Covering Tracks.....	15
Reporting.....	15
Purpose of Penetration Test Reporting.....	15
Executive Summary.....	16
Methodology, Limitations, and Assumptions.....	16
Detailed Findings.....	16
Risk Prioritization.....	16
Attack Narrative.....	17
Recommendations and Remediation.....	17
Section 3. Pre-Engagement Activities.....	17
Pre-Engagement Activities (OBJ 1.1, 1.2).....	17
Key Concepts.....	17
Engagement Planning and Objectives.....	18
Agreements, Communication, and Risk Management.....	18
Scope and Target Definition.....	18
Exam Focus.....	18
Regulations and Standards (OBJ 1.1, 1.2).....	18
Key Concepts.....	18
Regulatory Requirements.....	18
Industry Standards and Frameworks.....	19
Stakeholder Alignment.....	19
Exam Focus.....	19
Types of Assessments (OBJ 1.1).....	19
Key Concepts.....	19
Infrastructure and Network Assessments.....	19
Application and Platform Assessments.....	19
Cloud and Integration Assessments.....	20
Exam Focus.....	20

Types of Agreements (OBJ 1.1).....	20
Key Concepts.....	20
Confidentiality and Legal Protection.....	20
Business Relationship Management.....	21
Engagement Scope and Execution.....	21
Service Usage Conditions.....	21
Exam Focus.....	21
Legal and Ethical Considerations (OBJ 1.1, 1.2).....	21
Key Concepts.....	21
Authorization and Legal Approval.....	21
Disclosure and Professional Risk.....	21
Incident Handling and Escalation.....	21
Exam Focus.....	22
Lesson: Rules of Engagement (OBJ 1.1, 1.2).....	22
Key Concepts.....	22
ROE Structure and Purpose.....	22
Testing Constraints and Execution.....	22
Adaptive Testing and Reporting.....	22
Exam Focus.....	22
Target Selection (OBJ 1.1).....	22
Key Concepts.....	22
Network and Infrastructure Targets.....	23
External and Host-Level Targets.....	23
Application-Level Targets.....	23
Exam Focus.....	23
Shared Responsibility Model (OBJ 1.1).....	23
Key Concepts.....	23
Lesson Focus.....	23
Exam Focus.....	23
Section 4. Frameworks.....	23

Frameworks (OBJ 1.3).....	24
Key Concepts.....	24
Framework Overview and Purpose.....	24
Exam Focus.....	24
MITRE ATT&CK (OBJ 1.3).....	24
Key Concepts.....	24
ATT&CK Structure and Matrices.....	24
Penetration Testing Applications.....	24
Exam Focus.....	24
OWASP (OBJ 1.3).....	25
Key Concepts.....	25
OWASP Top 10 Focus.....	25
Web Application Testing Value.....	25
Exam Focus.....	25
MASVS (OBJ 1.3).....	25
Key Concepts.....	25
MASVS Control Domains.....	25
Exam Focus.....	25
PTES (OBJ 1.3).....	26
Key Concepts.....	26
PTES Seven Phases.....	26
Exam Focus.....	26
CREST (OBJ 1.3).....	26
Key Concepts.....	26
Defensible Penetration Testing Requirements.....	26
Professional Reporting Standards.....	27
Exam Focus.....	27
OSSTMM (OBJ 1.3).....	27
Key Concepts.....	27
Operational Security Measurement.....	27

Professional Development.....	27
Exam Focus.....	27
STRIDE (OBJ 1.3).....	27
Key Concepts.....	27
STRIDE Threat Categories.....	28
Exam Focus.....	28
Purdue Model (OBJ 1.3).....	28
Key Concepts.....	28
ICS Hierarchical Levels.....	28
Penetration Testing Value.....	28
Exam Focus.....	28
OCTAVE (OBJ 1.3).....	28
Key Concepts.....	28
OCTAVE Three Phases.....	29
Exam Focus.....	29
DREAD (OBJ 1.3).....	29
Key Concepts.....	29
DREAD Categories and Scoring.....	29
Severity Classification.....	29
Exam Focus.....	29
Section 5. Information Gathering.....	29
Information Gathering (OBJ 2.1).....	29
Key terms.....	30
• Exam Focus.....	30
Passive Reconnaissance (OBJ 2.1).....	30
Major passive sources include.....	30
Exam Focus.....	30
Network Sniffing (OBJ 2.1).....	31
Key concepts:.....	31

Exam Focus.....	31
Active Reconnaissance (OBJ 2.1).....	31
Core methods.....	31
Exam Focus.....	32
Port and Protocol Scanning (OBJ 2.1).....	32
TCP Scanning.....	32
UDP Scanning.....	32
Exam Focus.....	32
HTML Scraping and Cached Pages (OBJ 2.1).....	32
Techniques include.....	32
Cached pages provide historical insight.....	33
Exam Focus.....	33
Banner Grabbing (OBJ 2.1).....	33
Common tools.....	33
Exam Focus.....	33
Section 6. Open-Source Intelligence.....	33
Open-Source Intelligence (OSINT) (OBJ 2.1).....	33
Why OSINT Matters in a Penetration Test.....	34
OSINT Categories You Must Recognize.....	34
Exam Focus.....	34
Social Media and Job Boards (OBJ 2.1).....	34
Social Media Intelligence for Pretexting.....	34
Signals to look for on Platforms.....	35
Job Board Disclosures and Technology Mapping.....	35
Exam Focus.....	35
Information Disclosures (OBJ 2.1).....	35
Common Disclosure Vectors You'll See in the Wild.....	35
OSINT Discovery Techniques for Disclosures.....	36
How Disclosures Become Attack Paths.....	36
Exam Focus.....	36

Cryptographic Flaws (OBJ 2.1).....	36
Weak or Deprecated Protocols and Cipher Suites.....	36
Certificate Metadata as Infrastructure Intelligence.....	37
Revocation and Trust Validation Concepts.....	37
Exam Focus.....	37
DNS Lookups (OBJ 2.1).....	37
Tools and Lookup Types.....	37
DNS Security Risks That Show Up in Scenarios.....	37
Exam Focus.....	38
Certificate Transparency Logs (OBJ 2.1).....	38
What CT Logs Reveal to a Penetration Tester.....	38
Rogue or Misissued Certificates and Risk.....	38
Practical OSINT Workflow Using CT Data.....	38
Exam Focus.....	38
Search Engine Analysis (OBJ 2.1).....	39
Core Operators You Must Be Able to Use or Recognize.....	39
Common Findings from Search Engine Enumeration.....	39
Monitoring and Non-Text Discovery.....	39
Exam Focus.....	39
Section 7. Scanning and Enumeration.....	40
Scanning and Enumeration (OBJ 2.2).....	40
Active Reconnaissance Transition.....	40
Target Environment Mapping.....	40
Enumeration as Investigation.....	40
Exam Focus.....	40
Enumerating Protocols (OBJ 2.2).....	40
Network Layer Enumeration.....	41
Transport Layer Enumeration.....	41
Application and Remote Access Protocol Enumeration.....	41
Exam Focus.....	41

Enumerating DNS (OBJ 2.2).....	42
DNS Query Techniques.....	42
Zone Transfers and Record Enumeration.....	42
Brute Force and Advanced DNS Discovery.....	42
Exam Focus.....	42
Enumerating Directories (OBJ 2.2).....	42
Automated Directory Discovery.....	43
Metadata and Server Response Clues.....	43
Advanced Enumeration Techniques.....	43
Exam Focus.....	43
Enumerating Hosts (OBJ 2.2).....	43
Host Discovery Techniques.....	43
Alternative Discovery Methods.....	44
Service Scanning and OS Fingerprinting.....	44
Advanced and Credentialed Enumeration.....	44
Exam Focus.....	44
Enumerating Users (OBJ 2.2).....	44
Windows User Enumeration.....	44
Group Membership and Privilege Discovery.....	44
Linux User Enumeration.....	45
Attack Implications.....	45
Exam Focus.....	45
Enumerating Email (OBJ 2.2).....	45
Protocol-Based Email Enumeration.....	45
OSINT Email Collection.....	45
Breach Intelligence Sources.....	45
Validation and Testing.....	46
Exam Focus.....	46
Enumerating Permissions (OBJ 2.2).....	46
File System and Local Permission Enumeration.....	46

Database Permission Enumeration.....	46
Network Share and Active Directory Enumeration.....	46
Cloud Permission Enumeration.....	47
Exam Focus.....	47
Enumerating Wireless Devices (OBJ 2.2).....	47
Wireless Reconnaissance Techniques.....	47
Tools and Packet Analysis.....	47
WiGLE and Signal Amplification.....	47
Exam Focus.....	48
Enumerating Secrets (OBJ 2.2).....	48
Password Enumeration Techniques.....	48
Session Token Enumeration.....	48
Cloud Access Key Enumeration.....	48
API Key Enumeration.....	49
Exam Focus.....	49
Enumerating the Web (OBJ 2.2).....	49
Automated Web Crawling and Discovery.....	49
Manual Enumeration and Website Metadata.....	49
Plugin Enumeration and Vulnerability Research.....	49
Web Application Firewalls (WAFs).....	50
Exam Focus.....	50
Attack Path Mapping (OBJ 2.2).....	50
Mapping Techniques and Tools.....	50
Common Attack Progression Examples.....	50
Documentation and Continuous Improvement.....	50
Exam Focus.....	51
Section 8. Reconnaissance and Enumeration Tools.....	51
Reconnaissance and Enumeration Tools (OBJ 2.4).....	51
Tool-Driven Reconnaissance.....	51
Domain 2 Objective Alignment.....	51

Practical Application.....	51
Exam Focus.....	51
Wayback Machine (OBJ 2.4).....	51
Role in Reconnaissance.....	52
Historical Snapshot Intelligence.....	52
Limitations and Use.....	52
Exam Focus.....	52
theHarvester and Hunter.io (OBJ 2.4).....	52
theHarvester Capabilities.....	52
Applying Harvested Data.....	52
Hunter.io Email Verification.....	52
Exam Focus.....	52
OSINT Tools (OBJ 2.4, 4.8).....	53
OSINT Framework Directory.....	53
Maltego Relationship Mapping.....	53
SpiderFoot Automation.....	53
Exam Focus.....	53
DNSdumpster and Amass (OBJ 2.4).....	53
DNSdumpster Passive Mapping.....	53
Subdomain Discovery Value.....	53
Amass Active and Passive Enumeration.....	54
Exam Focus.....	54
Shodan and Censys.io (OBJ 2.4).....	54
Shodan Device Indexing.....	54
Censys Structured Intelligence.....	54
Automation and Monitoring.....	54
Exam Focus.....	54
tcpdump (OBJ 2.4).....	54
Role in Packet Visibility.....	54
Reconnaissance and Exploitation Support.....	55

Filters and Targeted Captures.....	55
Exam Focus.....	55
Wireshark (OBJ 2.4).....	55
Web Vulnerability Evidence.....	55
Cleartext Credential Exposure.....	55
DNS and Infrastructure Enumeration.....	55
Exam Focus.....	55
Wireless Analysis Tools (OBJ 2.4).....	55
Aircrack-ng Suite.....	56
InSSIDer RF Visibility.....	56
WiGLE Remote Mapping.....	56
Exam Focus.....	56
Section 9. Nmap and NSE.....	56
Section 9. Nmap and NSE (Expanded Standalone Version).....	56
Nmap and NSE (OBJ 2.4, 4.2).....	56
What the Nmap Scripting Engine Adds.....	56
How PenTest+ Tests Nmap Use.....	57
Exam Focus.....	57
Nmap Discovery Scans (OBJ 2.4, 4.2).....	57
Discovery Probe Types and When to Use Them.....	57
Timing, Stealth, and Realistic Engagement Considerations.....	57
Exam Focus.....	57
Nmap Port Scans (OBJ 2.4, 4.2).....	57
Key Scan Types and Tradeoffs.....	58
Interpreting Port States for Exploitation Planning.....	58
Exam Focus.....	58
Nmap Fingerprinting (OBJ 2.4, 4.2).....	58
Service Version Detection and OS Identification.....	58
Deep Enumeration with NSE Scripts.....	58
Exam Focus.....	59

Section 10. Scripting Basics.....	59
Scripting Basics (OBJ 2.3).....	59
Pseudocode as a Universal Language.....	59
Core Programming Building Blocks.....	59
PenTest+ Objective Relevance.....	59
Exam Focus.....	59
Shells and Programming Languages (OBJ 2.3).....	60
Bash and Linux Automation.....	60
PowerShell and Windows Scripting.....	60
High-Level Languages for Pentesting.....	60
Exam Focus.....	60
Variables and Constants (OBJ 2.3).....	60
Core Data Types.....	60
Variables as Dynamic Storage.....	60
Constants as Fixed Values.....	61
Exam Focus.....	61
Loops (OBJ 2.3).....	61
For Loops for Fixed Iterations.....	61
While Loops for Conditional Repetition.....	61
Do Loops for Guaranteed Execution.....	61
Exam Focus.....	61
Logic Control (OBJ 2.3).....	61
Boolean Decision Making.....	61
Arithmetic Comparisons in Scripts.....	62
String Matching for Responses.....	62
Exam Focus.....	62
Data Structures (OBJ 2.3).....	62
Key-Value Pairs and JSON.....	62
Arrays, Lists, and Dictionaries.....	62
CSV and Tree Structures.....	62

Exam Focus.....	62
Object Oriented Programming (OBJ 2.3).....	62
Functions and Reusable Tasks.....	63
Classes and Object Templates.....	63
Libraries and Dependencies.....	63
Exam Focus.....	63
Section 11. Modifying Scripts.....	63
Modifying Scripts (OBJ 2.3).....	63
What 'modify scripts' means in a penetration-testing scenario.....	63
A repeatable workflow for modifying scripts without breaking them.....	64
Operational discipline and scope controls.....	64
Exam Focus.....	64
Bash Fundamentals (OBJ 2.3).....	65
Script structure, the shebang, and comments.....	65
Variables, constants, and arrays for handling targets and results.....	65
Conditionals, loops, and input/output in automation.....	66
Exam Focus.....	66
Understanding a Bash Script (OBJ 2.3).....	66
Identifying inputs, validating arguments, and defining working directories.....	66
Tracing the loop and understanding per-target execution.....	67
Interpreting output handling, error capture, and scan 'loudness'.....	67
Exam Focus.....	68
Modifying a Bash Script (OBJ 2.3).....	68
Understanding the baseline behavior of a ping sweep.....	68
Identifying the safest modification points.....	68
Extending the script from reachability to reconnaissance.....	69
Exam Focus.....	69
PowerShell Fundamentals (OBJ 2.3).....	69
Comments, readability, and PowerShell help conventions.....	69
Variables, types, arrays, and hashtables.....	70

Conditionals, loops, pipelines, and input/output.....	70
Exam Focus.....	70
Understanding a PowerShell Script (OBJ 2.3).....	71
Collecting context: timestamps, host identity, and OS information.....	71
Branching logic to handle different Windows versions.....	71
Extracting actionable findings: drives, users, and SMBv1 status.....	71
Exam Focus.....	72
Modifying a PowerShell Script (OBJ 2.3).....	72
Baseline process enumeration and ownership filtering.....	72
Reliability improvements: try/catch and clean, sortable output.....	72
Adding an elevation signal to prioritize follow-up actions.....	73
Exam Focus.....	73
Python Fundamentals (OBJ 2.3).....	73
Syntax cues: indentation, blocks, and comments.....	74
Variables, types, type casting, and constant conventions.....	74
Data structures, slicing, and control flow for automation.....	74
Exam Focus.....	75
Understanding a Python Script (OBJ 2.3).....	75
Recognizing Python and identifying key components.....	75
Following the request flow: opener, headers, and response handling.....	75
Error handling and output decisions.....	76
Exam Focus.....	76
Modifying a Python Script (OBJ 2.3).....	76
Confirming baseline behavior and identifying modification goals.....	76
Refactoring the function: return the right data, not just different text.....	77
Updating output logic to enumerate headers cleanly.....	77
Exam Focus.....	77
Section 12. Analyzing Scans.....	77
Analyzing Scans (OBJ 3.2, 4.1).....	77
Interpreting Scan Output.....	78

Objectives 3.2 and 4.1 Alignment.....	78
Why Analysis Drives Exploitation.....	78
Exam Focus.....	78
Positive and Negative Results (OBJ 3.2).....	78
Understanding the Confusion Matrix.....	78
Operational Impact of False Positives.....	78
Reducing False Negatives.....	79
Exam Focus.....	79
Validating Scan Results (OBJ 3.2).....	79
Manual Verification Techniques.....	79
Reconnaissance and Enumeration Correlation.....	79
Safe Validation Practices.....	79
Exam Focus.....	79
Using CVEs and CVSS (OBJ 4.1).....	79
CVE Identification and Databases.....	80
CVSS Scoring Breakdown.....	80
Threat Intelligence and Advisories.....	80
Exam Focus.....	80
Exploit Prediction Scoring System (OBJ 4.1).....	80
EPSS vs CVSS.....	80
Probability-Based Prioritization.....	80
Operational Benefits.....	80
Exam Focus.....	81
Target Prioritization (OBJ 4.1).....	81
High-Value Asset Identification.....	81
End-of-Life Systems.....	81
Default Configurations.....	81
Exam Focus.....	81
Common Target Criteria (OBJ 4.1).....	81
Running Services Enumeration.....	81

Vulnerable Cryptography.....	81
Defensive Capabilities.....	82
Exam Focus.....	82
Scan Validations (OBJ 3.2).....	82
Configuration Validation.....	82
Authenticated vs Unauthenticated Scans.....	82
Completeness Checks.....	82
Exam Focus.....	82
Capability Selection (OBJ 3.2, 4.1).....	82
Tool Selection.....	82
Exploit Customization.....	83
Public Exploit Vetting.....	83
Exam Focus.....	83
Documenting the Attack (OBJ 4.1).....	83
Attack Path Documentation.....	83
Low-Level Diagrams.....	83
Storyboards for Stakeholders.....	83
Exam Focus.....	83
Other Attack Considerations (OBJ 4.1).....	84
Scope Limitations.....	84
Labeling Sensitive Systems.....	84
Managing Dependencies.....	84
Exam Focus.....	84
Section 13. Discovering Vulnerabilities.....	84
Discovering Vulnerabilities (OBJ 3.1).....	84
Vulnerability Discovery as Offensive Reconnaissance.....	84
Broad Attack Surface Coverage.....	84
From Findings to Exploitation Planning.....	85
Exam Focus.....	85
Application Scanning (OBJ 3.1).....	85

Dynamic Application Testing Mindset.....	85
DAST: External Runtime Scanning.....	85
IAST: Internal Execution Visibility.....	85
Exam Focus.....	85
Software Analysis (OBJ 3.1).....	86
Static Insight into Software Weaknesses.....	86
SCA: Third-Party Dependency Risk.....	86
SAST: Developer Code Weakness Detection.....	86
Exam Focus.....	86
Host-Based Scanning (OBJ 3.1).....	86
Host-Level Vulnerability Discovery.....	86
Authenticated vs Unauthenticated Perspectives.....	86
Secrets Scanning for High-Impact Exposure.....	87
Exam Focus.....	87
Network Scanning (OBJ 3.1).....	87
Network Reconnaissance Foundations.....	87
TCP and UDP Scanning Techniques.....	87
Stealth Scanning and Evasion.....	87
Exam Focus.....	87
Mobile Scanning (OBJ 3.1).....	87
Mobile Attack Surface Complexity.....	87
Static Mobile Analysis (SAST).....	88
Dynamic Testing and Permission Review.....	88
Exam Focus.....	88
Scanning IaC (OBJ 3.1).....	88
IaC as Systemic Risk Amplifier.....	88
Static Template Analysis.....	88
Policy Enforcement and Drift Detection.....	88
Exam Focus.....	88
ICS Vulnerability Discovery (OBJ 3.1).....	88

ICS Environments as High-Risk Targets.....	88
Manual and Configuration-Based Discovery.....	89
Passive Monitoring with Port Mirroring.....	89
Exam Focus.....	89
Wireless Scans (OBJ 3.1).....	89
Wireless Reconnaissance Importance.....	89
SSID and Channel Enumeration.....	89
Weak Encryption and Misconfiguration Discovery.....	89
Exam Focus.....	89
Section 14. Vulnerability Discovery Tools.....	89
Vulnerability Discovery Tools (OBJ 3.1).....	90
Tool-Based Vulnerability Discovery.....	90
Offensive Reconnaissance Acceleration.....	90
Tool Output Interpretation for Exploitation.....	90
Exam Focus.....	90
Trivy (OBJ 3.1).....	90
Filesystem and Secrets Scanning.....	90
Container Image Vulnerability Enumeration.....	90
Repository and Dependency Discovery.....	91
Exam Focus.....	91
BloodHound (OBJ 3.1).....	91
Active Directory Graph Mapping.....	91
SharpHound Data Collection.....	91
Privilege Escalation Path Discovery.....	91
Exam Focus.....	91
PowerSploit (OBJ 3.1).....	91
Post-Exploitation PowerShell Modules.....	91
Credential Harvesting and Privilege Escalation.....	92
Modern Detection and Tool Limitations.....	92
Exam Focus.....	92

Grype (OBJ 3.1).....	92
SBOM and Dependency Vulnerability Analysis.....	92
Container Image Scanning for Exploitable Packages.....	92
Filesystem Scanning After Initial Access.....	92
Exam Focus.....	92
Kube-Hunter (OBJ 3.1).....	92
Kubernetes Component Reconnaissance.....	93
Cluster Misconfiguration Discovery.....	93
Modern Tool Replacement and Limitations.....	93
Exam Focus.....	93
TruffleHog (OBJ 3.1).....	93
Secret Discovery in Git History.....	93
Local and Remote Repository Scanning.....	93
Credential Exploitation and Cloud Pivoting.....	93
Exam Focus.....	94
Section 15. Social Engineering Attacks.....	94
Social Engineering Attacks (OBJ 4.8).....	94
Why social engineering works in real engagements.....	94
Common social engineering outcomes that matter to an attacker.....	94
Ethical boundaries, authorization, and controlled demonstrations.....	95
Exam Focus.....	95
Methods of Influence (OBJ 4.8).....	95
Authority and urgency as compliance accelerators.....	95
Social proof and scarcity as behavior shapers.....	96
Likeness and fear as emotional shortcuts.....	96
Exam Focus.....	96
Phishing Campaigns (OBJ 4.8).....	96
Broad phishing and why volume still works.....	96
Targeted phishing: spearphishing, whaling, and BEC.....	97
Alternative channels: vishing and smishing.....	97

Exam Focus.....	97
Gophish (OBJ 4.8).....	97
Designing a campaign that mirrors real attacker tradecraft.....	98
Tracking outcomes: open, click, and credential submission signals.....	98
Chaining results into controlled follow-on demonstrations.....	98
Exam Focus.....	98
Impersonation (OBJ 4.8).....	98
Building a credible pretext and controlling the conversation.....	99
Physical impersonation and why roles bypass doors.....	99
Elicitation as the information engine of impersonation.....	99
Exam Focus.....	99
Surveillance Techniques (OBJ 4.8).....	100
Pattern observation and environmental mapping.....	100
Eavesdropping and shoulder surfing as low-cost intelligence.....	100
Dumpster diving and post-disposal intelligence.....	100
Exam Focus.....	100
Watering Hole (OBJ 4.8).....	101
Selecting the watering hole through behavioral reconnaissance.....	101
Compromise mechanics: drive-by delivery and browser exploitation.....	101
Post-compromise leverage and operational value.....	101
Exam Focus.....	101
Evilginx (OBJ 4.8).....	102
Reverse-proxy phishing and why it defeats simple MFA expectations.....	102
Operational flow: from lure to token capture.....	102
Impact: session hijacking and account takeover.....	102
Exam Focus.....	102
Tailgating and Piggybacking (OBJ 4.8).....	103
Tailgating as silent entry through routine behavior.....	103
Piggybacking as social compliance at the door.....	103
Post-entry value: why physical access accelerates compromise.....	103

Exam Focus.....	103
Browser Exploitation Framework (BeEF) (OBJ 4.8).....	104
Why browsers are high-value targets in offensive operations.....	104
Hooking mechanics and establishing control.....	104
Post-hook capabilities: reconnaissance, influence, and chaining.....	104
Exam Focus.....	105
Section 16. Wireless Attacks.....	105
Wireless Attacks (OBJ 4.7).....	105
Wireless attacks as an offensive workflow.....	105
Where wireless attacks create leverage.....	105
How this section maps to Objective 4.7.....	106
Exam Focus.....	106
Wireless Security (OBJ 4.7).....	106
Pre-shared keys as an attack surface.....	106
Encryption evolution and what it changes for the attacker.....	107
Convenience features that create bypass paths.....	107
Exam Focus.....	107
Wireless Signal Exploitation (OBJ 4.7).....	107
Disruption and forced behavior.....	108
Tactics as reconnaissance at scale.....	108
Using discovery platforms and analyzers effectively.....	108
Exam Focus.....	108
Aircrack-ng (OBJ 4.7).....	109
From passive capture to actionable authentication material.....	109
Deauthentication as a control lever.....	109
Offline cracking and what “success” really means.....	109
Exam Focus.....	110
Captive Portal Attacks (OBJ 4.7).....	110
Why captive portals are persuasive by default.....	110
Captive portal exploitation through rogue infrastructure.....	110

When the portal software becomes the vulnerability.....	111
Exam Focus.....	111
Evil Twin (OBJ 4.7).....	111
Impersonation at the network layer.....	111
Why framework-driven setups scale the attack.....	111
What interception really enables after association.....	112
Exam Focus.....	112
Kismet (OBJ 4.7).....	112
Passive detection as stealth reconnaissance.....	112
Hidden networks and the failure of obscurity.....	113
Turning captures into next-phase artifacts.....	113
Exam Focus.....	113
Wi-Fi Protocol Fuzzing (OBJ 4.7).....	113
Frame types and why they matter to fuzzing.....	114
How malformed frames expose weak validation.....	114
Operationalizing fuzzing in authorized testing.....	114
Exam Focus.....	114
Section 17. Network Attacks.....	115
Network Attacks (OBJ 4.2).....	115
Scope of Network Attacks.....	115
Attack Categories Covered in This Section.....	115
Exam Focus.....	115
Stress Testing (OBJ 4.2).....	115
Understanding Stress Testing.....	115
Architectural Constraints and Scaling.....	116
Tools and Load Generation Techniques.....	116
Exam Focus.....	116
Bypassing Segmentation (OBJ 4.2).....	116
Segmentation Bypass Techniques.....	116
VLAN Hopping Attacks.....	116

Multihomed Host Risks.....	116
Exam Focus.....	116
MAC Spoofing (OBJ 4.2).....	117
Spoofing and MAC Address Identity.....	117
Bypassing Layer 2 Access Controls.....	117
Tools for MAC Spoofing in Practice.....	117
Exam Focus.....	117
NAC Bypass (OBJ 4.2).....	118
How Network Access Control Works.....	118
Agent Models and Enforcement Types.....	118
Offensive Techniques for NAC Bypass.....	118
Exam Focus.....	118
Session-Based Attacks (OBJ 4.2).....	119
On-Path Interception Attacks.....	119
Replay and Relay Exploitation.....	119
SSL Stripping and Downgrade Attacks.....	119
Exam Focus.....	119
Service Exploitation (OBJ 4.2).....	119
Misconfigured Services.....	120
Share Enumeration and Exploitation.....	120
Certificate Services Attacks.....	120
Exam Focus.....	120
Packet Crafting (OBJ 4.2).....	120
Packet Crafting Fundamentals.....	120
Relay Attacks with Impacket.....	121
Custom Packets with Scapy.....	121
Exam Focus.....	121
Netcat (OBJ 4.2).....	121
Remote Shell Access.....	121
Bind vs Reverse Shells.....	121

Netcat for Shells and Exfiltration.....	122
Exam Focus.....	122
Default Network Credentials (OBJ 4.3).....	122
Why Default Credentials Matter.....	122
Exploiting Defaults During Engagements.....	122
Lateral Movement and Escalation Impact.....	122
Exam Focus.....	122
LLMNR/NBT-NS Poisoning (OBJ 4.3).....	123
LLMNR as a DNS Fallback Weakness.....	123
NBNS and Legacy Name Resolution Abuse.....	123
Responder Poisoning in Practice.....	123
Exam Focus.....	123
ARP Poisoning (OBJ 4.3).....	123
ARP Mapping and Trust Issues.....	123
ARP Spoofing to Redirect Traffic.....	123
Poisoning Tools and On-Path Positioning.....	124
Exam Focus.....	124
Intro to Metasploit (OBJ 4.3).....	124
Metasploit Framework Overview.....	124
Payload Delivery with msfvenom.....	124
Offensive Workflow and Exam Relevance.....	124
Exam Focus.....	124
Section 18. Authentication Attacks.....	124
Authentication Attacks (OBJ 4.2, 4.3, 4.5).....	125
Key Authentication Attack Types.....	125
Collaboration With Tools.....	125
Exam Focus.....	125
Types of Password Attacks (OBJ 4.3).....	125
Brute Force Attacks.....	125
Dictionary and Hybrid Attacks.....	125

Rainbow Table Attacks.....	125
Exam Focus.....	125
Credential Attacks (OBJ 4.3).....	126
Password Spraying.....	126
Password Masking.....	126
Credential Stuffing and MFA Fatigue.....	126
Exam Focus.....	126
Credential Passing Attacks (OBJ 4.3).....	126
Pass-the-Hash (PtH).....	126
Pass-the-Ticket (PtT).....	126
Pass-the-Token.....	126
Exam Focus.....	126
Directory Service Attacks (OBJ 4.2, 4.3).....	127
Kerberos-Based Attacks.....	127
LDAP Injection.....	127
CrackMapExec Automation.....	127
Exam Focus.....	127
CrackMapExec (CME) (OBJ 4.3).....	127
CME Core Purpose.....	127
Post-Exploitation Actions.....	127
Lateral Movement Efficiency.....	127
Exam Focus.....	127
SAML Attacks (OBJ 4.3).....	128
SAML Workflow.....	128
Token Manipulation and Replay.....	128
SAML Injection.....	128
Exam Focus.....	128
OpenID Connect (OIDC) Attacks (OBJ 4.3).....	128
OIDC Token Workflow.....	128
ID Token Manipulation and Replay.....	128

Injection, WebFinger, and SSRF.....	128
Exam Focus.....	128
Hash Attacks (OBJ 4.5).....	129
Baseline Hash Weaknesses.....	129
Salting and Key Stretching.....	129
Collision and Birthday Attacks.....	129
Exam Focus.....	129
Section 19. Host Attacks.....	129
Host Attacks (OBJ 4.4).....	129
Post-exploitation progression.....	129
Key host-based themes.....	129
Exam Focus.....	130
Privilege Escalation (OBJ 4.4).....	130
Vertical vs horizontal escalation.....	130
Windows escalation tooling.....	130
Exam Focus.....	130
Credential Harvesting (OBJ 4.4).....	130
Post-exploitation credential extraction.....	130
Key tools: Mimikatz and Rubius.....	130
Exam Focus.....	131
Misconfigured Endpoints (OBJ 4.4).....	131
PowerShell privilege escalation paths.....	131
Psexec lateral movement.....	131
Exam Focus.....	131
Unquoted Service Paths (OBJ 4.4).....	131
PowerShell detection workflow.....	131
Psexec exploitation.....	131
Exam Focus.....	131
Disabling Security Software (OBJ 4.4).....	132
Registry-based bypass techniques.....	132

Remote disabling with PsExec.....	132
Exam Focus.....	132
Payload Obfuscation (OBJ 4.4).....	132
PowerShell Base64 encoding.....	132
Remote delivery tools.....	132
Exam Focus.....	132
User-Controlled Access Bypass (OBJ 4.4).....	132
Kerberos manipulation with Rubius.....	133
Evil-WinRM post-exploitation channel.....	133
Exam Focus.....	133
Shell and Kiosk Escapes (OBJ 4.4).....	133
PowerShell escape vectors.....	133
Kiosk escape pathways.....	133
Exam Focus.....	133
Log Tampering (OBJ 4.4).....	133
PowerShell log clearing.....	133
Remote tampering with PsExec.....	134
Exam Focus.....	134
Living Off the Land (OBJ 4.4).....	134
Execution and persistence utilities.....	134
Data movement with trusted binaries.....	134
Exam Focus.....	134
Section 20. Web Application Vulnerabilities.....	134
Web Application Vulnerabilities (OBJ 4.3, 4.5).....	134
Offensive exploitation impact.....	134
Exam Focus.....	135
Race Conditions (OBJ 4.5).....	135
Memory-based race exploitation.....	135
TOCTOU and locking behavior.....	135
Exam Focus.....	135

Buffer Overflows (OBJ 4.5).....	135
Stack smashing and NOP slides.....	135
Integer overflow support and mitigations.....	135
Exam Focus.....	135
Authentication Flaws and Insecure References (OBJ 4.5).....	136
Weak session management risks.....	136
IDOR exploitation.....	136
Exam Focus.....	136
Improper Error Handling (OBJ 4.5).....	136
Verbose error leakage.....	136
Authentication enumeration paths.....	136
Exam Focus.....	136
Improper Headers (OBJ 4.5).....	137
Client-side protection headers.....	137
Cross-domain and feature control.....	137
Exam Focus.....	137
Code Signing (OBJ 4.5).....	137
Trust pipeline attack surface.....	137
Signed malware limitations.....	137
Exam Focus.....	137
Vulnerable Components (OBJ 4.5).....	137
REST, JSON, and SOAP risks.....	138
Browser-side execution models.....	138
Exam Focus.....	138
Software Composition (OBJ 4.5).....	138
SCA tool visibility.....	138
Additional software-level weaknesses.....	138
Exam Focus.....	138
Section 21. Web Application Attacks.....	138
Web Application Attacks (OBJ 4.5).....	138

Key Attack Topics Covered.....	139
Exam Focus.....	139
Directory Traversals (OBJ 4.5).....	139
Document root escape mechanics.....	139
File inclusion escalation.....	140
Offensive impact.....	140
Exam Focus.....	140
Cross-Site Scripting (XSS) (OBJ 4.5).....	140
XSS execution model.....	140
Attack sequence.....	140
Major XSS types.....	140
Exam Focus.....	140
Request Forgeries (OBJ 4.5).....	141
CSRF abuse.....	141
SSRF abuse.....	141
Trust exploitation.....	141
Exam Focus.....	141
SQL Injections (OBJ 4.5).....	141
SQL query manipulation.....	141
Boolean-based bypass.....	141
Mitigation awareness.....	142
Exam Focus.....	142
Injection Attacks (OBJ 4.5).....	142
Command injection.....	142
Server-side template injection (SSTI).....	142
XML injection.....	142
Exam Focus.....	142
File Inclusions (OBJ 4.5).....	142
Remote file inclusion (RFI).....	143
Local file inclusion (LFI).....	143

RCE escalation.....	143
Exam Focus.....	143
Arbitrary Code Execution (OBJ 4.5).....	143
Execution outcome.....	143
Deserialization abuse.....	143
Web shell persistence.....	143
Exam Focus.....	143
Web Application Session Hijacking (OBJ 4.5).....	144
Session fixation.....	144
Session replay.....	144
XSS-based theft.....	144
Exam Focus.....	144
OWASP ZAP (OBJ 4.5).....	144
Proxy visibility.....	144
Automated and manual scanning.....	144
HUD and enumeration.....	144
Exam Focus.....	145
Section 22. Cloud Attacks.....	145
Cloud Attacks (OBJ 4.6).....	145
Exam Focus.....	145
Identity and Access Management (IAM) Misconfigurations (OBJ 4.6).....	145
IAM structure and purpose.....	145
Common IAM misconfigurations.....	145
Credential exploitation.....	146
Exam Focus.....	146
Resource Misconfigurations (OBJ 4.6).....	146
Network segmentation failures.....	146
Misconfigured network controls.....	146
Publicly exposed assets.....	146
Exam Focus.....	146

Logging Information Exposures (OBJ 4.6).....	146
Sensitive data in logs.....	146
Insecure log storage.....	147
Unencrypted log transmission.....	147
Exam Focus.....	147
Metadata Service Attacks (OBJ 4.6).....	147
Metadata service purpose.....	147
SSRF as an attack vector.....	147
Credential extraction and pivoting.....	147
Exam Focus.....	147
Image and Artifact Tampering (OBJ 4.6).....	147
VM image tampering.....	147
Container image tampering.....	148
Software artifact tampering.....	148
Exam Focus.....	148
Supply Chain Attacks (OBJ 4.6).....	148
Third-party dependency compromise.....	148
Software update pipeline abuse.....	148
Upstream provider risks.....	148
Exam Focus.....	148
Container Exploits and Attacks (OBJ 4.6).....	148
Workload runtime attacks.....	148
Container escapes.....	149
Container assessment tools.....	149
Exam Focus.....	149
Trust Relationship Abuse (OBJ 4.6).....	149
Lateral movement via trust.....	149
Kerberos and SPN abuse.....	149
Cloud trust abuse.....	149
Exam Focus.....	149

Third-party Integration Exploits (OBJ 4.6).....	149
Insecure APIs.....	149
Vulnerable libraries.....	150
Insecure webhooks.....	150
Exam Focus.....	150
Cloud Security Testing (OBJ 4.6).....	150
ScoutSuite auditing.....	150
Pacu exploitation.....	150
Prowler and native tools.....	150
Exam Focus.....	150
Conducting Cloud Audits (OBJ 4.6).....	150
Audit objectives.....	150
Configuration review.....	151
Offensive value.....	151
Exam Focus.....	151
Section 23. Attacking Specialized Systems.....	151
Attacking Specialized Systems (OBJ 4.9).....	151
Exam Focus.....	151
Mobile Device Attacks (OBJ 4.9).....	151
Root-level compromise.....	151
Application installation risks.....	152
Mobile security controls.....	152
Exam Focus.....	152
Tools for Mobile Device Attacks (OBJ 4.9).....	152
MobSF analysis.....	152
Frida and Objection.....	152
Drozer and ADB.....	153
Exam Focus.....	153
Bluetooth Attacks (OBJ 4.9).....	153
Bluejacking.....	153

Bluetooth spamming.....	153
Discoverable mode risks.....	153
Exam Focus.....	153
NFC and RFID Attacks (OBJ 4.9).....	154
RFID cloning.....	154
NFC exploitation.....	154
Wireless authentication risks.....	154
Exam Focus.....	154
AI Attacks (OBJ 4.9).....	154
Prompt injection.....	154
Model manipulation.....	154
Operational impact.....	155
Exam Focus.....	155
Operational Technology (OT) (OBJ 4.9).....	155
OT characteristics.....	155
ICS components.....	155
SCADA environments.....	155
Exam Focus.....	155
OT Attacks (OBJ 4.9).....	155
CAN bus attacks.....	155
Modbus exploitation.....	156
Advanced OT protocols.....	156
Exam Focus.....	156
Testing OT Systems (OBJ 4.9).....	156
Wireshark risks.....	156
Tcpdump limitations.....	156
Scapy dangers.....	156
OT-safe tools.....	156
Exam Focus.....	156
Section 24. Automated Attacks.....	157

Automated Attacks (OBJ 4.10).....	157
Exam Focus.....	157
Empire/PowerSploit (OBJ 4.10).....	157
Empire command-and-control automation.....	157
Listener and agent workflow.....	157
PowerSploit privilege escalation and execution.....	157
Exam Focus.....	158
PowerView (OBJ 4.10).....	158
Domain user and group enumeration.....	158
Computer and trust discovery.....	158
Session awareness for targeting.....	158
Exam Focus.....	158
PowerUpSQL (OBJ 4.10).....	158
SQL Server instance discovery.....	159
Configuration and credential weaknesses.....	159
Privilege escalation and lateral movement.....	159
Exam Focus.....	159
AD Search (OBJ 4.10).....	159
Active Directory cmdlet automation.....	159
Filtering for high-value targets.....	159
Reporting and OU-wide scripting.....	160
Exam Focus.....	160
Caldera (OBJ 4.10).....	160
Breach and attack simulation (BAS).....	160
Abilities and attack chain automation.....	160
Agents and reporting.....	160
Exam Focus.....	160
Infection Monkey (OBJ 4.10).....	161
Controlled malware-like simulation.....	161
Agent and Monkey Island architecture.....	161

Actionable security insights.....	161
Exam Focus.....	161
Atomic Red Team (OBJ 4.10).....	161
Atomic tests and ATT&CK mapping.....	161
PowerShell execution workflow.....	161
Detection gap identification.....	162
Exam Focus.....	162
Section 25. Persistence.....	162
Persistence (OBJ 5.1).....	162
Exam Focus.....	162
Command and Control (OBJ 5.1).....	162
Empire.....	163
Covenant.....	163
Mythic.....	163
Exam Focus.....	163
Automating Persistence (OBJ 5.1).....	163
Scheduled Tasks and Cron Jobs.....	164
Service Creation.....	164
Registry Run Keys.....	164
Exam Focus.....	164
Remote Shells (OBJ 5.1).....	164
Reverse Shells.....	165
Bind Shells.....	165
Why Shell Direction Matters.....	165
Exam Focus.....	165
Backdoors (OBJ 5.1).....	165
Trojans.....	165
Web Shells.....	166
Rootkits.....	166
Exam Focus.....	166

Account Credentials (OBJ 5.1).....	166
Credential Acquisition Methods.....	166
Account Creation on Windows and Unix-like Systems.....	167
Hiding and Blending Accounts.....	167
Exam Focus.....	167
Browser-Based Persistence (OBJ 5.1).....	167
Malicious Browser Extensions.....	167
Cookies and Local Storage Abuse.....	167
Browser Settings and Vulnerability Exploitation.....	168
Exam Focus.....	168
Security Control Tampering (OBJ 5.1).....	168
Disabling Firewalls and Endpoint Protections.....	168
Policy, Registry, and Logging Manipulation.....	168
Startup and Scheduled Execution as a Fallback Layer.....	169
Exam Focus.....	169
Section 26. Lateral Movement.....	169
Lateral Movement (OBJ 5.2).....	169
Exam Focus.....	169
Pivoting and Relaying (OBJ 5.2).....	170
Network Pivoting.....	170
Application-Level Pivoting.....	170
Authentication Relay Attacks.....	170
Exam Focus.....	170
Enumerating for Lateral Movement (OBJ 5.2).....	171
Service Discovery.....	171
Network Traffic Discovery.....	171
Credential Capturing and Dumping.....	171
Exam Focus.....	171
Service Discovery (OBJ 5.2).....	171
Server Message Block (SMB).....	171

Remote Procedure Call (RPC).....	172
Distributed Component Object Model (DCOM).....	172
Exam Focus.....	172
Protocol Discovery (OBJ 5.2).....	172
Telnet.....	172
FTP.....	172
LDAP.....	173
Exam Focus.....	173
Remote Access Discovery (OBJ 5.2).....	173
Remote Desktop Protocol (RDP).....	173
Virtual Network Computing (VNC).....	173
Secure Shell (SSH).....	173
Windows Management Instrumentation (WMI).....	174
Exam Focus.....	174
Printer Discovery (OBJ 5.2).....	174
Line Printer Daemon (LPD).....	174
JetDirect and SNMP.....	174
Exam Focus.....	174
Discovering Internal Websites (OBJ 5.2).....	175
Default Credentials and Weak Authentication.....	175
Outdated Software and Session Hijacking.....	175
API Abuse and Request Smuggling.....	175
Exam Focus.....	175
Living Off the Land Tools (OBJ 5.2).....	175
Network Discovery Tools.....	175
Code Execution Binaries.....	176
Reconnaissance Utilities.....	176
Exam Focus.....	176
Section 27. Exfiltration.....	176
Exfiltration (OBJ 5.3).....	176

Understanding Exfiltration in Post-Exploitation.....	176
Covert Channels as Exfiltration Mechanisms.....	177
Alternate Data Streams and Hidden Storage.....	177
Common Data Exfiltration Strategies.....	177
Exam Focus.....	177
Covert Channels (OBJ 5.3).....	177
Steganography as a Covert Channel.....	177
DNS Covert Channels.....	178
ICMP Covert Channels.....	178
HTTPS Covert Channels.....	178
Exam Focus.....	178
Alternate Data Streams (ADS) (OBJ 5.3).....	178
Understanding NTFS Alternate Data Streams.....	178
Hiding Collected Information in ADS.....	179
Exfiltrating Hidden Streams.....	179
Detection Considerations.....	179
Exam Focus.....	179
Exfiltrating Data (OBJ 5.3).....	179
Virtual Drive Mounting.....	179
Email-Based Exfiltration.....	179
Cross-Account Cloud Abuse.....	180
Cloud Storage and Public Text Sites.....	180
Exam Focus.....	180
Section 28. Cleanup and Restoration.....	180
Cleanup and Restoration (OBJ 5.4).....	180
Purpose of Cleanup and Restoration.....	180
Domain and Objective Alignment.....	180
Core Cleanup Activities.....	181
Exam Focus.....	181
Persistence Removal (OBJ 5.4).....	181

Scheduled Tasks and Startup Automation.....	181
Registry-Based Persistence.....	181
Hidden Files and Unauthorized Accounts.....	181
Remote Access Tools and Backdoors.....	181
Exam Focus.....	181
Revert Configuration Changes (OBJ 5.4).....	182
Network and Firewall Rules.....	182
System and Application Security Settings.....	182
Database, Logging, and DNS Settings.....	182
Exam Focus.....	182
Created Credentials Removal (OBJ 5.4).....	182
User Account Removal.....	182
Authentication Authority Awareness.....	182
Revoking API Keys and Tokens.....	182
Exam Focus.....	183
Removal of Testing Tools (OBJ 5.4).....	183
In-Memory vs Persistent Tools.....	183
Secure Deletion of Artifacts.....	183
Removal of Automation Tasks.....	183
Exam Focus.....	183
Decommission Testing Infrastructure (OBJ 5.4).....	183
Identifying Created Resources.....	183
Terminating Compute and Storage.....	183
Networking and Log Cleanup.....	184
Exam Focus.....	184
Artifact Preservation (OBJ 5.4).....	184
Types of Artifacts.....	184
Secure Storage and Documentation.....	184
Handling Sensitive Data.....	184
Exam Focus.....	184

Secure Data Destruction (OBJ 5.4).....	184
Why Deletion Is Insufficient.....	184
Shredding and Overwriting.....	185
Storage Type Considerations.....	185
Exam Focus.....	185
Section 29. Remediation Recommendations.....	185
Remediation Recommendations (OBJ 1.5).....	185
Exam Focus.....	185
System Hardening (OBJ 1.5).....	185
System Hardening Fundamentals.....	186
Configuration and Access Controls.....	186
Patch Management with Risk Handling.....	186
Exam Focus.....	186
User Input Sanitization (OBJ 1.5).....	186
Trust Boundaries and Input Validation.....	186
Root-Cause Injection Remediation.....	187
Effective Remediation Communication.....	187
Exam Focus.....	187
Network and Infrastructure Controls (OBJ 1.5).....	187
Network Segmentation and Isolation.....	187
Infrastructure Security Controls.....	187
Clear Remediation Communication.....	187
Exam Focus.....	187
Authentication Recommendations (OBJ 1.5).....	188
Secrets Management.....	188
Multifactor Authentication (MFA).....	188
Cryptographic Trust Management.....	188
Effective Remediation Communication.....	188
Exam Focus.....	188
Encryption Recommendations (OBJ 1.5).....	188

Encryption Gaps and Risk.....	189
Strong Algorithms and Deployment.....	189
Key Management and Enforcement.....	189
Exam Focus.....	189
Patch Management Recommendations (OBJ 1.5).....	189
Patch Management Fundamentals.....	189
Operational Constraints and Risk Handling.....	189
Compensating Controls and Documentation.....	190
Exam Focus.....	190
Process Level Remediation (OBJ 1.5).....	190
Process-Level Fundamentals.....	190
Common Operational Weaknesses.....	190
Effective Remediation Communication.....	190
Exam Focus.....	190
Administrative Controls (OBJ 1.5).....	191
Role-Based Access Control.....	191
Password Policies and Credential Practices.....	191
Policies and Secure SDLC.....	191
Exam Focus.....	191
Physical Controls (OBJ 1.5).....	191
Physical Access Controls.....	191
Protection of Facilities and Assets.....	192
Monitoring and Awareness.....	192
Exam Focus.....	192
Operational Controls and Policies (OBJ 1.5).....	192
Security Awareness and Training.....	192
Personnel-Based Fraud Prevention.....	192
Time-Based Access Restrictions.....	192
Exam Focus.....	192
Section 30. Penetration Test Reporting.....	193

Penetration Test Reporting (OBJ 1.4).....	193
Purpose and Value of Reporting.....	193
Engagement Management Objectives.....	193
Reporting Lifecycle Overview.....	193
Exam Focus.....	193
Executive Summary Process (OBJ 1.4).....	193
Defining Purpose and Scope.....	194
Summarizing Methods and Key Findings.....	194
Overall Security Posture and Conclusion.....	194
Exam Focus.....	194
Root Cause Analysis (OBJ 1.4).....	194
Purpose and Value.....	194
Common Organizational and Technical Causes.....	194
Linking Findings to Causes.....	195
Exam Focus.....	195
Report Components (OBJ 1.2, 1.4).....	195
Core Report Sections.....	195
Technical Findings and Attack Narratives.....	195
Recommendations and Professional Presentation.....	195
Exam Focus.....	195
Risk Scoring and Prioritization (OBJ 1.4).....	195
Risk Scoring Fundamentals.....	196
Articulating Severity and Impact.....	196
Prioritization for Remediation.....	196
Exam Focus.....	196
Definitions in the Report (OBJ 1.4).....	196
Purpose of Definitions.....	196
Common Terms Explained.....	196
Exam Focus.....	196
Limits and Assumptions (OBJ 1.2, 1.4).....	197

Penetration Test Limitations.....	197
Constraints and Assumptions.....	197
Exam Focus.....	197
Special Considerations (OBJ 1.2, 1.4).....	197
Reporting and Secure Handling.....	197
Peer Review, Acceptance, and Retesting.....	197
Exam Focus.....	197
Section 31. Conclusion.....	198
Conclusion.....	198
Review of the Five Exam Domains.....	198
Navigating the Exam Objectives.....	198
Scheduling and Taking the Exam.....	199
Five Exam Day Success Strategies.....	199
Using Practice Exams Effectively.....	199
BONUS: What Comes Next?.....	200
Continuing Certification Paths.....	200
Website Course Benefits.....	200
Exam Voucher and Take-Two Options.....	200
Staying Connected and Advancing.....	200

Section 1. Introduction

3 Lessons | Does not include the instructor, FAQ, or walkthrough lessons.

Introduction

PenTest+ is an intermediate-level certification focused on real-world penetration testing and vulnerability assessment across on-premises, cloud, and hybrid environments.

Purpose of the PenTest+ Certification

The certification validates intermediate-level skills across the full lifecycle of a penetration test. This includes planning and scoping engagements, performing reconnaissance and enumeration, identifying and analyzing vulnerabilities, executing attacks and exploits, and finally reporting findings to stakeholders. The certification also introduces common penetration testing tools and foundational code analysis skills, which are important in modern, real-world engagements.

Expected Background and Prerequisites

The PenTest+ certification is designed for professionals with several years of hands-on security experience. Specifically, CompTIA recommends three to four years of broad information security or related technical experience. Security+ and CySA+ certifications are recommended prior to attempting PenTest+, but this is not a strict requirement. However, Security+ knowledge is assumed throughout this course, and foundational security concepts will not be re-taught in depth.

PenTest+ Exam Domains Overview

The PenTest+ exam is organized into five domains that represent different phases and responsibilities within a penetration test. These are:

- **Engagement Management**, which focuses on planning, scoping, regulatory considerations, and reporting.
- **Reconnaissance and Enumeration**, which covers information gathering techniques, both passive and active.
- **Vulnerability Discovery and Analysis**, which examines how weaknesses are identified and evaluated across systems and applications.
- **Attacks and Exploits**, the largest domain, which emphasizes executing attacks using appropriate tools and techniques.

- **Post-Exploitation and Lateral Movement**, which focuses on actions taken after initial access to expand control within an environment.

Course Structure and Objective Mapping

This course content does not follow the exam objectives in strict numerical order. Instead, objectives are presented in a logical, real-world sequence that mirrors how penetration tests are actually performed. This means that objectives related to reporting, for example, appear later in the course even though they are listed in Domain One objectives. Additionally, each section and lesson clearly identifies the domain and objective being covered to support targeted exam review.

Exam Question Style and Difficulty

PenTest+ exam questions frequently combine multiple objectives across different domains into a single scenario. These multi-layered questions require deeper analysis than entry-level certifications and often involve selecting tools, interpreting outputs, and applying techniques together. This course is structured so that concepts build upon one another to help reinforce these connections.

Exam Logistics and Scoring

The PenTest+ exam includes between 65 and 85 questions completed over 165 minutes. Most questions are multiple choice, with some multiple-select questions and several performance-based simulations. These simulations require performing tasks in a simulated technical environment. Overall, a passing score is 750 out of 900.

Study Tips and Student Resources

The associated video course includes human-generated closed captions and searchable transcripts to support different learning styles. Playback speed can also be adjusted to match individual preferences. Next, students are encouraged to use the provided study guide alongside the videos to take notes and reinforce learning. And finally, support is available through Dion Training's Facebook and Discord groups, as well as direct email support for additional assistance.

Exam Tips

CompTIA PenTest+ exam tips are designed to help you study more effectively throughout the course and reduce frustration on exam day by aligning your thinking with how CompTIA writes and evaluates exam questions. Exam tips include:

Understanding How Exam Questions Are Written

The PenTest+ exam does not use trick questions. Each question is carefully worded to test your understanding of the material presented in the course and the assumed background knowledge. Because of this, it is important to slow down and read each question carefully, even if it seems straightforward at first glance. Re-reading a question helps ensure you are answering exactly what is being asked, not what you assume is being asked.

Identifying Distractors and Keywords

Most exam questions include one or more distractor answers. These are choices that appear plausible but are intentionally designed to pull you away from the best answer. Eliminating obvious distractors significantly improves your chances of selecting the correct response. Additionally, CompTIA emphasizes important words by using formatting such as bold text, italics, or capitalization. In that effort, keywords like ALL, NOT, BEST, SELECT TWO, or SELECT THREE are deliberate signals from the test writers and must be carefully considered before choosing an answer.

Answering from the CompTIA Perspective

When answering questions, rely on the terminology, tools, and methodologies taught in this course rather than personal work, school, or independent research experience. This is because different organizations often use different approaches, but CompTIA exam questions are written strictly from CompTIA's perspective. So, when you are unsure, choose the answer that aligns most closely with the course material.

Choosing the Best Answer

PenTest+ questions often include multiple answers that may be technically correct in certain situations. Your task is to identify the best answer, meaning the one that applies most often and in the widest range of scenarios. So, avoid selecting answers based on rare or fringe cases. Instead, choose the option that represents the most common and practical solution.

Avoiding the Urge to Fight the Exam

Fighting the exam occurs when you try to disprove the most correct answer by inventing edge cases or overly complex scenarios. This approach often leads to incorrect answers. So, rather than challenging the question, focus on identifying the key concept being tested. And once you recognize that concept, select the answer that directly corresponds to it.

Recognizing Concepts and Tools

You are not required to memorize definitions word for word. Instead, you should be able to recognize concepts and terms when they appear in answer choices. Since the exam uses multiple-choice and multiple-select formats, familiarity and recognition are more important than rote memorization. When tools are referenced, you should understand their general purpose, basic usage, and the situations in which they are most appropriate.

By applying these exam strategies consistently throughout the course, you will improve both your confidence and performance because these techniques help you focus on understanding core concepts and apply them effectively under exam conditions.

100% Pass Guarantee

Dion Training's 100% Pass Guarantee places the risk on Dion Training, not you. So, the goal of the course is to teach you what you need to know to pass the PenTest+ exam, but success ultimately requires your effort and commitment. This course provides structured instruction, practice, and reinforcement to prepare you effectively for the certification exam.

Course Components and Learning Structure

The course is designed as a complete learning system that includes video lessons, this downloadable study guide, checkpoint quizzes, and full-length practice exams. These components work together to reinforce concepts and build exam readiness. This material has been used successfully by many students to earn their PenTest+ certification.

Checkpoint Quizzes and Progress Requirements

Each section of the course includes checkpoint quizzes to measure your understanding as you progress. A score of at least 75% is required for a quiz to be marked as complete. If you do not achieve a passing score on your first attempt, you are encouraged to review the lessons and retake the quiz. Here, the questions remain the same on each attempt, emphasizing mastery of the material rather than memorization.

Practice Exams as a Learning Tool

At the end of the course, you will complete multiple practice exams. Like the quizzes, a score of 75% is required to pass each practice exam. After completing an exam, you receive detailed explanations for every question, including why the correct answer is the best choice. Reviewing both correct and incorrect answers helps you understand the logic CompTIA uses when designing exam questions.

Using Practice Exams Effectively

Practice exams are not meant to be memorized. Instead, they should be used to identify weak areas, reinforce key concepts, and improve your ability to analyze questions. Incorrect answers are part of the learning process and provide opportunities to deepen your understanding of penetration testing concepts and exam strategy.

Completion, Support, and Certification

You are encouraged to complete the course in order by watching all lessons, passing all quizzes, and completing all practice exams. If you encounter errors in the material, technical issues, or have questions about the content, support is available through email. Once you reach 100% course completion, you receive a Dion Training certificate of completion. This certificate makes you eligible for the 60-day 100% Pass Guarantee, which should be obtained before scheduling your exam.

Section 2. PenTesting Overview

6 Lessons | Does not include demonstration lessons.

PenTesting Overview

This lesson provides a high-level overview of penetration testing, covering the full lifecycle of a penetration test from initial planning through final reporting. The goal of this section is to give you a realistic, end-to-end understanding of how penetration tests are conducted so you can better understand how individual techniques and tools fit together later in the course.

Ethical and Professional Use of Penetration Testing Skills

It is important to understand that the techniques and tools taught in this course are intended only for ethical, authorized use. Ethical penetration testers and malicious attackers may use similar methods, but they are fundamentally separated by intent, authorization, and professional integrity. As penetration testers, all of your activities must be conducted legally, with explicit permission, and in alignment with professional and ethical standards.

What Penetration Testing Is and Why It Matters

Penetration testing, often referred to as pen testing, is the practice of simulating real-world cyberattacks against systems, networks, or web applications to identify and exploit vulnerabilities. The purpose of a penetration test is not simply to break into systems, but to help organizations understand their weaknesses so they can fix them. Effective penetration testing improves an organization's overall security posture and supports compliance with industry standards and regulatory requirements.

Purpose of This Section

This section focuses on presenting the complete penetration testing process at a high level. Rather than concentrating on individual tools or objectives, it provides the broad context needed to understand how each phase connects to the next. This foundation will make the more detailed technical lessons later in the course easier to follow and apply. The overall sections of a penetration test that will be discussed are: Planning the Engagement, Information Gathering, Attacks and Exploits, Post-Exploitation, and Reporting.

Planning the Engagement

The penetration testing process begins with planning the engagement. During this phase, the scope, objectives, and rules of engagement are defined. This ensures that all parties clearly understand what systems can be tested, what techniques are allowed, and what boundaries must not be crossed. Planning concludes only after formal authorization to conduct the test is granted by the client.

Information Gathering

Information gathering involves collecting details about the target environment using both passive and active techniques. This phase helps identify potential attack paths and weaknesses. Common approaches include open-source intelligence gathering, network enumeration, and social engineering. The quality of information gathered here directly impacts the effectiveness of later phases.

Attacks and Exploits

During the attacks and exploits phase, the penetration tester attempts to exploit vulnerabilities discovered during information gathering. This may involve techniques such as social engineering attacks, system-level exploitation, or web application attacks. The objective here is to achieve initial access in a controlled and documented manner.

Post-Exploitation

Post-exploitation focuses on actions taken after successful compromise of a system. This includes maintaining access, escalating privileges, moving laterally to other systems, and staging data for potential exfiltration. These activities help demonstrate the real-world impact of vulnerabilities and the potential risk to the organization.

Reporting

Reporting is the final and one of the most important phases of a penetration test. This phase involves documenting findings, methodologies used, successful and unsuccessful exploits, and providing clear recommendations for remediation. This report communicates risk to stakeholders and provides actionable guidance to improve the organization's security posture.

Planning the Engagement

The planning the engagement phase lays the foundation for the entire test and ensures that both the client and the penetration tester share a clear understanding of

expectations, boundaries, and responsibilities. Proper planning also ensures that all legal, regulatory, and ethical considerations are addressed before any testing begins.

Importance of the Planning Phase

Planning is a critical phase because mistakes made here can impact every stage that follows. A well-planned engagement prevents misunderstandings, reduces risk to the client, and protects the penetration tester legally and professionally. This phase aligns technical goals with business realities so testing can be conducted safely and effectively.

Defining the Scope

One of the first steps in planning an engagement is defining the scope. The scope clearly identifies what systems, networks, applications, or environments are included in the penetration test, as well as what is explicitly excluded. For example, a tester may be authorized to assess a company's public-facing web application but restricted from testing internal networks or email systems. Overall, clear scoping prevents accidental overreach and helps avoid conflicts during the engagement.

Legal and Regulatory Considerations

Penetration tests must comply with applicable laws and regulations based on the organization's industry and geographic location. Different environments may impose strict requirements on how data is accessed, handled, and protected. For instance, testing a healthcare organization in the United States requires compliance with HIPAA regulations, which mandate the protection of patient data throughout the engagement.

Rules of Engagement

Rules of Engagement, often referred to as ROE, define the specific dos and don'ts of the penetration test. These rules may restrict certain attack types, such as distributed denial-of-service attacks, to avoid unintended business disruptions. ROE can also specify acceptable testing windows, such as limiting activities to non-peak business hours.

Non-Disclosure Agreements

A non-disclosure agreement, or NDA, is commonly established during the planning phase to protect sensitive information. This means that any confidential data discovered during testing must remain private and cannot be shared with third parties. For example, if a tester uncovers legacy and unpatched systems critical to operations, that information must remain confidential under the NDA.

Statement of Work

The statement of work, or SOW, defines the specific services the penetration tester will deliver. This document outlines tasks, timelines, milestones, and payment terms. It may include details such as performing vulnerability assessments, attempting exploitation, and delivering a final report within a defined timeframe. In some cases, the rules of engagement are included directly within the SOW.

Communication and Escalation Paths

Clear communication channels and escalation paths must be established before testing begins. Both the client and the penetration testing team should know who to contact if issues arise. For example, if a vulnerability is discovered that poses an immediate risk of data loss, predefined escalation contacts allow the issue to be addressed quickly.

Risk Tolerance and Business Impact

Understanding the client's risk tolerance is another key aspect of planning. This involves discussing how aggressive the testing can be and what level of potential disruption is acceptable. Some organizations may allow more rigorous testing even if minor disruptions occur, while others require a conservative approach to protect production systems.

Authorization to Proceed

At the conclusion of the planning phase, a formal authorization to proceed is issued and signed by the network owner. This document identifies the authorized testers and confirms approval of the testing plan. Authorization to attack is the legal foundation that separates ethical penetration testing from unauthorized access.

Overall, effective engagement planning includes defining scope, meeting legal requirements, establishing rules of engagement, securing NDAs, documenting the statement of work, setting communication paths, understanding risk tolerance, and obtaining formal authorization. Together, these steps ensure a professional and successful penetration testing engagement.

Information Gathering

This lesson covers the information gathering phase of a penetration test, also known as reconnaissance. During this phase, the penetration tester collects as much information as possible about the target environment before attempting any attacks. Effective information gathering helps identify potential attack paths and shapes the strategy for later phases of the engagement.

Information gathering is typically divided into two categories: passive reconnaissance and active reconnaissance. Both approaches aim to build an understanding of the target, but they differ in how directly they interact with the target's systems. Balancing these techniques allows a tester to gather useful data while managing the risk of detection.

Passive Reconnaissance

Passive reconnaissance involves collecting information without directly interacting with the target's network or systems. Because no traffic is sent to the target, these activities are difficult to detect. Here, passive techniques rely heavily on publicly available data sources and open-source intelligence, commonly referred to as OSINT.

Examples of passive reconnaissance include reviewing public records, searching online databases, and analyzing information found on social media, corporate websites, and public forums. Tools such as Maltego and Recon-ng are commonly used during this phase:

- Maltego provides graphical, relationship-based visualizations of entities like people, domains, email addresses, and IP addresses.
- Recon-ng is a command-line framework that automates OSINT collection and produces structured, exportable results.

Additional passive techniques include performing WHOIS lookups to gather domain registration details such as ownership information, expiration dates, and nameservers. IP address range identification using regional internet registries like ARIN can reveal network blocks owned by the target organization. Also, reviewing job postings may also disclose valuable insight into the technologies, platforms, and security tools used by the organization.

Active Reconnaissance

Active reconnaissance involves direct interaction with the target's systems to gather more detailed and accurate information. While this approach is more intrusive and carries a higher risk of detection, it provides deeper insight into the target's attack surface.

Common active reconnaissance activities include network scanning, service probing, and resource enumeration. Here, tools such as nmap are used to identify open ports, running services, and service versions. This information helps determine which systems may be vulnerable to exploitation.

Banner grabbing is another active technique where requests are sent to services and the responses are analyzed to identify software and version information. For example, connecting to a web server and examining HTTP headers with a tool like cURL can reveal the web server software and version, which may help identify known vulnerabilities.

Next, DNS enumeration is a valuable active reconnaissance method. By querying DNS servers using tools like nslookup or dig, a tester can discover subdomains, mail servers, and other DNS records. This helps map the target's infrastructure and uncover additional potential entry points.

Finally, Shodan is another important reconnaissance tool, often described as a search engine for internet-connected devices. Shodan indexes exposed services and devices across the internet, making it useful for identifying publicly accessible systems that belong to a target organization.

Documentation and Stealth Considerations

Throughout the information gathering phase, it is essential to document all findings carefully. This includes IP addresses, domain names, service versions, operating system details, and any other relevant information. Additionally, thorough documentation ensures the engagement remains organized, auditable, and repeatable.

Maintaining a balance between thoroughness and stealth is also important. While collecting detailed information is valuable, excessive or aggressive activity can alert the target. So, adjusting scan timing, limiting aggressiveness, and using anonymization techniques can help reduce the likelihood of detection.

Overall, information gathering is a critical early phase of a penetration test. By combining passive reconnaissance for stealth and active reconnaissance for detail, and by carefully documenting findings while minimizing detection, a penetration tester can plan more effective and targeted attack strategies.

Attacks and Exploits

This lesson covers the attacks and exploits phase of a penetration test. During this phase, the penetration tester actively attempts to exploit vulnerabilities identified during information gathering. The primary objective at this point is to gain an initial foothold in the target environment and demonstrate the real-world impact of discovered weaknesses.

Purpose of the Attacks and Exploits Phase

The attacks and exploits phase moves from theory into action. So, instead of just identifying potential weaknesses, the tester attempts to exploit them in a controlled and authorized manner. Successfully exploiting a vulnerability provides proof that an attacker could compromise systems or data, which helps communicate risk clearly to the client.

Target Prioritization

Before launching attacks, targets should be prioritized based on the results of the information gathering phase. High-value assets and critical vulnerabilities should be addressed first. For example, an outdated web server with known exploits presents a higher risk and should take priority over lower-impact findings.

Using Exploitation Frameworks

One of the most common approaches during this phase is using automated exploitation frameworks such as the Metasploit Framework. Metasploit allows penetration testers to develop and execute exploit code against remote targets and includes modules for exploiting known vulnerabilities. For instance, if a target is running an outdated version of a web server, Metasploit may provide a ready-made exploit to gain access.

Password Attacks

Password-based attacks are another common technique used during exploitation. These include brute force attacks, which attempt every possible password combination, and dictionary attacks, which rely on lists of commonly used passwords. Tools such as Hydra and John the Ripper automate these processes. For example, password hashes collected during reconnaissance can be attacked using John the Ripper to recover valid credentials.

Phishing Attacks

Phishing is a social engineering attack designed to trick users into revealing sensitive information. During a penetration test, this may involve sending emails that appear legitimate and directing users to a fake login page. Then, if users enter their credentials, the tester captures them to demonstrate how easily accounts could be compromised.

Web Application Attacks

Web applications are frequent targets during the attacks and exploits phase. One common technique here is SQL injection, where malicious SQL statements are inserted into input fields to manipulate backend databases. If successful, this can allow access

to sensitive data such as usernames and passwords. Here, tools like sqlmap can automate the discovery and exploitation of SQL injection vulnerabilities.

Another web-based attack is cross-site scripting, or XSS. XSS attacks involve injecting malicious scripts into web pages that execute in a victim's browser. These scripts can be used to steal session cookies or other sensitive information. For example, a vulnerable forum post could be used to capture session data from users who simply view the page.

Documentation and Evidence Collection

Throughout the attacks and exploits phase, it is essential to document all actions and results. This includes recording the methods used, the systems targeted, and the outcome of each attempt. Detailed documentation supports reporting, allows for repeatability, and ensures any changes made during testing can be reversed at the conclusion of the engagement.

Overall, the attacks and exploits phase focuses on exploiting high-priority vulnerabilities to gain access and demonstrate real-world risk. By carefully selecting targets, using appropriate tools and techniques, and documenting every step, penetration testers provide clear, actionable evidence that supports remediation efforts.

Post-Exploitation

This lesson covers the post-exploitation phase of a penetration test. Post-exploitation begins after initial access has been successfully obtained and focuses on understanding the depth of access, maintaining control of compromised systems, and expanding access within the target environment to demonstrate real-world impact.

Purpose of the Post-Exploitation Phase

The goal of post-exploitation is to determine what an attacker can realistically achieve after gaining access. This phase moves beyond simply proving a vulnerability exists and instead demonstrates how that access could be maintained, expanded, and used to access sensitive systems or data.

Establishing Persistence

Establishing persistence ensures continued access to a compromised system even if it is rebooted or the original exploit is patched. One common persistence technique is creating a new user account with elevated privileges. For example, after compromising a Windows server, a tester might create a new administrative user to maintain access.

Another persistence method involves task schedulers. On Windows systems, scheduled tasks can be created, while Linux systems use cron jobs for the same purpose. Either of these can be configured to automatically execute scripts or commands, such as launching a reverse shell at regular intervals, allowing access to be re-established if it is lost.

Privilege Escalation

After persistence is established, the next step is privilege escalation. If initial access was obtained with limited user rights, the tester attempts to gain administrative or root-level privileges. This can be achieved by exploiting local vulnerabilities, cracking passwords, or extracting credentials from memory.

On Windows systems, tools like Mimikatz are commonly used during this phase. Mimikatz can extract credentials, password hashes, PINs, and Kerberos tickets from memory, which can then be leveraged to elevate privileges or access additional systems.

Lateral Movement

Once elevated privileges are obtained, **lateral movement allows the tester to access other systems within the same network**. This involves moving from one compromised host to others to discover additional resources, credentials, or higher levels of access. For example, a compromised workstation might be used to execute commands on other systems.

Here, tools such as PsExec on Windows or SSH on Linux are commonly used to facilitate lateral movement. Successfully moving laterally expands the scope of the compromise and demonstrates how a single weakness can impact an entire network.

Pivoting to Other Network Segments

Pivoting is used to access additional network segments are not directly accessible from the initial point of compromise. By routing traffic through a compromised system, the tester can reach isolated systems that would otherwise be unreachable. This effectively turns the compromised host into a relay or proxy for further attacks.

Common pivoting techniques include using Metasploit pivoting modules, ProxyChains, or SSH dynamic port forwarding. Pivoting extends the attack surface and allows deeper exploration of the target's infrastructure.

Actions on Objectives and Data Exfiltration

During post-exploitation, testers gather sensitive information such as documents, databases, and emails. Once collected, this data may be exfiltrated from the

environment using methods designed to avoid detection. These activities are often referred to as “actions on objectives”. For example, sensitive data might be exfiltrated using DNS tunneling to bypass traditional monitoring controls.

The goal here is to demonstrate how valuable data could be removed from the target environment without triggering alerts.

Cleanup and Covering Tracks

The final step of post-exploitation is cleanup. This involves removing tools that were uploaded, clearing logs where possible, and undoing unnecessary changes that could reveal the tester’s presence. Proper cleanup helps avoid detection and preserves access if continued testing is required.

Overall, post-exploitation focuses on maintaining access, escalating privileges, expanding reach through lateral movement and pivoting, collecting valuable data, and cleaning up evidence. These activities help demonstrate the full impact of a successful compromise and provide meaningful insight for remediation.

Reporting

The reporting phase is the final stage of a penetration test and is where the results of the engagement are formally communicated to the client. A well-written report demonstrates the value of the penetration test, explains the risks discovered, and provides clear guidance on how the client can improve their security posture. This phase translates technical testing activities into information that both technical and non-technical stakeholders can understand and act upon.

Purpose of Penetration Test Reporting

The goal of penetration test reporting is to clearly document what was tested, what was discovered, and why it matters. The report serves as an official record of the engagement and provides evidence of vulnerabilities, successful exploits, and security weaknesses. It also allows the client to prioritize remediation efforts and track improvements over time.

Executive Summary

The report should begin with an executive summary written for executives, managers, and other stakeholders who may not have a technical background. This section provides a high-level overview of the organization’s overall security posture, highlights critical or high-risk vulnerabilities, and explains the potential business impact of those issues. The executive summary should avoid technical jargon and focus on risk, impact, and outcomes rather than tools or exploitation details.

Methodology, Limitations, and Assumptions

Following the executive summary, the report should describe the methodology used during the penetration test. This section explains what was tested, how it was tested, and which tools or techniques were used. Different testing activities, such as network testing, web application testing, vulnerability scanning, social engineering, and hands-on exploitation, should be clearly described.

It is also important to document any limitations and assumptions that affected the engagement. These may include systems that were out of scope, restrictions on testing hours, time constraints, or environmental factors that influenced testing. Finally, assumptions, such as the use of provided credentials or default configurations, should also be clearly stated so the client understands the context of the findings.

Detailed Findings

The detailed findings section forms the core of the penetration test report. Each finding should follow a consistent structure that includes a description of the vulnerability, how it was discovered, the potential impact, and the evidence collected. Supporting artifacts such as screenshots, logs, or exploit output should be included to validate each finding.

Where appropriate, the report should include enough detail for the client to reproduce and validate the vulnerability independently. For example, a SQL injection finding should document the affected application component, the type of database involved, and proof that the exploit was successful.

Risk Prioritization

Findings should be prioritized using a recognized risk-rating framework such as the Common Vulnerability Scoring System (CVSS). Severity ratings help classify vulnerabilities as critical, high, medium, or low and allow the client to focus remediation efforts on the most impactful issues first. Risk ratings should be adjusted based on the client's environment, threat model, and business priorities.

Attack Narrative

The attack narrative explains how individual vulnerabilities were chained together during the penetration test. This section provides a step-by-step account of how initial access was obtained, how privileges were escalated, how lateral movement or pivoting occurred, and how actions on objectives were achieved. By showing how vulnerabilities work together, the attack narrative helps the client understand the real-world impact of the findings.

Recommendations and Remediation

The report should conclude with clear and actionable remediation recommendations. Each recommendation should directly address a specific finding and provide practical guidance, such as applying patches, upgrading software, changing configurations, or improving development and security practices.

In addition to individual fixes, the report should also include broader security recommendations based on patterns observed during the engagement. These may include improving network segmentation, implementing incident response planning, increasing user security awareness, or enhancing continuous monitoring.

Overall, the reporting phase is where the results of a penetration test are clearly communicated to the client in a structured and actionable way. A strong report begins with an executive summary for non-technical stakeholders, followed by a transparent methodology that explains what was tested and under what conditions. Each vulnerability is documented with supporting evidence and prioritized using CVSS-based severity ratings, while the attack narrative demonstrates how issues were chained together to achieve real-world impact. Finally, clear remediation recommendations and a call to action help the client address critical risks, plan follow-up testing, and continuously improve their security posture.

Section 3. Pre-Engagement Activities

8 Lessons | Exam Objectives Covered: 1.1, 1.2 | Does not include demonstrations

Pre-Engagement Activities (OBJ 1.1, 1.2)

Key Concepts

- Pre-engagement activities establish alignment, legality, and scope before testing begins.
- Engagement management focuses on regulations, stakeholder alignment, and assessment planning.

Engagement Planning and Objectives

Pre-engagement activities and purpose:

- Establish the foundation for alignment, legality, and successful execution of the penetration test.
- Objectives alignment: 1.1 emphasizes summarizing pre-engagement activities and building a solid foundation before testing, and 1.2 emphasizes collaboration and communication among testers, clients, and stakeholders.

Agreements, Communication, and Risk Management

- Agreements and communication requirements, include NDAs, MSAs, SOWs, ToS, authorization letters, and communication responsibilities.
- Planning constraints and risk controls include escalation paths, exclusions, escalation processes, testing windows, and risk to the tester.

Scope and Target Definition

- Scope and target identification includes identifying IP address ranges, domains, and URLs to accurately define test scope.

Exam Focus

- Know the difference between technical testing tasks and pre-engagement planning.
- Pre-engagement activities appear frequently in scenario-based questions.

Regulations and Standards (OBJ 1.1, 1.2)

Key Concepts

- Regulations are legally binding requirements for data protection and privacy.

- Standards provide best practices and benchmarks but are not laws.

Regulatory Requirements

- Regulations are legally binding mandates that require strict controls for data protection.
- GDPR applies to EU businesses and organizations processing EU citizen data, including requirements for consent, impact assessments, and Data Protection Officer (DPO) roles.
- GLBA is a U.S. financial privacy law requiring safeguards like encryption, access controls, and ongoing security assessments.
- HIPAA is a U.S. healthcare law requiring physical, administrative, and technical safeguards, including encryption and audits.

Industry Standards and Frameworks

- Standards are not legally binding, but define best practices and due diligence expectations.
- PCI DSS protects cardholder data with requirements like secure networks, strong access controls, and monitoring and testing.
- The ISO/IEC 27000 series is a framework for ISMS, with ISO/IEC 27001 defining ISMS requirements and ISO/IEC 27002 providing control guidance.

Stakeholder Alignment

- Stakeholder alignment ensures shared understanding of purpose, scope, and outcomes across technical and executive teams.

Exam Focus

- Know which industries each regulation applies to.
- Understand the difference between laws and standards.

Types of Assessments (OBJ 1.1)

Key Concepts

- Assessments target specific technologies to identify vulnerabilities.
- Each assessment type focuses on a distinct attack surface.
- Infrastructure and Network Assessments

Infrastructure and Network Assessments

Network assessments:

- Evaluate network infrastructure security, including topology, firewall rules, and security policies, with focus on routing and switching devices.

Wireless assessments:

- Assess wireless security risks such as weak encryption and rogue access points using simulated attacks.

Application and Platform Assessments

Application Assessments:

- Review application code, dependencies, and configuration to identify exploitable vulnerabilities and support secure development lifecycle.

Mobile assessments:

- Evaluate mobile apps and platforms for issues like data leakage, improper session handling, and insecure storage.

Web assessments:

- Scrutinize web apps and sites for vulnerabilities such as SQL injection, XSS, and misconfigurations.

Cloud and Integration Assessments

Cloud assessments:

- Evaluate cloud infrastructure and resource configurations for misconfigurations and compliance issues, requiring platform-specific expertise.

API assessments:

- Tests APIs for weaknesses in authentication, authorization, and data validation used for integration and operational functionality.

Exam Focus

- Match assessment types to technologies in exam scenarios.
- Understand which assessment applies to cloud versus on-prem systems.

Types of Agreements (OBJ 1.1)

Key Concepts

- Agreements define legal, operational, and confidentiality expectations.
- Each document serves a distinct role in engagement management.

Confidentiality and Legal Protection

- Non-Disclosure Agreement (NDA) - Legally binding confidentiality contract protecting sensitive findings and potentially proprietary tools or methods for both parties.

Business Relationship Management

- Master Service Agreement (MSA) - Establishes foundational long-term business terms like payment framework, confidentiality, intellectual property, and liability, without defining project scope.

Engagement Scope and Execution

- Statement of Work (SOW) - Defines a specific engagement's scope, objectives, deliverables, timelines, responsibilities, and testing methods.

Service Usage Conditions

- Terms of Service (ToS) - Defines rules and conditions for service use, including client rights to deliverables and restrictions like prohibiting reverse-engineering tools.

Exam Focus

- Know which document defines scope versus long-term terms.
- Expect questions distinguishing NDA, MSA, and SOW.

Legal and Ethical Considerations (OBJ 1.1, 1.2)

Key Concepts

- Authorization is the legal foundation of penetration testing.
- Ethical testing minimizes harm and legal risk.

Authorization and Legal Approval

- Authorization letters legally grant permission to attack systems and define scope, personnel, timing, included assets, data handling, and reporting guidelines.
- Authorization authority and legality must be signed by the system or network owner with authority, otherwise testing is illegal for both technical and physical testing.

Disclosure and Professional Risk

- Mandatory reporting - Findings such as potential breaches are reported to the client and legal counsel to determine any required external disclosure.

- Risk to the professional - Unauthorized access can trigger civil or criminal liability under laws like CFAA. Unclear authorization increases personal risk.

Incident Handling and Escalation

- An escalation path defines chain of command, points of contact, and procedures for incidents like instability or interference with production operations.

Exam Focus

- Testing without authorization is illegal.
- Escalation paths are required for unexpected disruptions.

Lesson: Rules of Engagement (OBJ 1.1, 1.2)

Key Concepts

- Rules of Engagement define how testing is conducted.
- ROE protects business operations during testing.

ROE Structure and Purpose

- ROE purpose and placement - Formal guidelines typically included in the SOW to define process, objectives, and incident handling protocols.

Testing Constraints and Execution

- Exclusions identify specific assets that are off-limits even if inside broader scope due to sensitivity or disruption risk.
- Test cases are predefined scenarios based on known vulnerabilities, threat vectors, and client concerns to evaluate security systematically.
- Testing windows are approved timeframes for testing designed to minimize disruption and reflect operational conditions.

Adaptive Testing and Reporting

- Goal reprioritization - Formal ROE process to adjust goals when unexpected findings change the engagement trajectory.
- Business impact analysis defines how findings are documented and tied to operational, regulatory, financial, or reputational consequences in reporting.

Exam Focus

- ROE is typically part of the SOW.

Target Selection (OBJ 1.1)

Key Concepts

- Target selection defines scope and attack surface.
- Accurate targeting prevents unauthorized testing.

Network and Infrastructure Targets

- CIDR ranges define IP address blocks in scope for internal or external testing and help ensure coverage of intended systems.

External and Host-Level Targets

- Domains are human-readable targets for external testing, including primary domains and subdomains hosting different services.
- IP addresses represent individual nodes tested through scanning to identify ports and services that may be vulnerable.

Application-Level Targets

- URLs identify specific application endpoints and parameters used during web application testing.

Exam Focus

- Know the difference between domains, IP addresses, and URLs.
- CIDR notation frequently appears in exam questions.

Shared Responsibility Model (OBJ 1.1)

Key Concepts

- Security duties are divided among stakeholders.
- Cloud testing requires coordination.

Lesson Focus

- Cloud providers secure infrastructure and physical assets.
- Customers secure operating systems, applications, and data.
- Third parties secure integrated services.
- Penetration testers evaluate controls across environments.

Exam Focus

- Cloud providers secure security **OF** the cloud.
- Customers secure security **IN** the cloud.

Section 4. Frameworks

11 Lessons | Exam Objectives Covered: 1.3 | Does not include demonstrations

Frameworks (OBJ 1.3)

Key Concepts

- Pentest frameworks provide structured guidelines for conducting penetration tests from start to finish.
- Frameworks support consistency, legal alignment, and reduced risk to the professional.

Framework Overview and Purpose

- Penetration testing frameworks define rules, methodologies, and structured approaches for identifying weaknesses in systems and applications.

Exam Focus

- Expect exam questions asking which framework applies to a given testing scenario.

MITRE ATT&CK (OBJ 1.3)

Key Concepts

- MITRE ATT&CK catalogs adversary tactics, techniques, and procedures (TTPs) from real-world attacks.
- The framework supports threat modeling and threat-informed penetration testing.

ATT&CK Structure and Matrices

- ATT&CK matrices are organized into tactics and techniques across enterprise, mobile, and ICS environments.
- Tactics and techniques include categories like Initial Access, Execution, Persistence, Privilege Escalation, and Defense Evasion.

Penetration Testing Applications

- Threat simulation helps testers replicate known attacker behaviors such as spear phishing attachments.
- Defensive insights support understanding indicators of compromise and mitigation strategies.

Exam Focus

- Know ATT&CK as a TTP-based knowledge base used for adversary emulation.

OWASP (OBJ 1.3)

Key Concepts

- OWASP is a nonprofit foundation focused on improving web application security.
- The OWASP Top 10 identifies the most critical web application security risks.

OWASP Top 10 Focus

- Top 10 resource is updated every 4–5 years to reflect common exploitation risks.
- Top categories are Broken Access Control, Cryptographic Failures, and Injection are major exam-relevant risks.

Web Application Testing Value

- Assessment checklist provides a baseline for prioritizing common vulnerabilities in client web applications.

Exam Focus

- Expect OWASP Top 10 categories in vulnerability identification questions.

MASVS (OBJ 1.3)

Key Concepts

- MASVS (Mobile Application Security Verification Standard) is OWASP's baseline standard for mobile application security requirements.
- MASVS organizes mobile security controls into eight control groups.

MASVS Control Domains

- MASVS-STORAGE - Secure storage and prevention of data leakage.
- MASVS-CRYPTO - Strong encryption and key management.
- MASVS-AUTH - Authentication and authorization controls.
- MASVS-NETWORK - Protection of data in transit with TLS and certificate pinning.
- MASVS-PLATFORM - Secure interaction with mobile OS mechanisms.
- MASVS-CODE - Secure coding, updates, and input validation.
- MASVS-RESILIENCE - Defense against tampering and reverse engineering.
- MASVS-PRIVACY - Privacy controls aligned with laws and consent requirements.

Exam Focus

- Know MASVS as the OWASP standard equivalent for mobile applications.

PTES (OBJ 1.3)

Key Concepts

- PTES (Penetration Testing Execution Standard) provides a standardized lifecycle framework for penetration testing.
- PTES defines seven structured phases from planning through reporting.

PTES Seven Phases

- Pre-engagement Interactions - Defines scope, ROE, tools, timelines, and communication protocols.
- Intelligence Gathering - Includes OSINT, footprinting, and three levels of intelligence depth.
- Threat Modeling - Identifies critical assets, threats, and attack vectors.
- Vulnerability Analysis - Uses passive and active testing with validation to reduce false positives.
- Exploitation - Leverages weaknesses to gain access or escalate privileges.
- Post-exploitation - Simulates attacker actions like persistence and lateral movement.
- Reporting - Documents findings, impact, and remediation recommendations.

Exam Focus

- PTES is a full penetration testing execution standard with tactical CLI guidance.

CREST (OBJ 1.3)

Key Concepts

- CREST (Council of Registered Ethical Security Testers) is a global accreditation body ensuring penetration testing quality and professionalism.
- CDPT (CREST Diploma in Penetration Testing) defines standards for defensible and consistent penetration testing engagements.

Defensible Penetration Testing Requirements

- Accreditation and competence - Requires CREST-accredited organizations and skilled certified testers.
- Mutually agreed specification - Engagements must follow approved methodologies and documented scope.

Professional Reporting Standards

- Defensible reporting - Includes goals, scope, findings, CVSS scoring, and remediation guidance.

Exam Focus

- Know CREST as an accreditation framework emphasizing legal and commercial defensibility.

OSSTMM (OBJ 1.3)

Key Concepts

- OSSTMM (Open Source Security Testing Methodology Manual) is a peer-reviewed scientific methodology for measuring operational security.
- Maintained by ISECOM, OSSTMM covers digital, physical, human, and wireless environments.

Operational Security Measurement

- Evidence-based testing - Produces repeatable, verifiable results without assumptions.
- Holistic coverage - Includes physical security evaluation as a unique feature.

Professional Development

- OSSTMM certifications - Includes OSSTMM Professional Security Analyst (OPSA) and OSSTMM Professional Security Tester (OPST) for validating structured testing expertise.

Exam Focus

- OSSTMM is comprehensive and includes physical security as part of operational testing.

STRIDE (OBJ 1.3)

Key Concepts

- STRIDE is Microsoft's threat modeling framework categorizing six security threat types.
- Each STRIDE category maps to confidentiality, integrity, availability, and authorization risks.

STRIDE Threat Categories

- Spoofing - Identity impersonation targeting authentication.
- Tampering - Unauthorized modification of data affecting integrity.
- Repudiation - Lack of accountability due to missing audit trails.
- Information Disclosure - Exposure of confidential data without authorization.
- Denial of Service - Disruption of service availability through overload.
- Elevation of Privilege - Unauthorized escalation beyond assigned permissions.

Exam Focus

- STRIDE is used for systematic threat categorization during assessments.

Purdue Model (OBJ 1.3)

Key Concepts

- The Purdue Model is a layered architecture for securing industrial control systems (ICS).
- The model enforces segmentation between IT and OT environments.

ICS Hierarchical Levels

- Level 5–4: Enterprise network and business logistics systems.
- Level 3.5: DMZ buffer between IT and OT.
- Level 3–0: Manufacturing operations, control systems, PLCs, and physical processes.

Penetration Testing Value

- Segmentation evaluation - Helps identify weaknesses in isolation and trust relationships across ICS layers.

Exam Focus

- Know Purdue Model as a framework for OT network segmentation and resilience.

OCTAVE (OBJ 1.3)

Key Concepts

- OCTAVE (Operationally Critical Threat, Asset, and Vulnerability Evaluation) is a risk-based framework evaluating threats, assets, and vulnerabilities at an organizational level.
- Developed for self-directed assessments in small to medium organizations.

OCTAVE Three Phases

- Build security requirements - Identifies critical assets and threats through leadership and staff input.
- Identify infrastructure vulnerabilities - Maps assets to systems and evaluates weaknesses.
- Determine risk management strategy - Conducts risk analysis and develops mitigation plans.

Exam Focus

- OCTAVE links technical findings to operational and business risk priorities.

DREAD (OBJ 1.3)

Key Concepts

- DREAD is a Microsoft risk assessment model scoring threats across five categories.
- Scores range from Low to Critical based on total severity.

DREAD Categories and Scoring

- Damage Potential - Measures extent of harm caused by exploitation.
- Reproducibility - Evaluates how reliably an exploit can be repeated.
- Exploitability - Assesses required effort, tools, and access.
- Affected Users - Quantifies scope of user impact.
- Discoverability - Measures how easily attackers can find the vulnerability.

Severity Classification

- Critical: Total score 40–50 requiring immediate attention.
- High, Medium, Low: Ranked based on summed score thresholds.

Exam Focus

- DREAD provides structured vulnerability prioritization but can be subjective.

Section 5. Information Gathering

7 Lessons | Exam Objectives Covered: 2.1 | Does not include demonstrations

Information Gathering (OBJ 2.1)

Information Gathering is the foundation of every penetration test. Before exploitation, privilege escalation, or lateral movement can occur, an attacker (or penetration tester) must first understand what systems exist, how they are connected, and where weaknesses may be present.

This phase is not strictly linear. During later testing stages, newly discovered hosts, services, or applications may require returning to reconnaissance. Information gathering therefore becomes a continuous cycle throughout the engagement.

Key terms

- **Passive Reconnaissance:** Collecting information without touching the target directly (WHOIS, OSINT, archives).
- **Active Reconnaissance:** Directly interacting with target systems (ping sweeps, port scans).
- **OSINT:** Leveraging public sources like job boards, social media, and corporate websites.

Core techniques covered in this section include passive recon, sniffing, active recon, scanning, HTML scraping, cached pages, and banner grabbing.

- **Exam Focus**
Always consider detection risk and legality when choosing methods.

Passive Reconnaissance (OBJ 2.1)

Passive reconnaissance is the process of gathering intelligence without direct interaction with the target's infrastructure. Because no traffic is sent to the target systems, this method is stealthy and reduces detection risk.

Major passive sources include

- **Social Media:** Employees reveal tools, vendors, certifications, and sometimes sensitive operational details.
- **Job Boards:** Hiring postings often expose technologies in use (Apache versions, cloud providers, firewalls).

- Code Repositories: Public GitHub projects may leak credentials, API keys, or infrastructure paths.
- Password Dumps: Breach data can expose credential reuse patterns and corporate email exposure.

Passive reconnaissance builds a target profile that informs later scanning and exploitation.

Exam Focus

- Expect scenario questions asking which sources qualify as OSINT.
- Passive recon never generates packets directly to the target, making it lower risk but less detailed than active scanning.

Network Sniffing (OBJ 2.1)

Network sniffing captures and analyzes packets moving across a network. It provides deep insight into protocols, credentials, misconfigurations, and attacker activity.

Key concepts:

- PCAP Files: Stored packet capture data that can be replayed and analyzed.
- tcpdump: CLI packet capture utility commonly used in field engagements.
- Wireshark: GUI-based packet analyzer with powerful filtering and protocol inspection.
- IoT/OT Protocol Challenges: MQTT and Modbus traffic may require specialized dissectors.

Sniffing can uncover plaintext passwords, insecure protocols, unexpected outbound traffic, or malicious command-and-control patterns.

Exam Focus

- Know that sniffing is a reconnaissance/enumeration technique.
- Questions often test your ability to recognize PCAP usage, Wireshark filtering, and OT protocol risk in industrial environments.

Active Reconnaissance (OBJ 2.1)

Active reconnaissance involves direct interaction with the target environment. Unlike passive recon, active techniques generate traffic that may trigger IDS/IPS alerts.

Core methods

- Port and Protocol Scanning: Identifying open ports and active services.

- TCP SYN Scans: Half-open stealth scans that stop before completing the handshake.
- UDP Scans: Slower and less reliable due to lack of handshake and ambiguous responses.
- Banner Grabbing: Collecting service metadata such as versions and configurations.
- DNS Enumeration: Using dig to discover subdomains, MX records, and misconfigurations.

Active recon provides high detail but increases detection probability.

Exam Focus

- On the exam, you must distinguish active vs passive techniques.
- Active recon is higher fidelity but carries greater legal and operational risk.

Port and Protocol Scanning (OBJ 2.1)

Port scanning maps the attack surface by identifying open ports and the services behind them. TCP and UDP scanning behave differently.

TCP Scanning

- Uses handshake logic.
- SYN scans send SYN → receive SYN-ACK → reset connection.
- Less detectable than full connect scans.

UDP Scanning

- Connectionless, relies on ICMP errors or silence.
- Closed ports return ICMP Port Unreachable.
- No response yields open|filtered ambiguity.

Nmap is the primary tool for scanning, offering service detection, OS fingerprinting, and scripting.

Exam Focus

- Expect port-state interpretation questions.
- Know what open, closed, filtered, and open|filtered mean, especially for UDP.
- Recognize high-value ports like 22, 80, 443, 445.

HTML Scraping and Cached Pages (OBJ 2.1)

HTML scraping analyzes a website's source code to uncover hidden information such as comments, meta tags, exposed scripts, or backend technology identifiers.

Techniques include

- Manual Inspection: Viewing source or using browser DevTools.
- Automated Scraping: Using libraries like BeautifulSoup.
- Header Analysis: `curl -I` reveals server types and redirection info.

Cached pages provide historical insight

Google Cache or Wayback Machine may expose removed endpoints or documents.

Reverse image searches can reveal metadata leaks or sensitive background information.

Exam Focus

- Scenario questions may ask what method reveals outdated endpoints or deleted data.
- Cached archives are passive OSINT sources, often overlooked but valuable.

Banner Grabbing (OBJ 2.1)

Banner grabbing collects identifying information from services, such as software type, version, and configuration details.

Common tools

- `wget -S`: Retrieves HTTP headers.
- `curl -I`: Sends HEAD requests for server responses.
- `netcat`: Manual TCP interaction for raw banners.
- `nmap -sV --script=banner`: Automated service and version detection.

Banner information enables vulnerability research, such as identifying outdated Apache or OpenSSH versions.

Exam Focus

- Expect questions on which tool is best for banner grabbing and why version disclosure matters.
- Banner grabbing is active reconnaissance because it interacts directly with the service.

Section 6. Open-Source Intelligence

7 Lessons | Exam Objectives Covered: 2.1 | Does not include demonstrations

Open-Source Intelligence (OSINT) (OBJ 2.1)

Open-Source Intelligence (OSINT) is the collection and analysis of publicly available information to build an intelligence profile of a target organization during a penetration test. OSINT supports both social-engineering planning (people, roles, culture, routines) and technical reconnaissance (domains, services, certificates, exposed assets) without directly interacting with target systems.

Why OSINT Matters in a Penetration Test

OSINT is valuable because it reduces risk of detection while providing high-leverage context for later phases. A well-built OSINT profile helps you identify realistic pretexts for phishing or impersonation, prioritize which systems to enumerate first, and spot “low-hanging fruit” like forgotten subdomains or exposed documents that can lead to initial access.

OSINT Categories You Must Recognize

For PenTest+, OSINT techniques are usually presented as scenario clues. You should be able to recognize what type of information is being collected and what it implies about the target’s environment.

- People and org structure: employee names, titles, reporting chains, contact patterns.
- Technology disclosures: cloud providers, security tools, versions, and vendors referenced publicly.
- Infrastructure signals: DNS records, certificate data, subdomains, and third-party services.
- Public exposure and leakage: misconfigured storage, code repositories, paste sites, cached pages.

Exam Focus

- Recognize OSINT methods in scenarios and understand how public information can translate into attack vectors.

Social Media and Job Boards (OBJ 2.1)

Social media platforms and job boards are high-value passive sources used during footprinting. They can reveal the target’s people, culture, business priorities, and the

technologies being deployed which are often enough to support realistic social engineering and to guide which systems you should enumerate next.

Social Media Intelligence for Pretexting

Professional and personal posts often contain details that help you craft believable pretexts. LinkedIn is especially useful for mapping the org chart (executives, IT leadership, security staff, help desk) and identifying who would plausibly send or approve requests. Platforms like Instagram and Facebook can expose office layouts, badge styles, and event patterns that support physical or hybrid pretexts.

Signals to look for on Platforms

You are looking for repeatable patterns and corroboration across sources. A single post is a clue; multiple references across different employees becomes confidence.

- Role and authority: job titles, team names, and who reports to whom.
- Technology mentions: cloud providers, SIEM/EDR products, ticketing systems, VPN brands.
- Operational timing: weekend cutovers, maintenance windows, conference travel, “late-night deploy” posts.
- Media leakage: whiteboards, badges, screens, printer trays, or internal signage in photos.

Job Board Disclosures and Technology Mapping

Job postings frequently list exact tools, vendors, and sometimes even versions. These disclosures help you predict what you will later find during enumeration (for example, a posted requirement for Fortinet or Palo Alto, a specific cloud platform, or a virtualization stack). They can also reveal future projects (migrations, new SIEM rollouts, remote work tooling) that create transitional risk.

Exam Focus

- Expect exam questions where social media details enable pretexting or job postings reveal infrastructure. These are passive intelligence sources that shape later technical reconnaissance.

Information Disclosures (OBJ 2.1)

Information disclosure is the unintentional exposure of sensitive data through publicly accessible sources. OSINT-based discovery of disclosures can reveal credentials, configuration details, internal documents, or infrastructure metadata that supports initial access, privilege escalation, or more convincing social engineering.

Common Disclosure Vectors You'll See in the Wild

Disclosures typically come from mistakes, not malice. This means misconfigurations, overly verbose errors, or careless publishing. Your goal is to recognize where exposure happens and what types of data may be available. Examples include:

- Misconfigured servers: open directory listings, exposed status pages, or accidentally published backups.
- Unsecured cloud storage: public buckets/containers containing documents, exports, or logs.
- Public code and paste content: credentials, API keys, tokens, and environment variables.
- Verbose errors/logs: stack traces, file paths, service versions, or internal endpoints.

OSINT Discovery Techniques for Disclosures

Search engines often index exposed content. "Google dorking" can reveal files that should not be public, while Shodan/Censys can identify exposed services and cloud assets. Cached pages and archives can preserve disclosures long after the organization believes they were removed.

How Disclosures Become Attack Paths

Disclosed data is rarely the end goal, instead it is a stepping stone. Leaked credentials can be tested against portals, exposed environment variables can reveal service endpoints, and internal documents can provide names, processes, and terminology used to make phishing more believable.

Exam Focus

- PenTest+ scenarios often test your ability to identify information disclosure sources.
- Misconfigured cloud storage and exposed error output are frequent real-world examples.

Cryptographic Flaws (OBJ 2.1)

Cryptographic flaws include the use of weak or deprecated algorithms and poor certificate management. OSINT can reveal these weaknesses through public TLS scans, certificate metadata, and transparency records, helping you identify systems that may be vulnerable to interception, downgrade, or impersonation risks.

Weak or Deprecated Protocols and Cipher Suites

If a server supports outdated protocols (like SSLv3 or TLS 1.0) or weak ciphers (such as RC4 or 3DES), it may allow downgrade attacks or make encrypted traffic easier to break. Many servers retain weak options for compatibility, and failing to disable them increases exposure.

Certificate Metadata as Infrastructure Intelligence

Digital certificates contain fields that disclose useful OSINT, especially Common Name (CN) and Subject Alternative Names (SANs). SANs can reveal subdomains and services you should include in scope verification and later enumeration. Validity dates can reveal management hygiene, and signature algorithms show whether modern cryptography is being used.

Revocation and Trust Validation Concepts

Certificate revocation matters when keys are compromised or a certificate was issued incorrectly. CRLs (Certificate Revocation List) and OCSP (Online Certificate Status Protocol) are common revocation mechanisms. OCSP stapling further improves reliability by allowing the server to provide proof of status during the handshake, reducing dependence on the client reaching the CA (Certificate Authority) in real time.

Exam Focus

- Expect questions on weak cipher identification, certificate inspection, and how CT logs expose forgotten subdomains or rogue certificates.

DNS Lookups (OBJ 2.1)

DNS servers may support dozens of record types, but PenTest+ emphasizes a smaller set that maps directly to infrastructure clues.

- A / AAAA: map hostnames to IPv4/IPv6 addresses.
- CNAME: aliases one name to another (often CDNs or hosted services).
- MX: mail routing targets (often exposes third-party email providers).
- NS: authoritative name servers for a domain.
- PTR: reverse DNS mapping from IP to hostname.

Tools and Lookup Types

Tools like dig and nslookup are used for forward lookups and reverse lookups. Reverse DNS (PTR) can help you confirm who owns or operates an IP address. Understanding iterative vs. recursive resolution helps you recognize why recursion on the wrong server can become a security problem.

DNS Security Risks That Show Up in Scenarios

Misconfigured DNS can enable zone transfers (AXFR), amplification attacks (open resolvers with recursion), cache poisoning, and flooding. From an OSINT perspective, your takeaway is that DNS configuration issues can leak internal naming schemes and expand the attack surface.

Exam Focus

- PenTest+ tests DNS enumeration heavily.
- Know record types, forward vs reverse lookups, and how recursion misconfiguration enables amplification or data leakage.

Certificate Transparency Logs (OBJ 2.1)

Certificate Transparency (CT) logs are public, append-only records of issued SSL/TLS certificates. They improve accountability in the public PKI ecosystem and also provide a powerful OSINT source for discovering subdomains, test environments, and misissued or rogue certificates.

What CT Logs Reveal to a Penetration Tester

CT entries provide visibility into the domains covered by certificates, who issued them, and when they were issued. This can reveal patterns of automation, renewal habits, and trust relationships which are useful for mapping external infrastructure and identifying overlooked assets. Examples include:

- Hidden or forgotten subdomains (dev, staging, beta, internal portals exposed externally).
- Certificate issuance patterns that suggest automation and potential CI/CD dependencies.
- CA trust relationships and which issuers are commonly used for the domain.
- Potentially neglected assets (expired certs, inconsistent renewal behavior).

Rogue or Misissued Certificates and Risk

A certificate issued for a domain or subdomain without authorization is a serious red flag. In real incidents, misissuance can enable impersonation and on-path attacks if attackers obtain valid, trusted certificates. CT logs help detect these events because issuance becomes publicly visible.

Practical OSINT Workflow Using CT Data

Penetration testers typically use CT log search tools (such as crt.sh) and cross-reference discoveries with DNS, passive DNS history, and infrastructure mapping to confirm which subdomains belong to the target and where they resolve.

Exam Focus

- Expect scenario questions where CT logs reveal hidden infrastructure.
- Rogue certificates and exposed subdomains are common reconnaissance findings.

Search Engine Analysis (OBJ 2.1)

Search engine analysis (often called Google dorking) uses advanced operators to identify publicly indexed content that organizations did not intend to expose. This is a core OSINT technique because it can uncover documents, credentials, configuration files, open directories, and administrative interfaces without direct probing.

Core Operators You Must Be Able to Use or Recognize

Operators are combined to narrow scope, locate specific file types, and target high-value pages. On the exam, these often appear as dorks inside a scenario question, and include:

- Quotation marks (") for exact phrases.
- Minus sign (-) to exclude terms.
- AND/OR to combine or broaden results.
- site: to restrict results to a domain.
- filetype: to locate documents (pdf, docx, xls, csv, config).
- intitle: to locate pages with specific titles (admin panels, directory listings).
- inurl: to target common paths (login, admin, backup).

Common Findings from Search Engine Enumeration

Search engines may index exposed backups, environment files, debug pages, and directory listings. These findings can directly reveal credentials, internal endpoints, or software versions that support targeted exploitation or credential attacks.

Monitoring and Non-Text Discovery

Reverse image search tools can identify reused images, leaked screenshots, and metadata. Alerts (such as Google Alerts) provide passive monitoring for newly indexed exposures, leaked documents, or mentions of the target that expand the attack surface over time.

Exam Focus

- PenTest+ frequently tests your ability to recognize search operators and identify what types of exposed assets can be discovered through public indexing.

Section 7. Scanning and Enumeration

12 Lessons | Exam Objectives Covered: 2.2 | Does not include demonstrations

Scanning and Enumeration (OBJ 2.2)

Scanning and enumeration are the active phases where penetration testers move beyond passive reconnaissance and begin directly probing target systems. These techniques help identify services, protocols, users, and configurations that may expose exploitable weaknesses.

Active Reconnaissance Transition

- Scanning builds on reconnaissance by directly querying systems for open ports and exposed services.
- Enumeration extracts deeper details such as user accounts, protocol behavior, and system configuration.

Target Environment Mapping

- This phase provides a detailed picture of the network, highlighting entry points and sensitive assets.
- Information gathered here supports exploitation planning and later attack path development.

Enumeration as Investigation

- Enumeration goes beyond discovery by identifying how services are implemented and what data they expose.
- Detailed enumeration reduces guesswork and improves attack precision.

Exam Focus

- Know the difference between scanning for exposure and enumerating for detailed service intelligence.

Enumerating Protocols (OBJ 2.2)

Protocol enumeration identifies how systems communicate across the network and what services are running. By analyzing protocol behavior at multiple layers, penetration testers can uncover misconfigurations, weak implementations, and vulnerable services.

Network Layer Enumeration

- ICMP enumeration supports host discovery through ping sweeps and response analysis.
- Firewall behavior can be inferred by comparing which ICMP packet types are blocked or allowed.
- IPSec enumeration evaluates tunnel security by examining encryption algorithms and key exchange methods.

Transport Layer Enumeration

- TCP enumeration identifies open ports and service versions that may be vulnerable.
- SYN-based scanning reveals open services without completing full handshakes, reducing detection.
- UDP enumeration is critical for finding overlooked services such as SNMP on port 161.
- Default SNMP community strings like public or private can expose device configurations and routing data.

Application and Remote Access Protocol Enumeration

- HTTP/HTTPS enumeration includes banner grabbing and version detection to identify outdated servers.
- SNMP enumeration with tools like snmpwalk can reveal sensitive infrastructure details.
- SMB enumeration identifies accessible shares and misconfigurations, with SMBv1 posing major risks.
- RDP enumeration focuses on weak credentials, brute-force exposure, and outdated encryption.
- SSH enumeration reveals version details that can be matched against known vulnerabilities.

Exam Focus

- Expect protocol questions tied to ports, service weaknesses, and enumeration outputs.

- Know high-risk protocols like SNMP, SMB, RDP, and how version discovery supports exploitation.

Enumerating DNS (OBJ 2.2)

DNS enumeration is a reconnaissance and enumeration technique that reveals domain infrastructure, subdomains, mail servers, and misconfigurations. DNS records often expose valuable targets and expand the attack surface.

DNS Query Techniques

- Tools like nslookup and dig perform forward lookups to identify public-facing IP addresses.
- Simple DNS queries often reveal hosting providers, CDNs, and external dependencies.

Zone Transfers and Record Enumeration

- Zone transfers (AXFR) can leak an entire DNS zone if misconfigured.
- Attackers may attempt zone transfers early because success provides a complete domain map.
- Key PenTest+ record types include A, AAAA, MX, CNAME, NS, PTR, SOA, and TXT.
- MX queries reveal email infrastructure, while NS queries identify authoritative DNS providers.

Brute Force and Advanced DNS Discovery

- SRV records can expose hidden services such as SIP, LDAP, or VoIP endpoints.
- Reverse lookups (PTR) map IP addresses back to hostnames, revealing related infrastructure.
- Tools like dnsrecon and dnsenum brute force subdomains such as dev or staging environments.
- Online tools like DNSDumpster and VirusTotal provide historical DNS intelligence.

Exam Focus

- Zone transfer misconfigurations are a common exam scenario.
- Know DNS record purposes and how enumeration reveals hidden services and subdomains.

Enumerating Directories (OBJ 2.2)

Directory enumeration identifies hidden web paths, sensitive files, and misconfigured resources. By mapping directories and server responses, penetration testers uncover attack vectors that are not visible through normal browsing.

Automated Directory Discovery

- Tools like DirBuster, Gobuster, and Dirsearch brute force directory names using wordlists.
- Common findings include /admin, /backup, or /test directories that remain publicly accessible.
- Custom wordlists improve success by targeting organization-specific naming conventions.

Metadata and Server Response Clues

- robots.txt often reveals restricted paths that attackers can still access directly.
- HTTP response codes provide insight into directory existence.
 - 403 indicates the directory exists but is restricted.
 - 404 indicates the directory does not exist.
- Directory listing vulnerabilities expose file contents when no index file is present.

Advanced Enumeration Techniques

- URL fuzzing injects payloads into parameters to discover hidden functionality.
- Burp Suite Intruder automates fuzzing and highlights response differences.
- SSI injection may allow execution of server-side directives if improperly supported.
- Enumeration can extend into network shares and directory services using built-in OS tools.

Exam Focus

- Directory enumeration questions often involve tools, robots.txt findings, and interpreting HTTP status codes.
- Know the difference between hidden directories, restricted resources, and directory listing exposure.

Enumerating Hosts (OBJ 2.2)

Host enumeration identifies which systems are alive on a network and what services they expose. This step builds the foundation for exploitation by mapping devices, ports, operating systems, and high-value targets.

Host Discovery Techniques

- Host discovery determines which IP addresses respond and are active within the target scope.
- ICMP ping scans are common, but many networks block ICMP to reduce visibility.
- Nmap ping scanning with `-sn` quickly identifies live systems without port scanning.

Alternative Discovery Methods

- TCP ACK pings confirm host availability through RST responses even when ICMP is blocked.
- UDP pings use ICMP port unreachable messages to prove systems are online.
- TCP SYN pings reveal live hosts through SYN-ACK or RST responses.
- The `-Pn` option treats all hosts as alive when discovery is unreliable.

Service Scanning and OS Fingerprinting

- SYN scans (`-sS`) enumerate open TCP ports without completing full handshakes.
- OS fingerprinting (`-O`) identifies operating systems and versions for exploit targeting.
- Response timing and probe behavior help distinguish between platforms.

Advanced and Credentialed Enumeration

- Credentialed scans with tools like Nessus reveal deeper configuration and patch details.
- Windows enumeration commands like `net view` and `arp -a` expose hosts and shared resources.
- Active Directory enumeration with PowerShell identifies domain assets and privileged accounts.

Exam Focus

- Know multiple host discovery methods beyond ICMP.
- Scenarios often test scan flags like `-sn`, `-Pn`, `-sS`, and OS detection (`-O`).

Enumerating Users (OBJ 2.2)

User enumeration identifies accounts and group memberships on a system. This information supports privilege escalation, credential harvesting, and targeting high-value users during exploitation.

Windows User Enumeration

- *Get-LocalUser* lists local accounts with properties such as enabled status.
- *net user* provides detailed manual output including password policies and last logon.
- Enabled accounts represent primary compromise targets in penetration tests.

Group Membership and Privilege Discovery

- *net localgroup administrators* identifies users with full system control.
- *net localgroup* lists all groups that may provide specialized privileges.
- Groups like Backup Operators or Remote Desktop Users create lateral movement paths.

Linux User Enumeration

- */etc/passwd* is world-readable and lists all accounts, shells, and UIDs.
- Root accounts have UID 0 and represent the highest-value escalation targets.
- */etc/shadow* contains password hashes and is restricted to root by default.
- Misconfigurations exposing shadow hashes enable offline cracking attacks.

Attack Implications

- Weak accounts with valid shells and no passwords are exploitable entry points.
- Commands like *id* reveal privilege level and group memberships.
- Process enumeration can expose logged-in users and escalation opportunities.

Exam Focus

- Expect questions on Windows vs Linux enumeration artifacts.
- Know the difference between */etc/passwd* and */etc/shadow* and why group membership matters.

Enumerating Email (OBJ 2.2)

Email enumeration gathers valid email addresses within a target organization. This supports phishing, social engineering, and credential-based attacks by revealing real accounts and organizational structure.

Protocol-Based Email Enumeration

- SMTP servers may expose account validity through commands like VRFY or EXPN.
- Enumerating mailing lists can reveal departments, roles, and internal structures.
- Misconfigured SMTP responses provide attackers with confirmation of active users.

OSINT Email Collection

- Tools like theHarvester extract emails from public sources such as Google and LinkedIn.
- Social platforms often expose professional contact formats and naming conventions.
- WHOIS databases may reveal administrative or technical contact emails when available.

Breach Intelligence Sources

- Have I Been Pwned provides breach exposure data tied to specific addresses.
- Domain-wide breach searches require verified ownership and are not always usable in engagements.
- Leaked credentials support credential stuffing and password reuse testing.

Validation and Testing

- Services like Hunter.io confirm whether collected addresses are deliverable.
- Verified lists support phishing simulation testing and susceptibility assessment.

Exam Focus

- PenTest+ tests email enumeration as both a technical and social engineering enabler.
- Know SMTP enumeration commands and OSINT tools like theHarvester.

Enumerating Permissions (OBJ 2.2)

Permission enumeration identifies access controls across systems, shares, databases, Active Directory, and cloud environments. Misconfigured permissions often enable unauthorized access and privilege escalation.

File System and Local Permission Enumeration

- PowerView enumerates Windows shares and privilege assignments across network resources.
- *Get-NetShare* identifies accessible shared folders and potential overexposure.
- Linux tools like *ls -l* display permission bits, while *getfacl* reveals ACL details.
- Readable home directories may expose sensitive files even without write access.

Database Permission Enumeration

- SQLmap automates SQL injection testing and enumerates databases, roles, and privileges.

- Weak database permissions may expose customer data, credentials, or financial records.
- Enumeration helps determine how far compromise can extend beyond the application layer.

Network Share and Active Directory Enumeration

- SMBMap and CrackMapExec list shared folders and permission levels.
- *Get-SMBShareAccess* identifies users or groups with Read, Change, or Full control.
- *Get-ObjectAcl* reveals excessive rights on AD objects such as OUs.
- Over-permissioned AD structures enable persistence and escalation inside domains.

Cloud Permission Enumeration

- AWS CLI commands enumerate IAM users and attached policies.
- Overly broad roles or misconfigured policies create high-impact cloud compromise paths.
- Cloud enumeration is critical due to programmatic access through API permissions.

Exam Focus

- Expect scenario questions involving excessive permissions and privilege escalation paths.
- Know key tools: PowerView, SMBMap, SQLmap, AWS CLI, and getfacl.

Enumerating Wireless Devices (OBJ 2.2)

Wireless device enumeration identifies access points, encryption methods, and rogue wireless infrastructure. Because wireless networks extend beyond physical boundaries, they often provide attackers with easier entry paths into an environment.

Wireless Reconnaissance Techniques

- Wardriving involves scanning for wireless access points while moving through an area.
- War walking performs the same technique on foot in dense environments.
- War chalking historically marked public areas with symbols indicating nearby networks.
- Drones may extend wireless scanning into hard-to-reach or secured areas.

Tools and Packet Analysis

- Tools like Kismet, Aircrack-ng, and Wifite detect SSIDs and capture wireless traffic.
- Wireshark analysis reveals whether encryption is strong or sensitive traffic is exposed.
- Captured packets help assess authentication weaknesses and misconfigurations.

WiGLE and Signal Amplification

- WiGLE maps and indexes access points with location and encryption metadata.
- Filters can identify open or weakly protected networks in a target area.
- High-gain antennas extend scanning range and improve detection accuracy.
- Directional vs omni-directional antennas support different reconnaissance needs.

Exam Focus

- Know wardriving terminology and common wireless enumeration tools.
- PenTest+ scenarios often involve identifying rogue or misconfigured wireless access points.

Enumerating Secrets (OBJ 2.2)

Secret enumeration focuses on discovering sensitive authentication material such as passwords, session tokens, cloud keys, and API keys. Poor secret management provides attackers with direct access and escalation opportunities.

Password Enumeration Techniques

- Weak passwords are often uncovered through brute forcing and password spraying.
- Credential stuffing tests reused passwords across multiple systems or services.
- Exposed passwords provide immediate unauthorized access and escalation paths.

Session Token Enumeration

- Session tokens maintain authentication without repeated password entry.
- Session hijacking occurs when attackers intercept tokens during transmission and then use them.
- Replay attacks reuse captured tokens to impersonate legitimate users.

- Burp Suite intercepts HTTP/HTTPS traffic and exposes cookies and token values.

Cloud Access Key Enumeration

- Cloud keys authenticate programmatic access to environments like AWS.
- Keys are often stored in configuration files, environment variables, or metadata services.
- Pacu is an AWS exploitation framework that enumerates access keys and permissions.
- Compromised cloud keys can enable control of resources such as virtual machines.

API Key Enumeration

- API keys authorize requests to services and applications.
- Keys embedded in repositories or configs are common exposure points.
- Administrative API keys can enable large-scale unauthorized actions and data compromise.

Exam Focus

- Expect questions tying secret exposure to privilege escalation and impersonation.
- Know tools like Burp Suite and Pacu and common attack methods like token replay.

Enumerating the Web (OBJ 2.2)

Web enumeration discovers hidden resources, misconfigurations, plugins, and defensive controls in web applications. This process expands the attack surface and identifies exploitable entry points.

Automated Web Crawling and Discovery

- Web crawling maps site structure and uncovers hidden content.
- DirBuster brute forces directories and files using wordlists.
- Common findings include admin panels, backup files, and exposed configuration scripts.
- Enumeration reveals resources not linked through normal navigation.

Manual Enumeration and Website Metadata

- robots.txt discloses restricted paths through Disallow directives.
- Attackers use these paths as direct targets for further testing.

- sitemaps list URLs and may reveal hidden or forgotten application pages.
- Metadata such as last modification dates highlight outdated or unmaintained areas.

Plugin Enumeration and Vulnerability Research

- Plugins and extensions are frequent vulnerability sources when outdated.
- HTML source inspection can reveal plugin names and versions.
- Vulnerability databases like NVD and Exploit-DB confirm known weaknesses.
- Vulnerable plugins can enable full compromise through privilege escalation or backdoors.

Web Application Firewalls (WAFs)

- WAFs filter HTTP traffic and often act as reverse proxies.
- Penetration testers can attempt to identify the origin server behind the WAF.
- DNS misconfigurations may expose true origin IP addresses, enabling bypass of filtering.

Exam Focus

- PenTest+ emphasizes tools like DirBuster and metadata sources like robots.txt.
- Know how plugin versions and WAF misconfigurations expand attack opportunities.

Attack Path Mapping (OBJ 2.2)

Attack path mapping models how an attacker could progress through a network from initial access to high-value compromise. This technique connects enumeration findings into realistic escalation and lateral movement routes.

Mapping Techniques and Tools

- Attack paths begin with identifying entry points such as open ports or weak credentials.
- BloodHound visualizes Active Directory relationships using graph theory.
- Privilege escalation paths become clear through group memberships and trust links.
- MITRE ATT&CK provides a structured library of attacker tactics and techniques.

Common Attack Progression Examples

- Phishing can lead to malware installation and privilege escalation.

- Admin access enables scanning, lateral movement, and data exfiltration.
- Web vulnerabilities like SQL injection can expose databases and credentials.
- Credential reuse supports deeper compromise across internal systems.

Documentation and Continuous Improvement

- Attack paths must be documented step-by-step for remediation planning.
- Collaboration with IT and security teams improves accuracy and defensive alignment.
- Attack maps must remain updated as systems change and new vulnerabilities emerge.

Exam Focus

- Expect scenario questions combining enumeration results into escalation routes.
- Know BloodHound and MITRE ATT&CK as core frameworks for attack path reasoning.

Section 8. Reconnaissance and Enumeration Tools

9 Lessons | Exam Objectives Covered: 2.4, 4.8 | Does not include demonstrations

Reconnaissance and Enumeration Tools (OBJ 2.4)

Reconnaissance and enumeration tools allow penetration testers to gather detailed intelligence about a target before exploitation begins.

Tool-Driven Reconnaissance

Reconnaissance is the process of gathering as much useful information as possible before testing vulnerabilities. Enumeration builds on this by identifying specific systems, services, and data points that expand the attack surface.

Domain 2 Objective Alignment

This section focuses on Domain 2, Reconnaissance and Enumeration, and emphasizes selecting the appropriate tool based on the scenario, such as historical archives, DNS mapping, or packet analysis.

Practical Application

Using the right tool provides a wealth of data about targets, including web history, email structures, DNS records, exposed devices, captured traffic, and wireless networks.

Exam Focus

- Objective 2.4 expects tool recognition and correct selection in scenarios.
- Recon and enumeration tools often appear in exam questions tied to attack surface discovery.

Wayback Machine (OBJ 2.4)

The Internet Archive's Wayback Machine is an OSINT tool that provides historical snapshots of websites. Penetration testers use it to uncover content and structure that may no longer be visible on the live site.

Role in Reconnaissance

Archived pages may expose old directories, removed login portals, subdomains, or outdated frameworks. These findings provide historical context that can guide later testing.

Historical Snapshot Intelligence

Past job postings, documents, or sensitive content can support phishing and social engineering. Removed subdomains may still resolve or indicate legacy infrastructure.

Limitations and Use

The Wayback Machine does not capture all content, especially dynamic pages. Even with incomplete archives, it remains valuable for historical OSINT.

Exam Focus

- Expect questions where archived web content reveals forgotten assets.
- Historical subdomains can indicate potential takeover or legacy exposure.

theHarvester and Hunter.io (OBJ 2.4)

theHarvester and Hunter.io are OSINT tools designed to collect publicly available organizational contact data. They help penetration testers identify human-focused and technical attack vectors early in an engagement.

theHarvester Capabilities

theHarvester automates discovery of email addresses, subdomains, IP addresses, and domains from multiple public sources. Subdomains often represent additional attack surfaces.

Applying Harvested Data

Discovered emails can support spear-phishing campaigns, while subdomain enumeration expands endpoints to probe for misconfigurations or outdated services.

Hunter.io Email Verification

Hunter.io focuses on verifying company email addresses and revealing naming conventions such as `firstname.lastname@company.com`, enabling realistic address generation.

Exam Focus

- Know that theHarvester gathers emails and subdomains for footprinting.
- Hunter.io supports phishing preparation through verified addresses and patterns.

OSINT Tools (OBJ 2.4, 4.8)

Penetration testers rely on structured OSINT tools to collect and correlate intelligence at scale. Tools like OSINT Framework, Maltego, and SpiderFoot streamline reconnaissance and relationship discovery.

OSINT Framework Directory

The OSINT Framework organizes tools into categories such as email, domains, usernames, and geolocation. Its legend indicates installation needs, registration requirements, or Google dork usage.

Maltego Relationship Mapping

Maltego provides graphical visualization of linked entities. Transforms expand one entity, such as a domain, into related IP addresses, DNS records, or social accounts.

SpiderFoot Automation

SpiderFoot automates reconnaissance across 100+ sources, producing results such as external accounts, email addresses, IPs, and blacklisted infrastructure.

Exam Focus

- Maltego transforms connect domains, IP addresses, and organizational entities.
- SpiderFoot is used for automated large-scale OSINT enumeration.

DNSdumpster and Amass (OBJ 2.4)

DNSdumpster and Amass are domain-focused reconnaissance tools used to map external infrastructure. They help penetration testers identify subdomains, IP addresses, DNS records, and hidden services.

DNSdumpster Passive Mapping

DNSdumpster provides a visual overview of DNS infrastructure, identifying subdomains, MX records, and hosting relationships. This highlights exposed external assets.

Subdomain Discovery Value

Hidden subdomains may point to development or administrative portals that are often less hardened than production systems.

Amass Active and Passive Enumeration

Amass combines techniques such as certificate transparency analysis, recursive discovery, and brute-forcing. Subcommands like intel, enum, and db support scope definition and correlation.

Exam Focus

- DNSdumpster provides passive reconnaissance with visual DNS relationships.
- Amass supports deeper enumeration through active and passive techniques.

Shodan and Censys.io (OBJ 2.4)

Shodan and Censys.io provide Internet-scale reconnaissance by indexing exposed devices, services, and configurations. These tools reveal real-world attack surfaces beyond traditional web search engines.

Shodan Device Indexing

Shodan scans for Internet-connected devices and collects banners, metadata, software versions, and open ports. This supports identification of vulnerable services.

Censys Structured Intelligence

Censys performs daily scans of IPv4 address space and ports, normalizing data into queryable datasets. It enables advanced searches across certificates and protocols.

Automation and Monitoring

Both platforms support alerting and API access, allowing continuous monitoring of newly exposed assets or certificate changes.

Exam Focus

- Shodan indexes exposed devices and service banners, not web pages.
- Censys enables structured queries and API-driven reconnaissance workflows.

tcpdump (OBJ 2.4)

tcpdump is a lightweight command-line packet capture tool used to observe network traffic directly. Penetration testers use it for reconnaissance, exploitation validation, and evidence collection.

Role in Packet Visibility

tcpdump captures raw packets, exposing active protocols, communication patterns, and potential cleartext data leakage.

Reconnaissance and Exploitation Support

Captures may reveal sensitive information such as credentials, cookies, or authorization headers, confirming weak controls.

Filters and Targeted Captures

tcpdump uses flags and BPF filters to reduce noise and capture specific hosts, ports, or SYN-only scan behavior for later analysis.

Exam Focus

- tcpdump findings often justify high-severity issues like exposed credentials.
- Be aware that BPF filters define precise capture criteria for traffic.

Wireshark (OBJ 2.4)

Wireshark is a graphical packet capture and protocol analyzer that provides deep visibility into network communications. It supports discovery of vulnerabilities, misconfigurations, and exploitation evidence.

Web Vulnerability Evidence

Wireshark can reveal injection attempts through HTTP payloads and confirm successful bypasses by analyzing requests and responses.

Cleartext Credential Exposure

Protocols like Telnet transmit authentication data in plaintext. Wireshark reconstructs sessions through Follow TCP Stream for reproducible proof.

DNS and Infrastructure Enumeration

DNS captures expose internal hostnames, CNAME relationships, and mail infrastructure details that guide further enumeration.

Exam Focus

- Expect questions on identifying cleartext protocols and credential exposure.
- Wireshark evidence supports reporting of injection and network-layer findings.

Wireless Analysis Tools (OBJ 2.4)

Wireless reconnaissance tools support offensive operations such as handshake capture, signal mapping, and wireless footprinting. Core tools include Aircrack-ng, InSSIDer, and WiGLE.

Aircrack-ng Suite

Aircrack-ng captures packets, injects frames, forces handshakes, and performs offline cracking against WEP/WPA/WPA2 networks.

InSSIDer RF Visibility

InSSIDer provides situational awareness by displaying SSIDs, BSSIDs, channel usage, and signal strength, enabling precise targeting.

WiGLE Remote Mapping

WiGLE offers geolocated SSID and BSSID intelligence for pre-engagement planning and identifying SSID reuse across locations.

Exam Focus

- Aircrack-ng is central for handshake capture and offline cracking workflows.
- InSSIDer identifies channels and BSSIDs for accurate wireless targeting.
- WiGLE supports remote wireless mapping and reconnaissance planning.

Section 9. Nmap and NSE (Expanded Standalone Version)

4 Lessons | Exam Objectives Covered: 2.4, 4.2 | Does not include demonstrations

Nmap and NSE (OBJ 2.4, 4.2)

Nmap is one of the most important tools in penetration testing because it bridges the gap between passive reconnaissance and active enumeration. Once you have identified a target's address space or domain footprint, Nmap allows you to directly interact with hosts to determine what systems are alive, what ports are reachable, and what services may be exposed. This makes it a foundational tool for shaping an attacker's next move.

What the Nmap Scripting Engine Adds

Nmap becomes even more powerful through the Nmap Scripting Engine (NSE), which allows penetration testers to automate deeper checks using Lua scripts. Instead of stopping at open ports, NSE scripts can enumerate web directories, probe SMB shares, detect misconfigurations, and even identify known vulnerabilities. This automation makes scanning both broader and more precise when used responsibly within scope.

How PenTest+ Tests Nmap Use

For the PenTest+ exam, you are expected to understand not just that Nmap exists, but what its flags mean in realistic scenarios. Questions often test discovery scans, port scan types, version detection, OS fingerprinting, and how NSE scripts extend Nmap into vulnerability-focused enumeration.

Exam Focus

- Understand SYN vs Connect scans and common port states.
- Recognize -sV, -A, -O, and how NSE scripts extend enumeration.

Nmap Discovery Scans (OBJ 2.4, 4.2)

Discovery scans are used to determine which systems are actually online before you waste time scanning inactive addresses. In an engagement, discovery is like confirming which buildings have lights on before checking every door. A clean discovery phase gives you a focused target list and reduces unnecessary noise on the network.

Discovery Probe Types and When to Use Them

Nmap provides multiple probe types depending on environment. On a local LAN, ARP discovery is the most accurate because Layer 2 ARP traffic cannot be blocked the same way ICMP can. Across routed networks, TCP SYN pings, ACK probes, UDP probes,

and ICMP echo requests provide alternative methods when firewalls filter certain traffic. Combining probes improves reliability when targets respond selectively.

Timing, Stealth, and Realistic Engagement Considerations

Professional scanning requires balancing speed and detectability. Timing templates, retry limits, and host timeouts help match scanning behavior to defensive conditions. In restrictive environments, -Pn may be required to treat hosts as up, but it increases traffic significantly. Advanced methods like idle scans exist for stealth but are fragile and rarely used without careful validation.

Exam Focus

- Know which discovery probes apply on LAN vs routed networks.

Nmap Port Scans (OBJ 2.4, 4.2)

Port scanning narrows the massive theoretical port range into a practical list of reachable services. Each open port represents an entry point where software is listening, and each listening service may contain vulnerabilities, weak authentication, or misconfigurations. Attackers rarely exploit a host directly, instead they exploit the exposed services on that host.

Key Scan Types and Tradeoffs

Different scan types reveal different information. SYN scans are fast and semi-stealthy because they never complete the handshake. Connect scans are more visible but work without privileges. FIN, Null, and Xmas scans test unusual TCP behaviors and may bypass weak filters, but they are noisy against modern defenses. UDP scans are essential for discovering services like DNS or SNMP, but they are slower and often return ambiguous results.

Interpreting Port States for Exploitation Planning

The meaning of port states matters for prioritization. Open ports require immediate service enumeration. Closed ports confirm reachability but no listening service. Filtered ports indicate defensive controls blocking probes. States like open|filtered often occur in UDP and require follow-up testing. Reading these results correctly helps penetration testers decide where to focus next.

Exam Focus

- Expect scenario questions requiring the correct Nmap flag selection.

Nmap Fingerprinting (OBJ 2.4, 4.2)

Footprinting is passive mapping of public assets, while fingerprinting is active probing to identify exactly what software and platforms are running. Fingerprinting transforms a generic target into a specific one, such as identifying an IIS server on Windows Server 2016 rather than simply 'a web server.' This precision is what allows exploit research to become realistic.

Service Version Detection and OS Identification

Nmap's `-sV` performs service and version detection by comparing banners and responses against known fingerprints. Aggressive scans (`-A`) combine multiple techniques including OS detection, default scripts, traceroute, and version probing. OS detection (`-O`) relies on subtle TCP/IP stack behaviors, which helps align your attack approach with platform-specific weaknesses like SMB or RDP on Windows hosts.

Deep Enumeration with NSE Scripts

The Nmap Scripting Engine expands fingerprinting beyond identification into actionable enumeration. Scripts like `http-enum` reveal hidden directories, `smb-os-discovery` extracts Windows domain information, and `vuln-category` scripts check for known weaknesses. Script arguments allow customization, such as supplying username/password lists for brute-force testing. NSE is what turns Nmap into a full reconnaissance framework.

Exam Focus

- Expect scenario questions requiring the correct Nmap flag selection.

Section 10. Scripting Basics

4 Lessons | Exam Objectives Covered: 2.3| Does not include demonstrations

Scripting Basics (OBJ 2.3)

Scripting is a foundational skill in penetration testing because it allows testers to automate repetitive reconnaissance and enumeration tasks. Instead of manually executing the same commands repeatedly, scripts enable consistent workflows that save time and reduce human error.

Pseudocode as a Universal Language

This lesson emphasizes the use of pseudocode as a way to describe logic without being tied to a specific programming language. Pseudocode helps students understand algorithm structure before translating it into Bash, PowerShell, Python, or another language.

Core Programming Building Blocks

The building blocks introduced include variables, loops, logic control, data structures, and object-oriented concepts. These are the same constructs used in automation scripts for scanning, parsing results, or interacting with APIs.

PenTest+ Objective Relevance

PenTest+ Objective 2.3 expects you to recognize and modify scripts used in reconnaissance and enumeration. The exam focuses more on understanding what code does rather than writing complex scripts from scratch.

Exam Focus

- Recognize pseudocode as a planning tool before coding.
- Understand that scripting is used to automate recon and enumeration tasks.
- Expect questions asking what a script does or how to modify it.

Shells and Programming Languages (OBJ 2.3)

Penetration testers encounter many scripting environments. The exam expects you to interpret common scripting languages and understand how scripts automate command-line tasks.

Bash and Linux Automation

Bash is the default shell on Unix-like systems and is heavily used for automation. Bash scripts begin with a shebang (`#!/bin/bash`) and can combine commands like `grep`, `cut`, and redirection to parse logs or extract information quickly.

PowerShell and Windows Scripting

PowerShell is Microsoft's scripting language and uses readable Verb-Noun cmdlets such as `Get-EventLog`. It is commonly used to automate Windows enumeration, log analysis, and administrative tasks.

High-Level Languages for Pentesting

Python, Ruby, Perl, and JavaScript appear frequently in penetration testing. Python is widely used for automation and exploit development, Ruby powers Metasploit, Perl excels at text parsing, and JavaScript is essential for understanding web application behavior.

Exam Focus

- Be able to identify what a Bash or PowerShell script is doing.
- Know that PenTest+ focuses on reading scripts, not writing full programs.
- Recognize common languages used in pentesting workflows.

Variables and Constants (OBJ 2.3)

Variables and constants allow scripts to store and manage data during execution. Understanding data types is essential because automation scripts rely on accurate handling of values.

Core Data Types

Data types include booleans, integers, floats, characters, and strings. Scripts must correctly handle these types because operations differ depending on how values are stored.

Variables as Dynamic Storage

Variables are named memory locations that hold values that can change. In penetration testing, variables store dynamic data such as IP addresses, usernames, or scan results.

Constants as Fixed Values

Constants store values that never change, such as π in calculations. Constants are often written in uppercase to distinguish them from variables.

Exam Focus

- Know the difference between variables and constants.
- Expect questions involving data types and conversions.
- Understand why correct data handling matters in automation scripts.

Loops (OBJ 2.3)

Loops enable penetration testers to automate repetitive actions such as scanning multiple ports, iterating payloads, or testing credentials systematically.

For Loops for Fixed Iterations

For loops repeat a set number of times. They are ideal for iterating through known port ranges or a fixed list of targets during enumeration.

While Loops for Conditional Repetition

While loops continue running as long as a condition remains true. These are useful for retry logic, monitoring services, or brute-force attempts with stopping conditions.

Do Loops for Guaranteed Execution

Do loops execute at least once before checking the condition. They are valuable when an initial probe must always occur before evaluating whether to continue.

Exam Focus

- Recognize loop structures in pseudocode.
- Understand which loop type fits a given penetration testing scenario.
- Loops are key for automation in reconnaissance scripts.

Logic Control (OBJ 2.3)

Logic control allows scripts to make decisions based on scan results, responses, or conditional thresholds. This makes automation adaptive instead of static.

Boolean Decision Making

Boolean logic evaluates conditions as true or false. Scripts commonly use if/else statements to decide whether a host is reachable or a login succeeded.

Arithmetic Comparisons in Scripts

Arithmetic comparisons use operators like > or < to guide decisions based on numerical thresholds, such as scan response times or open port counts.

String Matching for Responses

String comparisons evaluate text-based data like usernames, file paths, or web responses. Case sensitivity is critical in many penetration testing contexts.

Exam Focus

- Expect questions involving if/else logic in scripts.
- Know the difference between numeric and string comparisons.
- Logic control enables scripts to respond intelligently during testing.

Data Structures (OBJ 2.3)

Data structures organize information in ways that make automation efficient. Penetration testers rely on structured data when parsing tool output or interacting with APIs.

Key-Value Pairs and JSON

Key-value pairs form the foundation of structured data. JSON builds on this concept and is widely used in APIs and configuration files encountered in pentesting.

Arrays, Lists, and Dictionaries

Arrays store ordered values of the same type, while lists allow mixed types. Dictionaries provide fast lookups by key, making them ideal for mapping scan results or credentials.

CSV and Tree Structures

CSV files store tabular data for exports from tools like Nessus. Trees represent hierarchical relationships, such as file systems or Active Directory structures.

Exam Focus

- Recognize JSON and key-value structures in scenarios.
- Understand when arrays vs dictionaries are used in scripts.
- Data structures help automate parsing and reporting.

Object Oriented Programming (OBJ 2.3)

Object-oriented programming organizes code around reusable objects. Penetration testing frameworks often rely on OOP principles for scalability and modularity.

Functions and Reusable Tasks

Functions perform specific reusable tasks, such as checking ports or parsing files. They reduce redundancy and simplify script maintenance.

Classes and Object Templates

Classes define blueprints for objects that bundle attributes and methods. This enables modular designs where scanning, exploitation, and reporting are separate components.

Libraries and Dependencies

Libraries provide prewritten code that saves time, such as Scapy for packet crafting. However, they introduce dependencies that must be trusted and maintained securely.

Exam Focus

- Know the role of functions vs procedures.
- Understand how classes support modular pentesting tools.
- Recognize that libraries speed development but add supply-chain considerations.

Section 11. Modifying Scripts

10 Lessons | Exam Objectives Covered: 2.3| Does not include all demonstrations

Modifying Scripts (OBJ 2.3)

You do not have to be a full-time developer to succeed on PenTest+ or to be effective in the field, but you do need to be able to take an existing script and tune it for the job at hand. On an engagement, you rarely get a perfect, ready-made tool; instead you inherit small scripts from teammates, public repos, or past projects, and then you adjust inputs, outputs, filters, or logic so the tool matches the current scope. This lesson frames script modification as a disciplined process: identify the language and entry points, trace the flow of data, decide what to change, and validate that your change behaves as intended.

What 'modify scripts' means in a penetration-testing scenario

In practice, modifying a script usually means changing one of four things: what targets it reads, what actions it performs, what it records, or how it handles errors. These changes can be as small as swapping an IP address range, adding a port list, or redirecting output to a new file, but the impact can be large because scripts run repeatedly and at scale.

The safest mindset is to treat the script like a pipeline. Inputs flow in (arguments, files, environment variables), the script transforms those inputs (loops, conditionals, external commands), and outputs flow out (screen, files, logs). When you change a script, you are altering one part of that pipeline, so you should know exactly where the input is read, where it is used, and where the results are written.

A repeatable workflow for modifying scripts without breaking them

Start by identifying the interpreter and the language cues. A Bash script advertises itself with a shebang like `#!/bin/bash`, PowerShell stands out with Verb-Noun cmdlets and `$variables`, and Python is recognizable by indentation, `def`, `import`, and `print()`. Once you know the language, find the entry points: command-line arguments, variables at the top of the file, and any hard-coded paths or URLs.

Next, trace the flow. Look for loops (`for/while`), decision points (`if/elif/else`), and the external commands or library calls that do the real work. In exam questions, the correct answer usually depends on understanding this flow at a high level rather than memorizing every flag or method.

Finally, validate the change in a controlled environment. Even simple edits can introduce subtle problems such as quoting errors, wrong variable scope, incorrect

indentation, or broken file paths. The goal is to make a minimal change that achieves the scenario requirement while preserving the script's original structure and safety controls.

Operational discipline and scope controls

Because scripts can generate significant traffic, always match the script's behavior to the rules of engagement. If the scope is limited to a specific subnet, ensure the target list, loop ranges, and any embedded tool flags stay within that boundary. If the environment is sensitive, tune timing, concurrency, and retries so you gather data without causing instability.

Treat output as evidence. Modify scripts so they produce clear, timestamped results that you can reproduce and explain later. This often means adding or improving logging, choosing consistent output formats, and ensuring error messages are captured instead of silently discarded.

Exam Focus

- Know how to identify Bash vs PowerShell vs Python by quick visual cues (shebang, \$variables/cmdlets, indentation/def/import).
- Be able to describe a script's purpose by tracing inputs -> processing -> outputs, even if you do not recognize every command flag.
- Expect scenario questions that ask you to change a target list, adjust what is scanned, or redirect output to a file for reporting.
- Look for common failure points when modifying scripts: quoting/spacing, variable scope, indentation, and hard-coded paths.

Bash Fundamentals (OBJ 2.3)

Bash is the default shell in many Unix-like environments and a common place where reconnaissance automation lives. For exam scenarios, your goal is to recognize Bash syntax, understand how variables and control flow work, and confidently predict what a short script will do. In the field, Bash is especially useful for chaining existing tools together quickly, which makes it ideal for lightweight automation during enumeration.

Script structure, the shebang, and comments

A Bash script is typically executed by the `/bin/bash` interpreter, and the conventional first line is the shebang: `#!/bin/bash`. This line tells the operating system which interpreter should run the file when you execute it as a program.

Comments begin with `#` and are ignored by Bash. In real scripts, comments are more than decoration: they explain intent, document expected inputs, and warn about

assumptions. When you are modifying a script, comments often reveal what the author expected the script to do and what can safely be changed without breaking logic.

Variables, constants, and arrays for handling targets and results

Bash variables are assigned with `name=value` and must not include spaces around the equals sign. When you read a variable, you prefix it with `$`, so Bash substitutes its value. This distinction is a frequent source of bugs when students edit scripts, because adding spaces changes meaning and missing `$` prevents substitution.

Bash can enforce certain behaviors using `declare`. Declaring an integer with `declare -i` helps ensure arithmetic works as intended, while declaring read-only values with `declare -r` is a practical way to signal constant configuration data. Although Bash does not offer true immutability guarantees across all patterns, read-only variables reduce accidental edits in longer scripts.

Arrays are a common way to store port lists, usernames, or sets of targets. Indexed arrays are referenced with `${array[index]}` and use zero-based indexing, which is consistent with many other languages. Associative arrays, declared with `declare -A`, allow key-value storage for simple structured data, such as mapping a hostname to a scan file path or storing a small set of metadata about a host.

Conditionals, loops, and input/output in automation

Bash uses test expressions inside `[ ]` or `[[ ]]`. Numeric comparisons often rely on operators like `-eq`, `-ne`, `-lt`, and `-gt`, while string comparisons use `=` or `!=`. Understanding which operators apply to which data type matters because Bash treats many values as strings unless you explicitly handle arithmetic contexts.

Loops drive enumeration. A `for` loop commonly iterates over a sequence, a list of ports, or lines in a file, while `while` and `until` loops repeat based on a condition that changes over time. When a script scans a /24 network by looping 1 through 254, that loop logic is the core behavior you should be able to explain on the exam.

Input and output patterns define where data comes from and where it is stored. `echo` prints to the console, `read` accepts user input, and redirection operators (`>`, `>>`, `2>&1`) control whether output overwrites, appends, or captures errors. When you modify scripts for reporting, output redirection and consistent file naming are often the most valuable changes you can make.

Exam Focus

- Recognize the Bash shebang and comment syntax, and know that spaces around `'='` break variable assignment.

- Be able to explain how Bash reads \$1, \$2, etc. as command-line arguments and how scripts validate required input.
- Understand the difference between numeric and string comparisons in Bash and which operators are used for each.
- Know what >, >>, and 2>&1 do when a script writes scan results to a file.

Understanding a Bash Script (OBJ 2.3)

Script interpretation questions often look intimidating because they include long tool commands, but the exam usually tests whether you can explain the script's purpose and flow. In this lesson, you practice reading a Bash script that automates scanning by loading targets from a file, iterating through them, running a scan command per target, and storing results in a predictable location.

Identifying inputs, validating arguments, and defining working directories

A common Bash pattern is to test whether the user provided an argument by checking \$1. If \$1 is empty, the script prints a usage message and exits. This guardrail prevents the script from running with missing inputs and gives the user a clear execution pattern, such as ./scan.sh targets.txt.

Once validated, the script assigns \$1 into a named variable like TARGETFILE. This improves readability and reduces repeated references to \$1. A second early decision is choosing where results are stored, such as defining a REPORTDIR like /var/log/nmap. This directory becomes part of the script's output contract: every target scanned results in a file written under that path.

Tracing the loop and understanding per-target execution

The script reads targets using a loop pattern like for TARGET in \$(cat \$TARGETFILE). Each loop iteration assigns a line from the file into the TARGET variable and executes the body once. From a reasoning perspective, this means the script scales linearly with the number of entries in the target file, and the number of scan executions equals the number of targets.

Inside the loop, a status message such as echo "Scanning \$TARGET ..." is not just cosmetic. It is a simple progress indicator that helps you associate output files with runtime behavior, which is useful both for troubleshooting and for evidence collection.

The core command is typically a tool invocation with many flags. For exam purposes, focus on the big picture: the script runs an automated scan against each target and records output to a uniquely named file. Even if you do not recall every flag, you can still

infer intent from patterns like output-to-file options, timing templates, and script categories.

Interpreting output handling, error capture, and scan 'loudness'

Many automation scripts combine tool output redirection with explicit output options. When you see a command that writes to a file and also appends with `>> plus 2>&1`, the author is attempting to capture both standard output and error messages in one place. For testers, this matters because errors like DNS failures or permission problems are part of the evidence trail and often explain gaps in results.

Long scanning commands may include both discovery probes and deeper enumeration features. This combination can be effective for thoroughness but may be noisy. From a script-reading standpoint, the key is to recognize that the script is designed for broad reconnaissance: it aims to identify live hosts, enumerate ports and services, and gather additional information using safe script categories.

When you modify a script like this, the typical changes involve narrowing scope (fewer ports/targets), adjusting output formats, or changing timing to fit the environment. Those edits preserve the core loop while aligning behavior with the engagement's requirements.

Exam Focus

- Be able to explain what `$1` represents and how argument checks control whether the script runs.
- Recognize the pattern of reading a target file and looping one scan per line, producing one output file per target.
- Identify where output is written and how redirection can capture errors (`2>&1`) for troubleshooting and reporting.
- When a scan command is long, focus on the script's purpose rather than memorizing every flag.

Modifying a Bash Script (OBJ 2.3)

Modifying scripts is often about transforming a basic workflow into something more useful for the current scenario. In this lesson, a simple ping sweep script becomes a more actionable reconnaissance tool by replacing the reachability check with a targeted port scan. The key skill is understanding which lines drive the logic and which lines can be swapped without changing the structure of the program.

Understanding the baseline behavior of a ping sweep

A ping sweep script typically accepts a network prefix as its input, validates that input is present, and then loops through host numbers in a `/24` range. The script constructs

each full IP address by concatenating the prefix with the current loop counter, producing addresses like 192.168.1.1 through 192.168.1.254.

The ping command usually runs with a small count and a short timeout to keep the sweep fast. Output is often redirected to /dev/null so the terminal stays readable. The script then checks the exit status (\$?) to decide whether the host responded. This pattern teaches a core automation idea: run a command, check whether it succeeded, and then branch behavior based on the result.

Identifying the safest modification points

When you modify a script, you want to change as little as possible. In a ping sweep, the loop structure and input validation are valuable and should remain intact. The most natural modification point is the command inside the loop that represents the 'action' being applied to each host.

Replacing ping with another tool changes what evidence is collected without rewriting the entire script. This is a standard exam pattern: take an existing loop, change the command it runs, and redirect output so results can be reviewed later.

Extending the script from reachability to reconnaissance

By replacing the ping command with a targeted port scan, the script shifts from simply identifying live hosts to identifying services worth investigating. Scanning common ports such as 22, 80, and 443 yields immediately actionable findings because these ports often map to remote administration and web services.

A practical improvement is to structure output so you can correlate each host with its scan results. Even without complex parsing, consistent per-host headers, timestamps, and a predictable output file name can turn a terminal demo into usable engagement evidence.

The same pattern can be extended further. Once you understand the loop, you can adjust the port list for the scenario, add a flag to control scan speed, or write results to a report directory. The point is not the exact flags, but the modification approach: preserve control flow, change one behavior, and validate the result.

Exam Focus

- Expect modifications that replace a command inside a loop while keeping input checks and loop ranges intact.
- Be able to explain how a /24 sweep is implemented by iterating 1..254 and constructing IP addresses from a prefix.

- Know how scripts use exit codes (\$?) to branch on success/failure, and how redirection keeps output readable.
- Understand why targeted port scanning produces more actionable results than reachability checks alone.

PowerShell Fundamentals (OBJ 2.3)

PowerShell is the dominant scripting environment on Windows and is common in both administration and post-exploitation workflows. You must be able to read and modify PowerShell scripts that gather host information, query logs, or enumerate configuration settings. The distinguishing feature of PowerShell is that it works with objects rather than plain text, which shapes how data flows through pipelines.

Comments, readability, and PowerShell help conventions

PowerShell uses `#` for single-line comments and supports multi-line comment blocks using `<#` and `#>`. These comment blocks are frequently used to document a script's purpose, describe parameters, and provide examples of usage.

In real environments, well-documented scripts are easier to modify safely because the author often states assumptions about permissions, required modules, or expected output formats. When you are reading a script for the exam, comments can also reveal the high-level goal before you interpret the command sequence.

Variables, types, arrays, and hashtables

PowerShell variables begin with `$` and are assigned using `=` with flexible spacing. Typed variables such as `[int]$i` or `[string]$name` help constrain values and reduce ambiguity when scripts perform comparisons or calculations.

Arrays are created using `@()` and indexed starting at 0, making patterns like `$array[1]` common for selecting the second item. Hashtables, created using `@{}`, provide key-value storage and are often used to structure related information such as names and numbers, target metadata, or configuration settings.

Because PowerShell returns objects, hashtables and custom objects are frequently used to produce clean output. When you see `[PSCustomObject]`, the script is likely formatting results for easier viewing, sorting, or exporting.

Conditionals, loops, pipelines, and input/output

PowerShell comparisons use operators like `-eq`, `-ne`, `-lt`, and `-gt`. These appear inside if statements that use parentheses for conditions and curly braces for code blocks.

Branching patterns like `if / elseif / else` are especially common in scripts that choose different behavior based on OS version or environment.

Loops include `for`, `do-while`, and `do-until`. A `for` loop's initialization, condition, and increment make it easy to reason about how many times a block will run. `Do-while` and `do-until` guarantee at least one execution, which can matter when a script must attempt an action before it decides whether to continue.

PowerShell's pipeline passes objects from one cmdlet to another. Commands like `Select-Object`, `Sort-Object`, and `Out-File` commonly appear at the end of a pipeline to shape output. When you modify a script, changing the selected fields, sort order, or output destination is often a safe and effective way to meet a scenario requirement.

Exam Focus

- Recognize PowerShell by Verb-Noun cmdlets (`Get-Date`, `Select-Object`) and `$variables`.
- Know the common comparison operators (`-eq`, `-ne`, `-lt`, `-gt`) and how they appear inside `if (...) { ... }` blocks.
- Understand arrays (`@()`) vs hashtables (`@{}`) and how scripts use `[PSCustomObject]` for clean, sortable output.
- Be able to explain the pipeline concept and how `Out-File` or redirection writes results for reporting.

Understanding a PowerShell Script (OBJ 2.3)

PowerShell scripts used in enumeration often follow a predictable pattern: collect context, branch based on what you found, and then query for high-value security settings. In this lesson, you interpret a script that timestamps execution, identifies the host, determines the Windows version, enumerates logged-in users and drive mappings, and checks whether SMBv1 is enabled.

Collecting context: timestamps, host identity, and OS information

When a script captures the current date early, it is establishing auditability. The timestamp can be printed, written to a report, or used to name output files so results can be correlated to a specific execution.

Pulling the computer name from environment variables like `$env:COMPUTERNAME` is a standard enumeration step. It ensures the script reports on the correct host and supports consistent labeling when results are aggregated from multiple systems.

Scripts often query OS information using WMI classes such as `Win32_OperatingSystem`. The resulting object can be filtered and cleaned to produce a normalized OS version string that can be compared in subsequent logic.

Branching logic to handle different Windows versions

Conditional logic is frequently used to choose different enumeration commands based on OS version because Windows tooling and output formats can differ between releases. In an exam scenario, you should be able to describe this as 'the script detects OS version and then runs the correct commands for that OS.'

The practical takeaway is that branching reduces errors. Instead of running a command that fails on older systems, the script selects an appropriate method, which improves reliability and produces more consistent results across heterogeneous environments.

Extracting actionable findings: drives, users, and SMBv1 status

Enumerating mapped drives and PowerShell drives can reveal data access paths and potential lateral movement opportunities, such as shared drives that contain scripts, credentials, or deployment artifacts. When a script uses Select-Object to narrow output fields, it is prioritizing what is useful for the operator.

Checking SMBv1 is a classic security configuration test because SMBv1 is legacy and associated with high-impact vulnerabilities. By reading a registry value under HKLM and evaluating its setting, the script turns a low-level configuration detail into a clear statement: enabled or disabled.

For analysis, the key is to interpret what the script is trying to answer: what system is this, what users and resources are active, and is a risky protocol enabled. These are the kinds of high-value questions that enumeration scripts aim to answer quickly after access is obtained.

Exam Focus

- Be able to identify context collection (Get-Date, \$env:COMPUTERNAME, Win32_OperatingSystem) as enumeration setup steps.
- Recognize that if/elseif branches often exist to run different commands depending on Windows version.
- Know that registry queries (HKLM:) are commonly used to check security settings such as SMBv1 enablement.
- Describe the script's purpose at a high level: post-access Windows enumeration and configuration checking.

Modifying a PowerShell Script (OBJ 2.3)

Modifying PowerShell scripts often involves improving filtering, adding fields to output, or enhancing error handling so the script can run reliably across different permission levels. In this lesson, a process enumeration script is analyzed and then extended

conceptually to identify which user-owned processes may be running with elevated privileges.

Baseline process enumeration and ownership filtering

The script begins by defining a target username in a variable. This design choice makes the script easy to adapt because changing one line changes the filter for the entire enumeration run.

`Get-WmiObject Win32_Process` retrieves process objects with properties such as name, process ID, and working set size. Because ownership is not always directly exposed as a simple property, the script calls a method like `GetOwner()` per process to retrieve the associated user and domain.

Filtering occurs with a conditional that checks both that owner information exists and that it matches the target username. When a match occurs, the script emits a structured object with exactly the fields the tester cares about, which is more useful than dumping the full WMI object.

Reliability improvements: try/catch and clean, sortable output

Ownership queries can fail for protected processes or due to insufficient permissions, which is why the script wraps `GetOwner()` in a try/catch block. This is a professional scripting pattern: handle expected failures gracefully so the script continues collecting partial results instead of crashing.

By constructing `[PSCustomObject]` output, the script creates consistent records that can be sorted, filtered, or exported. Sorting by memory usage is a practical triage enhancement because it brings high-activity processes to the top, which can be helpful when you are prioritizing what to inspect next.

Adding an elevation signal to prioritize follow-up actions

A natural enhancement is to add an `'IsElevated'` field so the operator can identify processes that may be running with administrative privileges. This is valuable because the same application can have dramatically different impact depending on whether it is running under standard user or elevated context.

In PowerShell, determining elevation typically requires inspecting the process token rather than relying on WMI properties. From a modification standpoint, the important idea is to add one computed field to the custom object and then preserve the rest of the script: the filter stays the same, the output becomes more informative, and the results remain sortable.

When you apply this approach on the exam, focus on where to insert the enhancement. You add logic after ownership is confirmed but before the object is output, so you compute elevation only for relevant processes and keep runtime overhead reasonable.

Exam Focus

- Be able to describe what `Win32_Process` provides and why `GetOwner()` is used to map processes to users.
- Recognize `try/catch` as error handling that prevents script failure when ownership cannot be queried.
- Know how `[PSCustomObject]` enables clean output and `Sort-Object` organizes results for analysis.
- Expect scenario edits that add a new output field (for example, an elevation indicator) without changing the script's core filtering logic.

Python Fundamentals (OBJ 2.3)

Python is widely used in penetration testing because it is readable, cross-platform, and supported by a rich ecosystem of libraries for networking, parsing, and automation. The key is recognizing Python syntax quickly and understanding how functions, variables, control flow, and library calls combine to accomplish a task. This lesson focuses on the Python patterns you are most likely to see in reconnaissance and enumeration scripts.

Syntax cues: indentation, blocks, and comments

Python uses indentation to define code blocks, which makes whitespace meaningful. Unlike languages that rely on braces or keywords to close blocks, Python expects consistent indentation, and misalignment triggers runtime errors that stop execution.

Comments begin with `#` and are ignored by the interpreter. Because Python does not have a dedicated multi-line comment syntax, scripts often use multiple `#` lines for longer explanations. Triple-quoted strings are commonly used as docstrings to document functions and recognizing them helps you distinguish documentation from runtime behavior.

Variables, types, type casting, and constant conventions

Python variables are created by assignment and do not require explicit type declarations. This makes scripts concise, but it also means you must infer types from context, such as whether values are quoted (strings) or unquoted (numbers).

Type casting functions like `int()`, `float()`, and `str()` are used when scripts convert input into a form suitable for arithmetic, comparisons, or string formatting. In reconnaissance scripts, casting commonly appears when parsing port numbers, timeouts, or numeric response codes.

Python does not enforce true constants, so uppercase names like `PI` are a convention used to signal 'do not change.' On an exam, you should interpret uppercase configuration values as fixed parameters even though Python technically allows reassignment.

Data structures, slicing, and control flow for automation

Lists and dictionaries are core containers. Lists store ordered collections such as discovered hosts or ports, while dictionaries store labeled values such as host metadata, scan results, or parsed JSON fields. Understanding indexing and key lookup is essential for predicting what a script will print or return.

String slicing is a practical parsing tool. Scripts often extract substrings from IP addresses, URLs, or log entries to normalize values or make decisions. Being comfortable with zero-based indexing and negative indices helps you read these operations quickly.

Control flow uses `if/elif/else` and loops such as `for` and `while`. In enumeration scripts, `for` loops often iterate over a list of targets or results, while `while` loops are used for retries, polling, or reading data until a condition changes. Input/output includes `print()`, `input()`, and file operations, which appear frequently in scripts that turn raw tool output into structured reports.

Exam Focus

- Recognize Python by indentation, `def`, `import`, and `print()`, and know that blocks are defined by whitespace.
- Understand type casting (`int/float/str`) and why scripts convert data before arithmetic or comparisons.
- Be able to reason about list indexing, dictionary key lookups, and string slicing in short code snippets.
- Know the common control flow patterns used in automation: `if/elif/else`, `for` loops, `while` loops, and file I/O.

Understanding a Python Script (OBJ 2.3)

Python script interpretation often comes down to identifying library usage and then describing the high-level goal of the code. In this lesson, you analyze a script that makes a web request to a simple service and prints the result. Even if you have not used the specific library before, the structure of imports, function definition, error handling, and printing reveals the script's intent.

Recognizing Python and identifying key components

Python stands out because variables are not prefixed with \$, blocks are indented, and functions are defined with `def`. The presence of print statements and import lines makes it clear the script is intended to run as an interpreted program.

When you see a networking-oriented import such as `urllib2` in older scripts, you can infer that the script will contact a URL, retrieve content, and then process or display it. On the exam, recognizing the type of library often gives you the purpose of the script before you trace every line.

Following the request flow: opener, headers, and response handling

The script creates a request 'opener' and sets a User-Agent header to mimic a standard browser. This is a common pattern because some web servers return different responses depending on client identification, and basic filters may block requests that look like automation.

The core request usually opens the target URL and reads the response body. For a service designed to return a single value as raw text, the response body becomes the script's key output, and the script stores it in a variable so it can be returned from a function and printed.

Error handling and output decisions

Try/except blocks are used to handle failures such as HTTP errors, DNS issues, or network timeouts. Instead of crashing, the script prints an error message and returns a value like `None`, which allows the calling code to decide what to do next.

At the end of the script, a conditional checks whether a result was retrieved and prints either the value or a 'not found' message. This pattern is common in reconnaissance tools because network operations are inherently unreliable, and scripts must communicate failure clearly for troubleshooting and reporting.

Exam Focus

- Identify the script's purpose by linking the imported library (`urllib/HTTP`) to a web request workflow.
- Recognize User-Agent spoofing as a technique to make requests look like a browser and avoid basic blocking.
- Be able to explain how try/except controls error handling and how `None`-style results trigger alternate output.
- Describe the script's output at a high level: it retrieves a value from a URL and prints it.

Modifying a Python Script (OBJ 2.3)

Python modification questions typically test whether you can repurpose an existing working script by changing what data it extracts and how it prints results. In this lesson, a script that retrieves a public IP address is transformed into a lightweight banner-grabbing tool that prints HTTP response headers. The key idea is that the network request stays the same, but the script returns different parts of the response object and formats output differently.

Confirming baseline behavior and identifying modification goals

A disciplined modification begins by confirming what the original script does. If the script requests a URL and prints a single value, it is likely decoding the response body and treating that as the primary output.

The new goal changes the definition of 'useful output.' Instead of the body, you want metadata about the server response. HTTP headers provide that metadata and are often used in reconnaissance to identify server software, content handling, and caching behavior.

Refactoring the function: return the right data, not just different text

Renaming a function is a readability improvement that reduces confusion for future users, but the functional change comes from what the function returns. In the original version, the function decodes response data into a string. In the modified version, it returns the response object or its headers field so the caller can iterate over key-value pairs.

This is a common exam pattern: the function already works, so you do not rewrite it from scratch. You change one or two lines so the data you need is preserved and accessible, then you adjust the return statement accordingly.

Updating output logic to enumerate headers cleanly

Once the function returns headers, the main script updates its variable names, changes the target URL to a standard website, and prints a clear title line so the user knows what the output represents.

The key output change is a loop over the header collection, printing each header name and its value on its own line. This is an example of turning a single-value script into a reporting script by changing output formatting while keeping the underlying request workflow intact.

A professional consideration is validation. Even in a lab, you should ensure the script handles request failures gracefully and prints an explicit error message when headers cannot be retrieved, preserving the original script's error handling pattern.

Exam Focus

- Expect questions that repurpose scripts by changing what field is returned (body vs headers) and how output is looped and printed.
- Know that naming changes improve clarity, but the real modification is returning the correct object or data structure for the new task.
- Be able to describe header enumeration as a reconnaissance technique and explain why it is useful for banner grabbing.
- Recognize that error handling patterns should be preserved when modifying network-request scripts.

Section 12. Analyzing Scans

11 Lessons | Exam Objectives Covered: 3.1, 4.1 | Does not include demonstrations

Analyzing Scans (OBJ 3.2, 4.1)

Analyzing Scans builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Interpreting Scan Output

Analyzing scans is the process of separating real attack opportunities from background noise. During reconnaissance and vulnerability discovery, tools generate huge amounts of output, but penetration testers must determine which findings represent exploitable weaknesses. This requires understanding context, recognizing scanner limitations, and focusing on results that directly support attack planning.

Objectives 3.2 and 4.1 Alignment

PenTest+ Objective 3.2 expects you to interpret results from scanning and enumeration, while Objective 4.1 expects you to prioritize and prepare attacks based on those results. This means you must translate technical scan data into a realistic exploitation path rather than treating reports as final answers.

Why Analysis Drives Exploitation

Scan analysis is not passive reporting. It is an offensive decision-making process. Every open port, outdated service, or misconfiguration becomes part of a target profile. Correct interpretation allows testers to chain weaknesses into an attack sequence while avoiding time wasted on false positives.

Exam Focus

- Recognize how scan analysis supports attack planning.
- Understand validation, prioritization, CVSS, EPSS, and documentation.
- Apply safe and ethical testing practices within scope.

Positive and Negative Results (OBJ 3.2)

Positive and Negative Results builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Understanding the Confusion Matrix

Scan outcomes are categorized as true positives, true negatives, false positives, and false negatives. A true positive means the scanner correctly identified a real

vulnerability. A false positive means the scanner reported an issue that does not actually exist. False negatives are the most dangerous because real vulnerabilities remain undetected.

Operational Impact of False Positives

False positives waste engagement time and reduce trust in automated tools. Pentesters must manually verify suspicious findings before attempting exploitation. Organizations also lose confidence when remediation efforts target issues that are not real.

Reducing False Negatives

False negatives occur when scanners miss vulnerabilities due to outdated signatures, limited scan scope, or defensive filtering. Combining automated scanning with manual validation and multiple tools helps reduce blind spots and ensures true weaknesses are not overlooked.

Exam Focus

- Recognize how scan analysis supports attack planning.

Validating Scan Results (OBJ 3.2)

Validating Scan Results builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Manual Verification Techniques

Validation is the process of confirming scanner findings through direct testing. For example, if a scan reports an outdated Apache version, a tester should verify the banner, check installed packages, or confirm service behavior to ensure exploitability.

Reconnaissance and Enumeration Correlation

Validation also applies to recon and enumeration data. Email addresses, subdomains, and services discovered through OSINT must be cross-checked for accuracy. Confirmed data strengthens later phishing, exploitation, or lateral movement planning.

Safe Validation Practices

Validation must remain controlled and non-destructive. Rate limiting, safe NSE scripts, and explicit authorization prevent outages. Pentesters validate enough to prove risk without causing disruption or exceeding scope.

Exam Focus

- Apply safe and ethical testing practices within scope.

Using CVEs and CVSS (OBJ 4.1)

Using CVEs and CVSS builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

CVE Identification and Databases

CVEs provide standardized identifiers for publicly known vulnerabilities. Penetration testers use CVE and NVD records to confirm scanner results, research exploit details, and map findings to remediation guidance.

CVSS Scoring Breakdown

CVSS assigns severity scores based on exploitability and impact. Base metrics describe attack vector, complexity, privileges required, user interaction, and confidentiality/integrity/availability impact. Understanding the vector string allows testers to quickly interpret risk.

Threat Intelligence and Advisories

Beyond CVSS, resources like CISA's KEV catalog highlight vulnerabilities actively exploited in the wild. MITRE CWE explains underlying weakness patterns, while CAPEC describes common attack methods. These sources help testers prioritize realistic exploitation paths.

Exam Focus

- Understand validation, prioritization, CVSS, and documentation.

Exploit Prediction Scoring System (OBJ 4.1)

Exploit Prediction Scoring System builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

EPSS vs CVSS

CVSS measures severity, but EPSS predicts likelihood of exploitation. A vulnerability may be critical in theory but unlikely to be attacked, while another with moderate severity may be actively exploited.

Probability-Based Prioritization

EPSS scores update daily based on exploit activity, proof-of-concept availability, and threat intelligence. Testers can combine EPSS with asset value and CVSS to determine what should be attacked or patched first.

Operational Benefits

EPSS supports evidence-based remediation decisions. It improves communication with stakeholders by showing which vulnerabilities represent immediate real-world risk rather than theoretical impact.

Exam Focus

- Understand validation, prioritization, EPSS, and documentation.

Target Prioritization (OBJ 4.1)

Target Prioritization builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

High-Value Asset Identification

Pentesters prioritize systems that directly support business operations, such as authentication servers, financial databases, or customer-facing applications. Exploiting these assets often produces the greatest impact.

End-of-Life Systems

EOL systems lack vendor patches and are highly exploitable. Attackers actively seek unsupported platforms because known vulnerabilities remain permanently available.

Default Configurations

Default credentials, unnecessary services, and insecure settings provide low-effort entry points. Testing against defaults is a common first step because it often leads to rapid compromise.

Exam Focus

- Understand EOL systems and why they are vulnerable.

Common Target Criteria (OBJ 4.1)

Common Target Criteria builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Running Services Enumeration

Services define the attack surface. Pentesters enumerate exposed ports, identify service versions, and correlate them with CVEs and exploit probability signals.

Vulnerable Cryptography

Weak encryption protocols, deprecated ciphers, and improper certificate management can enable interception, downgrade attacks, or credential theft. These issues often represent high-impact systemic weaknesses.

Defensive Capabilities

Detection maturity influences how aggressively testers proceed. IDS/IPS, SIEM coverage, and logging gaps determine where attackers may remain stealthy and where defenders can respond effectively.

Exam Focus

- Understand why weak cryptography is a vulnerability.

Scan Validations (OBJ 3.2)

Scan Validations builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Configuration Validation

Scan validation ensures targets, credentials, scope, and policies are correctly configured. Misconfigured scans lead to incomplete coverage and unreliable results.

Authenticated vs Unauthenticated Scans

Authenticated scans provide deeper insight into patch status and configuration weaknesses, while unauthenticated scans simulate an external attacker's view. Both are needed for completeness.

Completeness Checks

Review scan logs, exclusion lists, and tool discrepancies to ensure no systems are missed. Comparing outputs across scanners reduces false negatives.

Exam Focus

- Recognize how scan analysis supports attack planning.

Capability Selection (OBJ 3.2, 4.1)

Capability Selection builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Tool Selection

Capability selection begins with choosing tools suited to the environment: Nmap for discovery, Nessus/OpenVAS for vulnerability analysis, Burp Suite for web testing, and wireless tools where appropriate.

Exploit Customization

Exploits often require modification to match target versions, bypass defenses, or improve reliability. Pentesters must analyze exploit code and adjust payloads responsibly.

Public Exploit Vetting

Public exploit code must be verified in controlled labs before use. Outdated or unstable exploits can cause outages or fail unexpectedly, so careful testing and documentation are essential.

Exam Focus

- Recognize how scan analysis supports attack planning.
- Understand validation, prioritization, CVSS, EPSS, and documentation.
- Apply safe and ethical testing practices within scope.

Documenting the Attack (OBJ 4.1)

Documenting the Attack builds core penetration testing skills for analyzing scan data and preparing exploitation paths.

Attack Path Documentation

Attack documentation is a reproducible chronological record of actions, including tools, commands, timestamps, outputs, and artifacts. This ensures findings can be validated and remediated.

Low-Level Diagrams

Diagrams visualize exploited hosts, services, and lateral movement paths. They help engineering teams understand technical exposure and remediation priorities.

Storyboards for Stakeholders

Storyboards provide high-level narratives showing business impact. They translate technical exploitation into clear decision-making context for leadership.

Exam Focus

- Recognize how scan analysis supports attack planning.

- Understand validation, prioritization, CVSS, EPSS, and documentation.
- Apply safe and ethical testing practices within scope.

Other Attack Considerations (OBJ 4.1)

Other Attack Considerations builds core penetration testing skills by analyzing scan data and preparing exploitation paths.

Scope Limitations

Scope defines legal and ethical boundaries. Testing outside scope can cause outages, legal exposure, and loss of trust. Always confirm authorization before proceeding.

Labeling Sensitive Systems

Critical systems containing PII or financial data require extra caution. Clients may restrict testing or require replica environments to avoid harm.

Managing Dependencies

Modern environments rely on interconnected services. Understanding dependencies prevents indirect outages and ensures testing remains accurate and safe.

Exam Focus

- Understand how dependencies affect vulnerabilities.

Section 13. Discovering Vulnerabilities

9 Lessons | Exam Objectives Covered: 3.1| Does not include demonstrations

Discovering Vulnerabilities (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Vulnerability Discovery as Offensive Reconnaissance

Discovering vulnerabilities is the penetration tester's process of identifying weaknesses before exploitation. Instead of reacting after an incident, offensive security focuses on proactively locating blind spots in systems, applications, and infrastructure that an adversary could leverage. This phase shapes realistic attack paths and ensures testing aligns with how real attackers approach environments.

Broad Attack Surface Coverage

Objective 3.1 requires testers to conduct vulnerability discovery across multiple technology domains. This includes applications, host systems, networks, mobile platforms, cloud configurations, industrial systems, and wireless environments. Each domain introduces unique weaknesses, meaning discovery techniques must adapt.

From Findings to Exploitation Planning

The purpose of discovery is not only detection, but prioritization. Penetration testers must connect discovered issues to realistic exploitation chains, validate high-risk weaknesses, and document results so remediation can be applied effectively.

Exam Focus

- PenTest+ Objective 3.1 requires applying the correct vulnerability discovery technique for each target type.
- Expect scenario questions comparing scanning approaches (DAST vs IAST, authenticated vs unauthenticated, TCP vs UDP).
- Know how discovery results translate into attack prioritization and exploitation planning.

Application Scanning (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Dynamic Application Testing Mindset

Application scanning evaluates live applications the way an attacker would, by manipulating inputs and observing responses. Web applications are frequent targets because they expose interfaces directly to users and the internet.

DAST: External Runtime Scanning

Dynamic Application Security Testing (DAST) probes applications from the outside, sending crafted payloads to detect issues like SQL injection, XSS, authentication flaws, and insecure session handling. Tools such as Burp Suite and OWASP ZAP automate discovery but require manual validation to reduce false positives.

IAST: Internal Execution Visibility

Interactive Application Security Testing (IAST) instruments applications internally, tracking data flows and code execution in real time. This provides stronger context than DAST alone, identifying exactly where vulnerabilities originate inside the codebase.

Exam Focus

- Apply the correct vulnerability discovery technique for each target type.
- Expect scenario questions comparing scanning approaches (DAST vs IAST).

Software Analysis (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Static Insight into Software Weaknesses

Software analysis focuses on understanding how applications are built, what dependencies they include, and where unsafe logic exists before runtime. This gives penetration testers deeper awareness of exploit opportunities.

SCA: Third-Party Dependency Risk

Software Composition Analysis (SCA) inventories open-source libraries and compares them to vulnerability databases. Because modern software heavily relies on third-party components, outdated libraries can introduce severe risks.

SAST: Developer Code Weakness Detection

Static Application Security Testing (SAST) analyzes custom code for insecure patterns such as injection flaws, buffer overflows, or improper API usage. It provides precise file and line-level findings but may require manual review.

Exam Focus

- Know how discovery results translate into attack prioritization and exploitation planning.

Host-Based Scanning (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Host-Level Vulnerability Discovery

Host-based scanning examines individual systems for patch gaps, misconfigurations, weak permissions, and exposed services. A single compromised host often becomes the pivot point for deeper compromise.

Authenticated vs Unauthenticated Perspectives

Credentialed scanning reveals internal weaknesses such as missing patches or weak passwords, while unauthenticated scanning simulates an external attacker identifying exposed services and vulnerable versions.

Secrets Scanning for High-Impact Exposure

Secrets scanning detects exposed API keys, passwords, and credentials stored improperly. These findings often lead directly to privilege escalation or lateral movement opportunities.

Exam Focus

- Expect scenario questions comparing scanning approaches (authenticated vs unauthenticated).

Network Scanning (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Network Reconnaissance Foundations

Network scanning identifies open ports, services, and reachable hosts. It forms the basis of attack surface mapping and helps testers locate entry points.

TCP and UDP Scanning Techniques

TCP scans leverage handshake behavior to confirm open ports, while UDP scans rely on indirect responses such as ICMP errors. UDP scanning is slower and less predictable but critical for discovering services like DNS or SNMP.

Stealth Scanning and Evasion

Stealth techniques such as SYN scans, NULL scans, fragmentation, and idle scans reduce detection by IDS/IPS systems and help testers gather intelligence quietly.

Exam Focus

- Expect scenario questions comparing scanning approaches (TCP vs UDP).

Mobile Scanning (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Mobile Attack Surface Complexity

Mobile applications integrate local storage, sensors, APIs, and backend services, making them unique targets requiring tailored scanning approaches.

Static Mobile Analysis (SAST)

Mobile SAST decompiles APK/IPA packages to uncover hardcoded secrets, insecure logic, and embedded endpoints. Tools like MobSF and JADX support this.

Dynamic Testing and Permission Review

Runtime testing with proxies and instrumentation tools reveals insecure API calls, certificate pinning weaknesses, and authorization flaws. Permissions analysis identifies overprivileged apps and exported components that attackers can abuse.

Exam Focus

- Expect scenario questions comparing scanning approaches (DAST vs SAST).

Scanning IaC (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

IaC as Systemic Risk Amplifier

Infrastructure as Code defines entire cloud environments through templates, meaning misconfigurations can replicate at scale.

Static Template Analysis

Static IaC scanning identifies insecure defaults such as public S3 buckets or overly permissive security groups using tools like TFLint.

Policy Enforcement and Drift Detection

Policy-as-code prevents unsafe deployments, while drift detection identifies when real infrastructure diverges from secure templates, exposing new weaknesses.

Exam Focus

- Explain Policy-as-code.

ICS Vulnerability Discovery (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

ICS Environments as High-Risk Targets

Industrial Control Systems support critical infrastructure and often rely on legacy protocols, making them fragile and uniquely vulnerable.

Manual and Configuration-Based Discovery

ICS testing emphasizes manual review and low-impact assessment because aggressive scanning can disrupt operations.

Passive Monitoring with Port Mirroring

Port mirroring enables traffic analysis without interfering with sensitive processes, helping identify unauthorized commands or protocol misuse.

Exam Focus

- Explain the reasons that ICS testing is different than network testing.

Wireless Scans (OBJ 3.1)

This lesson explains the key penetration testing concepts and techniques relevant to vulnerability discovery.

Wireless Reconnaissance Importance

Wireless networks broadcast traffic openly, making scanning essential for identifying attackable networks and rogue access points.

SSID and Channel Enumeration

SSID scanning maps nearby networks, while channel scanning reveals congestion, overlap, and mismanagement that may weaken security.

Weak Encryption and Misconfiguration Discovery

Scanning uncovers insecure authentication, outdated encryption, and segmentation failures that attackers can later exploit using Aircrack-ng tools.

Exam Focus

- Be prepared for scenario-based questions involving SSID discovery, channel mapping, and signal analysis, often using tools like Kismet, airodump-ng, or Wireshark.
- Know what wireless scan results reveal, such as weak encryption (WEP), misconfigured WPA/WPA2 settings, open networks, or poor segmentation that can lead to follow-on attacks.

Section 14. Vulnerability Discovery Tools

7 Lessons | Exam Objectives Covered: 3.1| Does not include demonstrations

Vulnerability Discovery Tools (OBJ 3.1)

This lesson covers Vulnerability Discovery Tools as a vulnerability discovery tool used in offensive security assessments.

Tool-Based Vulnerability Discovery

Tool-Based Vulnerability Discovery is a key capability of Vulnerability Discovery Tools. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Offensive Reconnaissance Acceleration

Offensive Reconnaissance Acceleration is a key capability of Vulnerability Discovery Tools. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Tool Output Interpretation for Exploitation

Tool Output Interpretation for Exploitation is a key capability of Vulnerability Discovery Tools. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Recognize vulnerability discovery tools and how their findings translate into exploitable attack paths.

Trivy (OBJ 3.1)

This lesson covers Trivy as a vulnerability discovery tool used in offensive security assessments.

Filesystem and Secrets Scanning

Filesystem and Secrets Scanning is a key capability of Trivy. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Container Image Vulnerability Enumeration

Container Image Vulnerability Enumeration is a key capability of Trivy. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Repository and Dependency Discovery

Repository and Dependency Discovery is a key capability of Trivy. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Know how Trivy identifies CVEs, misconfigurations, and secrets across containers and repositories.

BloodHound (OBJ 3.1)

This lesson covers BloodHound as a vulnerability discovery tool used in offensive security assessments.

Active Directory Graph Mapping

Active Directory Graph Mapping is a key capability of BloodHound. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

SharpHound Data Collection

SharpHound Data Collection is a key capability of BloodHound. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Privilege Escalation Path Discovery

Privilege Escalation Path Discovery is a key capability of BloodHound. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Expect scenario questions where BloodHound reveals hidden escalation chains in AD.

PowerSploit (OBJ 3.1)

This lesson covers PowerSploit as a vulnerability discovery tool used in offensive security assessments.

Post-Exploitation PowerShell Modules

Post-Exploitation PowerShell Modules is a key capability of PowerSploit. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Credential Harvesting and Privilege Escalation

Credential Harvesting and Privilege Escalation is a key capability of PowerSploit. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Modern Detection and Tool Limitations

Modern Detection and Tool Limitations is a key capability of PowerSploit. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Recognize PowerShell-based post-exploitation tradeoffs and detection risk.

Grype (OBJ 3.1)

This lesson covers Grype as a vulnerability discovery tool used in offensive security assessments.

SBOM and Dependency Vulnerability Analysis

SBOM and Dependency Vulnerability Analysis is a key capability of Grype. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Container Image Scanning for Exploitable Packages

Container Image Scanning for Exploitable Packages is a key capability of Grype. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Filesystem Scanning After Initial Access

Filesystem Scanning After Initial Access is a key capability of Grype. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Be able to interpret Gype output and prioritize vulnerable libraries for exploitation.

Kube-Hunter (OBJ 3.1)

This lesson covers Kube-Hunter as a vulnerability discovery tool used in offensive security assessments.

Kubernetes Component Reconnaissance

Kubernetes Component Reconnaissance is a key capability of Kube-Hunter. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Cluster Misconfiguration Discovery

Cluster Misconfiguration Discovery is a key capability of Kube-Hunter. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Modern Tool Replacement and Limitations

Modern Tool Replacement and Limitations is a key capability of Kube-Hunter. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Understand how Kubernetes scanners find exposed dashboards, API servers, and insecure RBAC.

TruffleHog (OBJ 3.1)

This lesson covers TruffleHog as a vulnerability discovery tool used in offensive security assessments.

Secret Discovery in Git History

Secret Discovery in Git History is a key capability of TruffleHog. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Local and Remote Repository Scanning

Local and Remote Repository Scanning is a key capability of TruffleHog. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Credential Exploitation and Cloud Pivoting

Credential Exploitation and Cloud Pivoting is a key capability of TruffleHog. In penetration testing, this provides actionable insight into exploitable weaknesses and attack surface expansion.

Exam Focus

- Exam questions often test how leaked API keys enable rapid escalation into cloud infrastructure.

Section 15. Social Engineering Attacks

10 Lessons | Exam Objectives Covered: 4.8| Does not include demonstrations

Social Engineering Attacks (OBJ 4.8)

Social engineering attacks are offensive techniques that rely on psychological manipulation instead of software flaws. In practical engagements, they are used to create an initial foothold, obtain privileged credentials, bypass controls through persuasion, or open a pathway for technical exploitation. Because modern environments often have layered technical defenses, social engineering remains one of the highest-yield options for an attacker who wants to shortcut those layers by targeting the decision-making process of a human.

Why social engineering works in real engagements

Social engineering succeeds because organizations are built for productivity, not continuous skepticism. Users are trained to be helpful, responsive, and fast, and they operate in environments where requests arrive through familiar channels like email, chat, ticketing systems, phone calls, and face-to-face interactions. Here, an attacker exploits the gap between a user's normal workflow and the verification steps security expects. When a message appears "close enough" to legitimate, the target's brain fills in the missing certainty, especially when the request aligns with a routine task like credential validation, document review, or approving access.

Common social engineering outcomes that matter to an attacker

In a penetration test, social engineering outcomes are measured by what the attacker can do next. A successful outcome may be credential capture, session token capture, malware execution, physical access to restricted areas, or information disclosure that reduces future guesswork. Even when the immediate output is "only" a click or a reply, that interaction can be chained into deeper impact: password reuse across services, access to internal portals, OAuth consent grants, or a browser hook that enables follow-on reconnaissance. The most important mental model is that social engineering is rarely a single step—it is often the first link in a chain.

Ethical boundaries, authorization, and controlled demonstrations

Professional social engineering is always governed by explicit rules of engagement. That means defining scope, approved pretexts, allowed targets, acceptable payload types, and required safety controls before any campaign starts. In practice, the difference between a legitimate assessment and an unacceptable operation is documentation and authorization: who approved what, when, and under what

constraints. A controlled engagement also emphasizes proof-limited validation, meaning you demonstrate risk without creating unnecessary harm, such as avoiding disruptive payloads, minimizing sensitive data collection, and using safe landing pages and reporting mechanisms.

Exam Focus

- Be prepared to determine the most appropriate social engineering method for a given objective, such as credential harvesting, MFA fatigue abuse, information gathering, or physical access.
- Understand rules of engagement requirements, including written authorization, defined scope, approved pretexts, and limitations on payloads or data collection during an assessment.
- Know how social engineering results translate into technical impact, such as credential reuse, session hijacking, privilege escalation, or internal pivoting.

Methods of Influence (OBJ 4.8)

Methods of influence are the psychological levers that make social engineering reliable. In offensive operations, these levers reduce resistance by shaping how a target interprets a request, how quickly they decide, and whether they feel safe challenging it. The key is not memorizing buzzwords, but understanding which emotional state an attacker is trying to create and why that state increases compliance.

Authority and urgency as compliance accelerators

Authority works because most environments teach people to comply with requests that appear to come from someone in a position of power or from a trusted external entity. When a request sounds like it comes from someone “who has the right to ask,” targets often treat verification as optional. Urgency amplifies this effect by shortening the target’s decision window. When time feels scarce, people default to the fastest path to resolution—which is exactly where security steps are most likely to be skipped. Together, authority and urgency create a pressure pairing where the target is made to feel both obligated and rushed.

Social proof and scarcity as behavior shapers

Social proof exploits the human tendency to look for cues about what others are doing. If a request implies that peers have already complied—or that the action is standard practice—targets become less likely to question it. Scarcity creates a different kind of push by framing an opportunity as limited, triggering fear of missing out and prompting fast action. In phishing campaigns, scarcity is often expressed through limited-time

access invites or “last chance” compliance notices. Operationally, both methods replace careful evaluation with emotion-driven momentum.

Likeness and fear as emotional shortcuts

Likeness works when the attacker creates rapport or affinity that lowers skepticism through shared interests, friendly tone, or insider language. Fear pushes in the opposite direction by creating a perceived threat that must be resolved immediately, such as a security alert or account compromise warning. Fear narrows attention, making targets focus on stopping the threat rather than verifying the messenger, which is why fear-based lures remain so effective.

Exam Focus

- Recognize methods of influence (authority, urgency, scarcity, social proof, likeness, fear) in scenario-based questions and identify which psychological trigger is being leveraged to increase compliance.
- Be prepared to determine why a specific influence method would be effective for a given objective, such as accelerating credential capture, bypassing verification steps, or prompting rapid action before a target evaluates legitimacy.

Phishing Campaigns (OBJ 4.8)

Phishing campaigns are structured social engineering operations designed to obtain credentials, session access, sensitive information, or execution opportunities through deceptive communication. From an offensive testing perspective, phishing is a campaign design problem: selecting a target population, crafting a believable pretext, choosing a delivery channel, and defining what success looks like without exceeding engagement boundaries.

Broad phishing and why volume still works

Broad phishing targets many users at once and relies on probability. Even when most recipients ignore the message, a small percentage of clicks can be enough to obtain a foothold, especially if credentials are reused across multiple services. Broad phishing often uses common brand impersonation, shipping notices, password resets, or document sharing invites because these fit normal workflows. In assessment scenarios, broad phishing typically tests baseline awareness, inbox hygiene, and reporting behavior.

Targeted phishing: spearphishing, whaling, and BEC

Targeted phishing increases success by making the message feel personally relevant. Spearphishing uses role context, internal tools, projects, and naming conventions to

remove the “this looks generic” warning sign. Whaling focuses on executives or high-impact roles where a single compromise can authorize payments, approve access, or reveal strategic data. Business Email Compromise escalates risk further by abusing legitimate mailboxes to request sensitive actions, because the request originates from an authentic account and bypasses many spoofing cues. Targeted phishing is fundamentally about credibility: every accurate detail reduces the target’s need to verify.

Alternative channels: vishing and smishing

When email defenses are strong or the environment relies heavily on mobile communication, attackers shift channels. Vishing leverages voice calls to create pressure and authority through tone and conversational control, often using caller ID spoofing to appear legitimate. Smishing uses SMS to exploit the informal, high-trust nature of text messages, pushing users toward malicious links or credential prompts. Operationally, these channels matter because users often verify less on phones than they do on desktops.

Exam Focus

- Distinguish between phishing types (broad phishing, spearphishing, whaling, BEC, vishing, smishing) and recognize which delivery channel and targeting strategy is being used in a scenario.
- • Be prepared to link phishing campaign outcomes to offensive impact and engagement limits, such as credential harvesting, session access, or action-based compromise while staying within defined rules of engagement and proof-limited validation.

Gophish (OBJ 4.8)

Gophish is an open-source phishing simulation platform used to plan, deliver, and measure phishing campaigns in controlled engagements. Its value in offensive assessments is repeatability and measurable outcomes, allowing testers to quantify risk in terms of user behavior. The goal is to demonstrate how quickly an attacker could obtain credentials or clicks and which roles would most likely become the entry point.

Designing a campaign that mirrors real attacker tradecraft

A strong simulation begins with campaign design: selecting a pretext that matches organizational workflows, crafting email content that feels routine, and building a landing experience that produces measurable outcomes. In Gophish, this means creating message templates, defining a sending profile, and associating a landing page that records whether a user clicked or submitted credentials. Campaign realism comes from

alignment with normal business events like policy updates, password expirations, document requests, or support tickets, because the most effective lures feel ordinary.

Tracking outcomes: open, click, and credential submission signals

Gophish distinguishes outcomes that represent progressively deeper compromise. Email opens show visibility, clicks indicate interaction that could trigger payload delivery, and credential submissions represent direct account compromise potential. Timeline and per-user detail reveal which departments respond fastest and which roles are most susceptible. This matters because risk is not evenly distributed; one privileged credential submission can be far more damaging than many low-privilege clicks.

Chaining results into controlled follow-on demonstrations

In real attacks, a click is rarely the end state, instead it is the entry point. In a penetration test, follow-on actions must remain within scope, but testers can still demonstrate realistic chaining. A click can lead to controlled credential capture, a safe browser hook demonstration, or a scoped access attempt to a non-production test system. By documenting what the user action would enable next, such as account takeover, session hijacking, or access to internal resources, you translate “user clicked” into measurable technical risk.

Exam Focus

- Gophish is a phishing simulation and measurement platform for controlled engagements.
- Know the meaning of key metrics: open (visibility), click (interaction), credential submission (compromise potential).
- Expect scenario questions about selecting targets and interpreting campaign results by role criticality.

Impersonation (OBJ 4.8)

Impersonation is the deliberate assumption of a trusted identity to exploit default trust behaviors. It works because organizations rely on roles, like IT support, delivery services, contractors, auditors, and people are trained to treat these roles as legitimate. In offensive engagements, impersonation can be physical, voice-based, or written, but the goal is consistent: lower scrutiny by making the request feel routine and safe to comply with.

Building a credible pretext and controlling the conversation

Effective impersonation begins with a pretext that explains who you are, why you are present, and what you need in a way that fits the target environment. Credibility is

created through small details like correct terminology, references to internal systems, and a tone that matches the role. During the interaction, attackers guide attention toward completing the task rather than verifying identity, often using light pressure and friendly rapport to keep the exchange moving forward.

Physical impersonation and why roles bypass doors

Physical impersonation leverages inconsistent access enforcement. A uniform, clipboard, branded badge, or equipment case acts as a visual shortcut that signals legitimacy. Employees may hold doors or provide access because the impersonated role is expected to be present. The offensive value is that physical access converts an external engagement into internal opportunity: observation of workflows, identification of network assets, and authorized placement of assessment devices when permitted.

Elicitation as the information engine of impersonation

Elicitation extracts sensitive information through casual conversation rather than direct questioning. Attackers ask operational questions that feel normal, like who owns a system, where a device is located, which portal is used, and the target fills gaps with context. This reduces attacker uncertainty and reveals details like internal hostnames, authentication patterns, escalation routes, and the identities of high-value personnel that shape the next technical steps.

Exam Focus

- Impersonation exploits role-based trust across physical, voice, and written channels.
- A strong pretext explains identity, purpose, and request in a way that matches normal operations.
- Physical impersonation works because visual cues and social norms bypass strict verification.
- Elicitation extracts information indirectly through normal-sounding questions that reduce attacker uncertainty.
- Expect scenario questions that ask which impersonation approach best fits an environment and target behavior.

Surveillance Techniques (OBJ 4.8)

Surveillance techniques support social engineering by gathering intelligence without direct interaction. In offensive work, the goal is to learn routines, access patterns, and information leakage points so the eventual approach feels natural and is timed to moments of reduced scrutiny. Surveillance may be broad and passive, direct and observational, or focused on what organizations discard and forget to protect.

Pattern observation and environmental mapping

Broad surveillance starts with watching how the target environment behaves: badge entry habits, visitor handling, guard presence, and shift changes. Attackers look for predictable patterns because predictable patterns are exploitable. When you know when staff arrive, where they congregate, and which entrances are least monitored, you can align an impersonation or access attempt with the moments most likely to succeed. Surveillance reduces randomness and turns the engagement into a planned operation.

Eavesdropping and shoulder surfing as low-cost intelligence

Direct observation techniques extract sensitive information from normal behavior. Eavesdropping collects details revealed in conversation, such as names, internal tools, project timelines, and system issues. Shoulder surfing collects what users reveal through screens and keypads, including login prompts, badge PINs, and device identifiers. Even without advanced tooling, these techniques can provide enough context to strengthen future pretexts and reduce defensive suspicion.

Dumpster diving and post-disposal intelligence

Dumpster diving targets discarded materials that were not properly destroyed. Printed emails, network diagrams, shipping labels, badge templates, and decommissioned hardware can leak intelligence that strengthens future operations. The impact is both data exposure and operational insight: internal system names, vendor relationships, office layouts, and workflow artifacts. In legitimate assessments, recovered materials must be handled as sensitive evidence and remain within scope and local legal constraints.

Exam Focus

- Surveillance reduces attacker uncertainty by revealing routines, weak points, and timing opportunities.
- Eavesdropping collects information from conversations; shoulder surfing collects information from screens and keypads.
- Dumpster diving targets discarded materials that were not properly destroyed or sanitized.
- Expect scenario questions that connect surveillance outputs to pretext quality and attack timing.
- Treat any collected physical intelligence as sensitive evidence under the rules of engagement.

Watering Hole (OBJ 4.8)

A watering hole attack compromises a site that a specific target group naturally visits, turning routine browsing into a delivery mechanism for malware or browser-based compromise. Unlike phishing, the attacker does not need to lure the target to a new destination. Instead, the attacker positions themselves where the target already goes and waits for normal behavior to deliver victims.

Selecting the watering hole through behavioral reconnaissance

The first step is identifying where the target group gathers information, such as vendor portals, industry forums, partner sites, or niche community resources. The attacker selects a point of trust and habit, because habit produces repeat visits and trust reduces suspicion. In ethical assessments, this concept is often demonstrated through controlled environments because real third-party compromise is typically out of scope, but the attacker model is critical to understand.

Compromise mechanics: drive-by delivery and browser exploitation

Once the attacker controls site content, they inject malicious code that executes when a victim loads the page. This is often framed as a drive-by download where the victim may not need to click anything for the compromise chain to begin. The technical delivery method varies, but the social component remains constant: the destination is familiar, so users are less suspicious. This is why watering holes can bypass defenses that focus primarily on suspicious email links.

Post-compromise leverage and operational value

After initial execution, watering hole compromises can yield credentials, session tokens, or a foothold that enables lateral movement. The operational advantage is scale: one compromised site can impact many users over time. In exam scenarios, focus on the chain, meaning predictable group behavior leads to a shared compromise point, and that shared compromise point produces multiple access opportunities.

Exam Focus

- Watering hole attacks target the sites a group already trusts and visits, not individual lures.
- The technique scales because one compromised site can impact many targets over time.
- Drive-by delivery can begin on page load depending on the exploit chain, reducing the need for user interaction.

Evilginx (OBJ 4.8)

Evilginx is an attacker-in-the-middle framework used for reverse-proxy phishing that can bypass multi-factor authentication by capturing session tokens. In modern environments, MFA reduces the value of stolen passwords, so attackers increasingly target the session itself. Evilginx demonstrates this risk by making login look legitimate while collecting the authentication artifacts that matter.

Reverse-proxy phishing and why it defeats simple MFA expectations

Evilginx places itself between the victim and the real service, proxying requests and relaying responses in real time. Because the victim is effectively interacting with the real service through a middle layer, the page content and authentication flow appear normal. This normality is what makes the technique dangerous: the victim completes MFA, and the attacker captures the session token issued after authentication.

Operational flow: from lure to token capture

The attacker begins with a lure that routes the victim to the reverse-proxy site. When the victim submits credentials and completes MFA, the service issues a session token that grants access without repeated MFA prompts. Evilginx captures the token as it passes through the proxy and enables replay, allowing the attacker to become the authenticated user. The key concept is that token capture changes the problem from guessing MFA to stealing proof that MFA already succeeded.

Impact: session hijacking and account takeover

Session hijacking is powerful because it sidesteps MFA by reusing the authenticated session. With a valid token, an attacker can access email, files, internal portals, and workflows as the victim. In controlled assessments, the focus is on demonstrating risk safely and documenting session behavior, including what triggers reauthentication and which controls would reduce token replay exposure.

Exam Focus

- Evilginx is a reverse-proxy attacker-in-the-middle framework used to capture session tokens.
- MFA can be bypassed when attackers steal session tokens rather than passwords alone.
- Expect questions about token replay and session hijacking as the core risk and outcome.

Tailgating and Piggybacking (OBJ 4.8)

Tailgating and piggybacking are physical techniques used to bypass restricted entry points by leveraging an authorized person. They differ primarily in whether the authorized person is aware of the unauthorized entry. In penetration testing, these methods show how physical access can bypass strong technical controls when human behavior overrides policy at the door.

Tailgating as silent entry through routine behavior

Tailgating occurs when an unauthorized person slips in behind an authorized individual without being noticed. This works because many people do not treat doors as security checkpoints in practice, and because busy entryways and slow-closing doors create predictable windows for entry. Tailgating minimizes interaction, relying instead on routine and inattention.

Piggybacking as social compliance at the door

Piggybacking involves the authorized person knowingly allowing entry, usually due to politeness or social pressure. The attacker creates a reason for courtesy, such as carrying equipment, claiming a badge problem, or appearing late, and the target chooses convenience over verification. This demonstrates how social norms can override access control procedures even when badge systems function correctly.

Post-entry value: why physical access accelerates compromise

Once inside, the attacker's options expand: observing workflows, identifying network assets, accessing unattended systems, and placing authorized assessment devices. Physical presence also enables low-tech information gathering through screens, documents, and conversations. The key reporting point is the chain from physical entry to technical risk, showing how quickly a door failure can become internal access, credential exposure, or persistence opportunity.

Exam Focus

- Tailgating is unauthorized entry without awareness; piggybacking is entry with knowing assistance from an authorized person.
- Both exploit human behavior and routine rather than technical weaknesses in badge scanners.
- Expect questions that distinguish the two based on awareness and interaction.
- Physical access can bypass digital controls by enabling internal placement and observation.

Browser Exploitation Framework (BeEF) (OBJ 4.8)

BeEF is a browser-focused exploitation framework that turns a victim's browser into an interactive foothold for reconnaissance, social engineering, and controlled exploitation. Browsers are valuable because they process untrusted content, maintain authenticated sessions, and interact with internal and external resources continuously. When a browser is successfully hooked, the attacker gains a channel that can be used to observe, influence, and sometimes extend access beyond the original point of contact.

Why browsers are high-value targets in offensive operations

Browsers handle credentials, cookies, and session tokens while rendering complex content from many sources. They also operate from inside the user's network context, which can reveal internal details that external scanning cannot. This makes browser exploitation especially relevant when paired with social engineering, because the initial execution path is commonly a click on a link or a visit to a prepared page.

Hooking mechanics and establishing control

BeEF establishes control by delivering a small JavaScript payload that runs in the victim's browser and connects back to the BeEF command server. The hook can be delivered through a malicious page or injected into a vulnerable application through issues like XSS. Once connected, the browser appears as a managed session in the BeEF interface. The key point is that the compromise is the browser session, which can still provide meaningful capability even if the endpoint itself is not fully compromised.

Post-hook capabilities: reconnaissance, influence, and chaining

After a hook is established, BeEF can perform browser fingerprinting, gather environment details, and run modules that influence user behavior, such as presenting convincing prompts or redirecting the user. In controlled demonstrations, BeEF illustrates how small user actions can cascade into larger risk when web application security is weak. The engagement focus remains ethical: capture only necessary evidence, avoid harmful actions, and map each demonstrated capability to business impact.

Exam Focus

- BeEF controls and leverages a victim's browser session after a JavaScript hook executes.
- Browsers are valuable because they store sessions and operate from inside the user's network context.

- Hooking occurs when the victim loads a page that executes the hook script, including via XSS scenarios.

Section 16. Wireless Attacks

8 Lessons | Exam Objectives Covered: 4.7| Does not include demonstrations

Wireless Attacks (OBJ 4.7)

Wireless attacks shift the battleground from cables and switches to the air itself. In offensive work, that matters because the air is a shared medium: anyone within range can observe frames, imitate infrastructure, and influence client behavior without ever touching a physical port. A wireless assessment is therefore less about “hacking Wi-Fi” as a single trick and more about chaining reconnaissance, signal control, protocol abuse, and user deception until you have a usable foothold that maps to real business impact.

Wireless attacks as an offensive workflow

A professional wireless engagement typically begins with passive discovery, like identifying access points, client devices, encryption modes, channels, and any “helpful” features like WPS. From there, the tester decides whether the fastest path is credential recovery (capturing authentication material for offline cracking), traffic interception (becoming an on-path adversary through rogue infrastructure), or disruption to force predictable behaviors (such as reconnect events that generate handshakes). The key mindset is that wireless attacks are rarely one-step wins; they are repeatable workflows where each phase produces artifacts, such as handshakes, client lists, SSIDs, captive portal flows, tokens, or credentials that enable the next phase.

Where wireless attacks create leverage

Wireless is attractive to attackers because compromise can be both low-friction and high-yield. A single reused pre-shared key can unlock guest access, which then becomes internal pivot space if segmentation is weak. A rogue access point that looks “familiar” can harvest enterprise credentials or session tokens. Even purely passive collection can reveal inventory and weak points, such as hidden SSIDs, legacy encryption, misconfigured channels, and unmanaged IoT devices. For a penetration tester, the goal is to turn radio observations into a defensible attack narrative: how an attacker could enter, what they could access, and what controls failed along the way.

How this section maps to Objective 4.7

Objective 4.7 expects you to choose the right wireless techniques and tools given a scenario. That means understanding not just what a tool does, but when it makes sense: when passive sniffer data is enough, when you must force a handshake, when you should pivot into an Evil Twin or captive portal, and when protocol-level testing like

fuzzing is appropriate and authorized. The strongest exam answers connect technique to outcome—credential theft, session hijack, traffic manipulation, or denial of service—and explain why that technique is the best fit.

Exam Focus

- Wireless attacks often start with passive reconnaissance to map SSIDs, channels, encryption modes, clients, and convenience features (like WPS) before any active interference.
- Offensive wireless workflows commonly chain discovery → handshake capture → offline cracking or discovery → rogue AP (Evil Twin) → captive portal credential harvesting → on-path traffic manipulation.

Wireless Security (OBJ 4.7)

To attack wireless effectively, you have to understand what the protocol is trying to protect and where it fails under real-world conditions. Wireless “security” is a combination of cryptography (how frames are protected), authentication (how devices prove they should join), and convenience controls (features that make onboarding easier but often weaken defenses). From an offensive perspective, you are looking for the weakest link in that chain: weak secrets, fragile handshakes, downgrade paths, or bypassable controls.

Pre-shared keys as an attack surface

Pre-shared keys (PSKs) are simple: the same password is shared across multiple users and devices. That simplicity is exactly why they become an offensive opportunity. The more widely a PSK is distributed, the more likely it is to be reused, written down, shared with third parties, or chosen for memorability over strength. Once a PSK is known, wireless encryption does not matter—the attacker becomes a legitimate client. Even when the PSK is not known, the shared-secret model makes offline guessing feasible when the attacker can capture authentication material, because the “proof” of the password can be tested without further interaction with the network.

Encryption evolution and what it changes for the attacker

The progression from WEP to WPA/WPA2 and then WPA3 is largely a story of eliminating brittle designs attackers exploited. WEP’s weaknesses made key recovery practical with captured traffic. WPA introduced TKIP as a stopgap, but it carried forward legacy ideas that eventually became attackable. WPA2’s adoption of AES-CCMP improved confidentiality and integrity substantially, but PSK networks remained vulnerable to offline cracking when a handshake could be captured and tested against wordlists. WPA3 shifted the game again by replacing the classic PSK handshake with SAE, which is designed to resist the traditional offline cracking workflow and provides

properties like forward secrecy. In practice, WPA3 pushes offensive testers toward misconfiguration hunting, downgrade conditions, weak transition modes, and attacking adjacent trust surfaces such as captive portals, client behavior, and rogue infrastructure.

Convenience features that create bypass paths

Two “security” features repeatedly show up as offensive entry points: WPS and MAC filtering. WPS was designed to simplify onboarding, but its PIN validation structure reduces the search space enough that attackers can brute force it and recover the underlying PSK even when WPA2 is otherwise configured correctly. MAC filtering is even more deceptive: it looks restrictive, but MAC addresses are not secrets. Once an attacker observes an allowed MAC in the air, they can spoof it and attempt to join as that device. In an engagement, the presence of WPS or strict MAC filtering is a signal that the environment may prioritize convenience and legacy practices, which is exactly where wireless assessments tend to find high-impact weaknesses.

Exam Focus

- PSK-based wireless networks create offensive opportunities because shared secrets can be captured during authentication and tested offline with wordlists.
- WEP is fundamentally broken; WPA/TKIP is legacy; WPA2/AES-CCMP is strong but PSK handshakes can still support offline guessing; WPA3/SAE is designed to resist classic offline cracking workflows.
- WPS and MAC filtering are common “convenience controls” that attackers can exploit: WPS can be brute-forced to recover the PSK, and MAC filtering can be bypassed through MAC spoofing.

Wireless Signal Exploitation (OBJ 4.7)

Wireless signal exploitation is about using the physics and geography of radio to your advantage. Before you crack a password or stand up a rogue access point, you need visibility: where are signals strongest, which networks exist, which devices connect, and what patterns repeat throughout the day. Signal exploitation also includes tactics that influence availability and client behavior because forcing reconnections or steering users toward a stronger signal can be the difference between “we observed it” and “we captured what we need.”

Disruption and forced behavior

Disruption is the blunt instrument of wireless exploitation: if clients can be forced to disconnect or lose stability, they will often reconnect automatically and generate the authentication artifacts attackers want. In lawful, authorized testing, the focus is usually not “jamming the spectrum” but creating controlled conditions that reveal weaknesses such as whether clients roam to a stronger AP without user awareness, whether a

network's transition mode enables downgrade opportunities, or whether a client leaks information during probe behavior. The offensive takeaway is that availability pressure creates predictability, and predictability creates capture opportunities.

Tactics as reconnaissance at scale

Wardriving (and its variations like war walking) is reconnaissance with a spatial dimension. Instead of asking "what networks exist," you ask "where are they, how are they configured, and what does the environment look like from the outside?" For offensive work, that matters because an organization's wireless footprint often extends beyond its physical perimeter: signal bleed into parking lots, adjacent buildings, or shared spaces can make the air a public attack surface. Logging SSIDs, channels, encryption types, and signal strength across an area helps you identify weak networks, unmanaged access points, and opportunities for stealthy positioning during later phases.

Using discovery platforms and analyzers effectively

Tools that aggregate and visualize wireless data turn raw observations into targeting intelligence. A large-scale logging platform can help you predict density and identify repeated SSIDs across multiple locations. A local analyzer that shows channel congestion, security modes, and real-time signal behavior helps you decide where to capture traffic and which networks are most likely to yield exploitable conditions. The offensive goal is not just to collect "Wi-Fi names," but to build a map of trust boundaries: where users connect, which networks are open or legacy, and what patterns can be exploited to steer a user or capture a handshake.

Exam Focus

- Wireless signal exploitation combines reconnaissance and behavioral influence: mapping networks by location, signal strength, and configuration, then creating conditions that produce useful artifacts like reconnection events.
- Wardriving-style reconnaissance helps identify the organization's wireless footprint beyond its perimeter, including signal bleed and unmanaged access points.
- Discovery and analysis tools are used to translate SSIDs, channels, encryption modes, and signal behavior into target selection and positioning for later attacks.

Aircrack-ng (OBJ 4.7)

Aircrack-ng is one of the most recognized wireless auditing suites because it supports an entire offensive workflow: observe, capture, force reconnection, and attempt key recovery. For WPA/WPA2-PSK networks, the critical concept is that cracking usually happens offline—meaning the attacker first captures authentication material and then

tests candidate passwords locally without touching the network again. That separation is what makes the workflow stealthier and more repeatable during an assessment.

From passive capture to actionable authentication material

The practical barrier to WPA/WPA2 cracking is not “math,” it’s acquisition: you need the four-way handshake (or equivalent authentication exchange) that proves a client knows the network secret. With a wireless adapter in monitor mode, you can observe the target environment, identify the access point, and record frames associated with client authentication. When a handshake is captured correctly, it becomes a portable artifact, which is something you can validate later, store as evidence, and test against curated wordlists without generating additional wireless noise.

Deauthentication as a control lever

Deauthentication attacks exploit the reality that clients reconnect automatically when they believe they were legitimately disconnected. By spoofing management frames, an attacker can force a client off the AP, which commonly triggers a reconnection attempt and produces the handshake the attacker wants to capture. In offensive work, deauthentication is less about denial of service and more about timing, meaning it is used to create a predictable authentication event when natural reconnects are too slow. This also becomes a pivot point into broader attacks, because once clients are unstable, they may roam to stronger signals, which is exactly what an Evil Twin setup tries to exploit.

Offline cracking and what “success” really means

Once the handshake is captured, offline cracking becomes a password quality test. A strong PSK with high entropy and length makes dictionary attacks impractical; a human-chosen PSK makes them realistic. The offensive value is twofold: if the key is recovered, you gain legitimate network access; if it isn’t, you still produced evidence about password strength and can pivot toward other wireless vectors such as WPS exposure, rogue access points, captive portal harvesting, or client-side weaknesses. In many assessments, the real outcome is not merely recovering a key but demonstrating which control failed first, weak PSK selection, shared password distribution, or lack of monitoring for forced reconnect behavior.

Exam Focus

- Aircrack-ng supports a workflow of monitor mode capture → handshake acquisition → offline password guessing against WPA/WPA2-PSK.
- Deauthentication is commonly used to force a reconnect event so the tester can capture the handshake needed for offline cracking.

- Successful cracking depends heavily on PSK quality; when cracking fails, testers pivot to other wireless vectors such as WPS weaknesses or rogue access point techniques.

Captive Portal Attacks (OBJ 4.7)

Captive portals exist because public and guest Wi-Fi needs a “gate” before full internet access is granted, but that gate is also a trust anchor: users expect it, interact with it, and often submit sensitive data into it. From an offensive perspective, a captive portal is valuable because it moves the attack from cryptography to human behavior. Instead of cracking encryption, the attacker convinces the user to hand over credentials or uses flaws in the portal itself to gain a deeper foothold.

Why captive portals are persuasive by default

In many environments, users have been trained by experience to accept captive portals as normal in environments such as hotels, airports, conference venues, and cafés. That expectation lowers suspicion, especially when the portal is branded with familiar logos and language. Technically, the portal intercepts outbound traffic until a condition is met, but psychologically it feels like a routine “step” before browsing. For an attacker, that makes captive portals a high-conversion moment: users are already primed to click, accept, and type.

Captive portal exploitation through rogue infrastructure

The most common offensive path is to combine captive portals with a rogue access point. When the attacker controls the access point, they control the portal flow. By mimicking the SSID and presenting a portal that matches the legitimate experience, the attacker can harvest whatever the portal requests, such as email addresses, passwords, voucher codes, or other identifiers then optionally forward the victim to real internet access to avoid suspicion. The key offensive lesson is that the portal is not the vulnerability by itself; the vulnerability is the user’s inability to authenticate the network and the portal as legitimate.

When the portal software becomes the vulnerability

Even when the captive portal is legitimate, it is still a web application that can be poorly built. If the portal mishandles input, sessions, or authentication, it can be exploited like any other web system. That matters in offensive testing because a portal often sits in a privileged position: it sees first-touch client traffic, sometimes integrates with payment systems, identity providers, or internal network controls, and may have administrative interfaces exposed within the same environment. A vulnerability here can become more than credential theft; it can become control of the access workflow or even a bridge into adjacent infrastructure.

Exam Focus

- Captive portals are effective social engineering surfaces because users expect them on guest/public Wi-Fi and often submit information without verifying legitimacy.
- Attackers commonly pair captive portals with rogue access points (Evil Twin-style) to harvest credentials while maintaining the appearance of normal internet access.
- Captive portal implementations can also contain classic web vulnerabilities (e.g., input validation, session handling) that may allow bypass or escalation beyond credential theft.

Evil Twin (OBJ 4.7)

An Evil Twin attack turns wireless trust against the victim by impersonating a legitimate access point. Instead of defeating encryption directly, the attacker defeats the user's decision-making: if the SSID looks familiar and the signal is strong, many clients connect automatically or with minimal scrutiny. Once the victim is on the attacker's network, the attacker becomes an on-path adversary with control over resolution, routing, and (when possible) content delivery.

Impersonation at the network layer

An Evil Twin is a rogue access point that copies the SSID and often mimics the security posture of the legitimate network. The success condition is simple: the victim associates to the attacker's AP instead of the real one. That can happen because the rogue signal is stronger, because clients prefer previously seen networks, or because the attacker creates instability that encourages roaming. The offensive value is immediate: the attacker is now the gateway to the internet and can observe metadata, capture credentials through portal flows, and attempt to downgrade or intercept traffic where protections are weak.

Why framework-driven setups scale the attack

Wireless attack frameworks streamline Evil Twin deployments by bundling SSID cloning, captive portals, DNS manipulation, traffic capture, and optional traffic tampering into one workflow. This matters during engagements because the attacker's goal is not just to stand up a fake network, but to manage victims: present a believable portal, redirect key domains, and keep the user experience stable enough that the victim remains connected. In other words, a good Evil Twin is not loud; it is comfortable and users stay on it because it 'works.'

What interception really enables after association

Once a victim connects, the attacker can perform a spectrum of on-path actions. Unencrypted traffic can be observed directly. Domain name resolution can be manipulated to steer users to attacker-controlled destinations. Portal and credential

prompts can be inserted at high-trust moments. Even when traffic is encrypted, the attacker can still harvest value through phishing, session abuse, and selective manipulation especially if users ignore certificate warnings or if misconfigurations allow protocol downgrades. For offensive testing, the Evil Twin demonstrates that wireless compromise can be achieved by exploiting trust and convenience rather than breaking cryptography.

Exam Focus

- An Evil Twin attack impersonates a legitimate Wi-Fi network by cloning the SSID and attracting client associations through stronger signal or client roaming behavior.
- Frameworks used for Evil Twin attacks often combine rogue AP creation with captive portals, DNS manipulation, and traffic capture to manage victims at scale.
- Once victims associate to the rogue AP, the attacker can intercept and manipulate traffic as an on-path adversary, with credential harvesting often being the highest-impact outcome.

Kismet (OBJ 4.7)

Kismet is a reconnaissance force multiplier because it is designed to listen quietly. In wireless offensive work, passive collection is valuable: it reduces detection risk while still producing actionable intelligence about access points, clients, encryption modes, channels, and even non-Wi-Fi radios. Kismet helps you build the picture of the RF environment so you can choose the right next move, whether that is handshake capture, rogue access point deployment, or simply reporting unmanaged wireless exposure.

Passive detection as stealth reconnaissance

Unlike active scanners that transmit probes, Kismet primarily operates by capturing what the environment is already broadcasting. That means you can map SSIDs, channels, signal strength, encryption types, and client associations without prompting a response from infrastructure. For offensive testing, this reduces noise and makes your data more representative of real conditions. It also helps you identify high-value situations, such as access points with many connected clients, legacy encryption, or unusual channel behavior that suggests misconfiguration or interference.

Hidden networks and the failure of obscurity

Disabling SSID broadcast is often treated as a security feature, but it is closer to a cosmetic choice. Even when an SSID is not advertised in beacons, it appears during client activity when devices probe for and associate to the network. Because Kismet watches management traffic, it can reveal 'hidden' SSIDs without forcing deauthentication or interacting with clients. Offensively, this matters because it exposes

internal networks that defenders believed were invisible and helps you identify whether hidden networks are used for sensitive operations or simply legacy configurations.

Turning captures into next-phase artifacts

Kismet's output is not just a dashboard; it is evidence and material for deeper analysis. Captures can be inspected for protocol behavior, exported for later review, and used to guide more targeted attacks with other tools. Additionally, Kismet's ability to detect multiple radio technologies expands your scope: a wireless assessment may include Bluetooth or IoT-related signals that represent unmanaged attack surfaces. In an engagement, this multi-protocol visibility often reveals that 'wireless' risk is broader than the organization expects.

Exam Focus

- Kismet is primarily a passive wireless detector/sniffer used for stealth reconnaissance to identify SSIDs, channels, encryption types, and client associations.
- Hidden SSIDs can still be discovered because the SSID appears in management traffic during client probing and association; obscurity does not prevent detection.
- Captured data can be exported for deeper analysis and can guide later phases of a wireless attack chain, including targeting high-activity or weakly secured networks.

Wi-Fi Protocol Fuzzing (OBJ 4.7)

Wi-Fi protocol fuzzing is about breaking assumptions. Wireless devices expect frames to be well-formed, fields to be sane, and values to be within normal ranges. Fuzzing intentionally violates those assumptions to see whether firmware and drivers validate input safely. From an offensive perspective, the value is discovering fragile implementations: crashes, lockups, resets, parsing errors, or more severe failures that could hint at memory corruption and deeper exploitability.

Frame types and why they matter to fuzzing

IEEE 802.11 traffic is built from management frames, control frames, and data frames. Management frames define relationships like beacons, probes, authentication, association, and deauthentication. Control frames coordinate the radio conversation. Data frames carry the payload. Fuzzing can target any of these, but management frames are particularly attractive because they influence connection state and are processed by many devices even when not fully associated. When a device handles a malformed management frame incorrectly, it may expose stability weaknesses that are reachable without credentials.

How malformed frames expose weak validation

Fuzzing works by mutating fields a device should reject: invalid reason codes, unexpected element lengths, inconsistent sequence behavior, malformed checks, or unusual combinations of flags. A robust implementation discards bad frames quietly. A fragile one leaks clues like log errors, interface resets, connection instability, or full driver crashes. In offensive testing, those reactions are the signal: they tell you where the parsing boundary is weak and where deeper research might find a vulnerability worth reporting or demonstrating.

Operationalizing fuzzing in authorized testing

Practical fuzzing requires an environment that supports raw frame injection and capture so you can reproduce results and measure impact. The tester crafts frames, transmits them in controlled volumes, and monitors the target's behavior and logs for anomalies. The goal is not random disruption; it is systematic exploration by varying one field at a time, isolating conditions that trigger failure, and capturing enough evidence to explain the risk. Because malformed frame testing can destabilize networks and devices, it must be scoped carefully and performed only with explicit authorization and safety boundaries that protect operations.

Exam Focus

- Wi-Fi protocol fuzzing sends malformed or unexpected 802.11 frames to test whether wireless firmware/drivers validate input safely.
- Management frames are common fuzzing targets because they affect connection state and may be processed even without full association.
- Successful fuzzing results are observable device failures or anomalies (crashes, resets, parsing errors) that indicate fragile implementations and potential vulnerability research paths.

Section 17. Network Attacks

13 Lessons | Exam Objectives Covered: 4.2| Does not include demonstrations

Network Attacks (OBJ 4.2)

Network attacks encompass a broad range of techniques designed to exploit weaknesses within a network's infrastructure. These attacks may involve gaining unauthorized access, intercepting sensitive data, or disrupting communications.

Scope of Network Attacks

Network attacks operate across multiple layers of the network stack, ranging from physical-layer disruption to application-layer exploitation. Understanding this diversity is essential because attackers can adapt their approaches depending on the target environment, weaknesses, and defenses in place.

Attack Categories Covered in This Section

This section introduces stress testing, segmentation bypassing, MAC spoofing, NAC bypass, session-based attacks, service exploitation, packet crafting, Netcat usage, default credential exploitation, poisoning attacks, ARP attacks, and Metasploit fundamentals. Each topic builds toward understanding how attackers compromise network integrity.

Exam Focus

- Network attacks target weaknesses at multiple OSI layers.
- Attack techniques range from load-based DoS to exploitation frameworks.

Stress Testing (OBJ 4.2)

Stress testing evaluates how systems behave under extreme load conditions. Unlike fuzzing, which focuses on malformed inputs, stress testing overwhelms system resources through high volume or intensity. Offensive testers use stress testing to identify breaking points that could lead to denial-of-service conditions.

Understanding Stress Testing

Stress testing identifies the point at which a system stops functioning correctly due to resource exhaustion. This can reveal instability, slowdowns, or crashes. Attackers leverage these weaknesses to disrupt availability.

Architectural Constraints and Scaling

Stress testing exposes whether systems rely on vertical scaling, where resources are increased on a single server, or horizontal scaling, where load is distributed across multiple servers. Organizations often discover their true limits only through aggressive load testing.

Tools and Load Generation Techniques

Stress testing can be performed with custom scripts, open-source tools like Grinder, or SaaS platforms like LoadView. More aggressive methods include packet storms and legacy protocol abuse, such as CHARGEN reflection attacks.

Exam Focus

- Stress testing reveals system breaking points under extreme load.
- Vertical vs. horizontal scaling determines resilience.
- Packet storms and CHARGEN abuse can simulate DoS conditions.

Bypassing Segmentation (OBJ 4.2)

Segmentation bypassing involves circumventing network isolation controls. Attackers exploit misconfigurations in firewalls, routing, VLAN design, or host architecture to move laterally between restricted segments.

Segmentation Bypass Techniques

Network segmentation is intended to reduce lateral movement. However, weak firewall rules or misconfigured routing paths can allow attackers to access systems assumed to be protected, demonstrating breakdowns in isolation strategy.

VLAN Hopping Attacks

VLAN hopping targets Layer 2 segmentation. Techniques include switch spoofing, where attackers impersonate trunk ports, and double tagging, where packets are crafted with multiple VLAN tags to reach unauthorized VLANs.

Multihomed Host Risks

Multihomed hosts connect multiple network segments through multiple interfaces. If compromised, they can become pivot points that bridge isolated networks, nullifying segmentation controls and expanding attacker reach.

Exam Focus

- Segmentation bypass enables lateral movement across isolated zones.
- VLAN hopping exploits Layer 2 switch misconfigurations.

- Multihomed hosts increase attack surface by bridging networks.

MAC Spoofing (OBJ 4.2)

This lesson explores MAC Spoofing from an offensive penetration testing perspective, focusing on how attackers exploit weaknesses in network identity, access control, and session integrity.

Spoofing and MAC Address Identity

Spoofing is a category of network attacks in which an attacker falsifies identity information to appear as another trusted system. At Layer 2, this identity is represented through the Media Access Control (MAC) address, which is a unique 48-bit identifier assigned to a network interface card. Switches rely on MAC addresses to forward Ethernet frames correctly by storing mappings inside their CAM tables. Because the network assumes MAC identity is trustworthy, attackers can exploit this reliance as an entry point into restricted environments.

Bypassing Layer 2 Access Controls

MAC spoofing allows an attacker to overwrite their device's MAC address so it matches an authorized system. Many organizations still rely on MAC filtering or basic Network Access Control mechanisms that permit or deny devices based on hardware identity. Since operating systems allow MAC overrides in software, attackers can impersonate trusted endpoints, bypass admission controls, and blend into legitimate traffic patterns. This demonstrates why MAC filtering alone is not sufficient for modern network security.

Tools for MAC Spoofing in Practice

Attackers commonly use utilities such as `ifconfig` on macOS or `MACchanger` on Linux to assign either a specific cloned MAC address or a randomized one. For example, `ifconfig` can temporarily force an interface to adopt a new address until reboot, while `MACchanger` provides quick rotation options for evasion. These tools make Layer 2 impersonation extremely accessible, enabling penetration testers to validate whether an organization's access controls are vulnerable to spoofing.

Exam Focus

- Understand the offensive purpose and risks of MAC spoofing.
- Know the tools and protocols commonly associated with the attack.
- Be able to explain how penetration testers exploit these weaknesses in real engagements.

NAC Bypass (OBJ 4.2)

This lesson explores NAC Bypass from an offensive penetration testing perspective, focusing on how attackers exploit weaknesses in network identity, access control, and session integrity.

How Network Access Control Works

Network Access Control (NAC) is designed to evaluate devices before granting access to protected networks. NAC solutions inspect posture and health, including antivirus status, patch levels, configuration integrity, and presence of required security agents. Devices that pass compliance checks are admitted to the production network, while non-compliant systems are quarantined into remediation VLANs. NAC serves as a first defensive gate, making it a key offensive target.

Agent Models and Enforcement Types

NAC implementations may rely on persistent agents installed permanently on corporate endpoints, non-persistent agents deployed temporarily through captive portals, or agentless solutions that inspect endpoints remotely. Each model introduces different strengths and weaknesses. Persistent agents provide deeper enforcement, while agentless approaches are common in BYOD environments but offer attackers more opportunity for impersonation or evasion.

Offensive Techniques for NAC Bypass

Attackers typically bypass NAC either by compromising an already authorized host or by impersonating a device type that NAC automatically trusts. Many NAC appliances maintain allow lists for printers, VoIP phones, and conferencing devices that cannot run compliance agents. By spoofing MAC identity to match these trusted systems, attackers can slip past enforcement. Once admitted, VLAN hopping or lateral movement techniques may be required to escape restricted segments.

Exam Focus

- Understand the offensive purpose and risks of NAC bypass.
- Know the tools and protocols commonly associated with the attack.
- Be able to explain how penetration testers exploit these weaknesses in real engagements.

Session-Based Attacks (OBJ 4.2)

This lesson explores Session-Based Attacks from an offensive penetration testing perspective, focusing on how attackers exploit weaknesses in network identity, access control, and session integrity.

On-Path Interception Attacks

An on-path attack occurs when an attacker positions themselves between two communicating hosts so that traffic flows through their device. Techniques such as ARP poisoning, DNS poisoning, rogue access points, or malicious switching infrastructure allow attackers to intercept authentication sessions. Once in place, attackers can monitor or manipulate sensitive traffic invisibly.

Replay and Relay Exploitation

With an interception position established, attackers can execute replay attacks by capturing legitimate authentication material and retransmitting it to gain access. Relay attacks are even more powerful, turning the attacker into a live proxy that forwards and modifies traffic between client and server. These attacks enable credential theft, session hijacking, and command manipulation without breaking encryption directly.

SSL Stripping and Downgrade Attacks

Encryption limits the effectiveness of interception, so attackers often attempt SSL stripping or downgrade attacks. SSL stripping forces victims to communicate over plaintext HTTP instead of HTTPS, removing TLS protection entirely. Downgrade attacks interfere with protocol negotiation, forcing weaker legacy encryption such as SSL 2.0 or TLS 1.0. These methods weaken secure communication channels and restore attacker visibility.

Exam Focus

- Recognize interception-based attacks in scenarios, including ARP poisoning, DNS poisoning, replay, relay, SSL stripping, and downgrade attempts, and determine what condition makes them possible (e.g., lack of TLS enforcement, weak protocol negotiation, disabled SMB signing).
- Be prepared to identify the attacker's position and goal in a session attack, such as on-path placement for credential capture, session hijacking, or live relay without cracking encryption.

Service Exploitation (OBJ 4.2)

This lesson introduces key offensive concepts relevant to Service Exploitation, explaining how penetration testers use these techniques to exploit weaknesses and expand access.

Misconfigured Services

Misconfigured services are one of the fastest offensive entry points during a penetration test. When services are deployed with default settings, overly permissive access controls, exposed directories, or unnecessary privileges, they can leak sensitive data or allow unauthorized access without complex exploitation. Attackers often search for directory listings, forgotten backup files, and verbose error logs because these misconfigurations provide immediate intelligence that supports deeper compromise.

Share Enumeration and Exploitation

Share enumeration focuses on identifying accessible shared resources through protocols such as SMB. Penetration testers use tools like smbclient to list shares, validate authentication requirements, and determine whether sensitive documents or writable shares exist. Weakly secured shares can expose credentials, business data, or even provide a location to upload payloads for execution.

Certificate Services Attacks

Certificate services attacks target weaknesses in PKI trust infrastructures. If certificate authorities rely on outdated cryptographic algorithms or misconfigured issuance policies, attackers may forge certificates, impersonate trusted services, or intercept encrypted communications. Weak hashing algorithms such as MD5 create opportunities for collision attacks, undermining the integrity of digital trust.

Exam Focus

- Expect scenario questions that require identifying exploitable service weaknesses, such as default credentials, exposed shares, excessive permissions, or insecure certificate authority configurations.
- Be able to determine how a misconfigured service enables privilege escalation or lateral movement, especially through SMB share abuse, writable directories, or weak PKI issuance policies.

Packet Crafting (OBJ 4.2)

This lesson introduces key offensive concepts relevant to Packet Crafting, explaining how penetration testers use these techniques to exploit weaknesses and expand access.

Packet Crafting Fundamentals

Packet crafting is the manual creation and manipulation of network packets to test how devices, firewalls, and protocols respond to unexpected traffic. By controlling header fields and payload structure, attackers can probe weaknesses, bypass filtering rules, or simulate malicious behavior with precision.

Relay Attacks with Impacket

Impacket provides offensive protocol libraries that enable relay attacks against SMB and LDAP services. Instead of cracking passwords, attackers intercept NTLM authentication attempts and relay them to another host, gaining the same access level as the victim user. These attacks remain highly effective when SMB signing is disabled or when NTLM authentication is still widely trusted.

Custom Packets with Scapy

Scapy offers low-level control over packet construction across IP, TCP, UDP, and application layers. Penetration testers can craft SYN packets, malformed payloads, or protocol-specific probes to analyze responses, test IDS detection, or build proof-of-concept exploits. Its flexibility makes it a powerful tool for advanced network testing.

Exam Focus

- Recognize when packet manipulation is being used to bypass controls or test protocol behavior, such as crafting custom TCP flags, malformed payloads, or spoofed source addresses.
- Be prepared to identify scenarios where packet-level manipulation enables relay attacks, firewall evasion, or IDS testing, rather than traditional exploitation of an application vulnerability.

Netcat (OBJ 4.2)

This lesson introduces key offensive concepts relevant to Netcat, explaining how penetration testers use these techniques to exploit weaknesses and expand access.

Remote Shell Access

Penetration testers often aim to gain interactive shell access to a target system. A remote shell provides command execution capability as if the attacker were locally present. Tools like Netcat enable this through raw TCP/UDP socket communication.

Bind vs Reverse Shells

Bind shells require the victim to open a listening port, allowing the attacker to connect inward. Reverse shells invert the direction by having the victim initiate the outbound connection, making them more effective in environments where inbound traffic is blocked by firewalls or NAT.

Netcat for Shells and Exfiltration

Netcat is known as the 'Swiss army knife' of networking because it can establish shells, transfer files, and exfiltrate data through commonly allowed ports. By redirecting input/output streams, attackers can move sensitive files out of restricted networks without relying on traditional file transfer services.

Exam Focus

- Recognize when Netcat is being used to establish bind shells, reverse shells, or simple listeners, and determine why a reverse shell is preferred in environments protected by NAT or inbound firewall rules.
- Be prepared to identify Netcat usage in scenarios involving shell access, port listening, banner grabbing, or basic data exfiltration, especially when raw TCP/UDP socket communication is sufficient without a full exploitation framework.

Default Network Credentials (OBJ 4.3)

This lesson examines how attackers exploit vendor default credentials to gain rapid initial access during internal and external penetration tests.

Why Default Credentials Matter

Default credentials are preconfigured usernames and passwords assigned to devices and software at shipment. Attackers rely on them because they are publicly documented and often left unchanged.

Exploiting Defaults During Engagements

Penetration testers identify exposed services with tools like Nmap and attempt known vendor credential pairs. Successful logins can lead directly to administrative access.

Lateral Movement and Escalation Impact

Default credentials often serve as the starting point for deeper compromise. Once one device is accessed, attackers can pivot, harvest configs, and escalate privileges across the environment.

Exam Focus

- Recognize default credential abuse as an initial access technique, especially against routers, IoT devices, printers, and admin portals that were never hardened after deployment.
- Be prepared for scenarios where a simple vendor default login enables privilege escalation or lateral movement, and know that this is a misconfiguration issue rather than an exploit requiring payload delivery.

LLMNR/NBT-NS Poisoning (OBJ 4.3)

This lesson explains how attackers abuse legacy name resolution protocols to capture authentication material within internal networks.

LLMNR as a DNS Fallback Weakness

LLMNR broadcasts hostname queries on local networks when DNS fails. Because responses are unauthenticated, attackers can spoof answers.

NBNS and Legacy Name Resolution Abuse

NBNS provides another fallback mechanism for short hostnames. Like LLMNR, it can be poisoned to redirect victims.

Responder Poisoning in Practice

Responder listens for LLMNR/NBNS requests and replies maliciously, capturing NTLM hashes or redirecting traffic to attacker-controlled systems.

Exam Focus

- Know the conditions that make LLMNR/NBT-NS attacks possible, such as broadcast-based fallback resolution, lack of authentication, and Windows environments where these legacy protocols remain enabled.

ARP Poisoning (OBJ 4.3)

This lesson covers how attackers manipulate ARP trust relationships to position themselves as an on-path adversary inside a local network.

ARP Mapping and Trust Issues

ARP resolves IP addresses to MAC addresses on local networks. Since ARP has no authentication, attackers can manipulate ARP caches.

ARP Spoofing to Redirect Traffic

Attackers forge ARP replies so victims associate the attacker's MAC with a legitimate IP, causing traffic interception.

Poisoning Tools and On-Path Positioning

Tools like arpspoof and Metasploit modules automate poisoning, enabling interception, manipulation, and credential harvesting.

Exam Focus

- Be prepared to identify ARP poisoning outcomes in questions, such as session hijacking, credential sniffing, or traffic redirection within the same LAN or VLAN segment.

Intro to Metasploit (OBJ 4.3)

This lesson introduces the Metasploit Framework as a modular platform for exploitation, payload delivery, and post-exploitation control.

Metasploit Framework Overview

Metasploit provides modular exploits, payloads, and auxiliary tools for attacking vulnerable systems and conducting post-exploitation.

Payload Delivery with msfvenom

Msfvenom generates custom payloads such as reverse shells or Meterpreter sessions, which can be delivered via phishing or exploitation.

Offensive Workflow and Exam Relevance

Understanding module selection, payload configuration, and post-exploitation actions is critical for penetration testing scenarios.

Exam Focus

- Expect exam scenarios where you must choose the correct Metasploit module type, such as exploit vs auxiliary vs post-exploitation, based on the engagement goal (scanning, gaining access, maintaining control).

- Know how payload selection and configuration matters, including when msfvenom-generated reverse shells or Meterpreter sessions are appropriate and how misconfigured options can cause exploitation failure.

Section 18. Authentication Attacks

9 Lessons | Exam Objectives Covered: 4.2, 4.3, 4.5 | Does not include demonstrations

Authentication Attacks (OBJ 4.2, 4.3, 4.5)

Authentication attacks target identity verification mechanisms to gain unauthorized access by exploiting weaknesses in password management, authentication protocols, and identity processes.

Key Authentication Attack Types

This section covers password attacks, credential attacks, credential passing methods, and modern SSO abuses including SAML and OpenID Connect weaknesses.

Collaboration With Tools

Industry-standard tools such as Hydra, John the Ripper, Hashcat, CrackMapExec, and Mimikatz are central to executing authentication attack scenarios.

Exam Focus

- Understand that hashes, tickets, and tokens can be abused without cracking plaintext passwords.
- Expect authentication attacks to appear frequently in scenario-based exam questions.

Types of Password Attacks (OBJ 4.3)

Brute Force Attacks

Brute-force attacks attempt every possible password combination until success using tools like Hydra or John the Ripper, but are slowed by complexity and lockout controls.

Dictionary and Hybrid Attacks

Dictionary attacks use wordlists such as rockyou.txt, while hybrid attacks apply rule-based variations like appending digits or symbols.

Rainbow Table Attacks

Rainbow tables rely on precomputed hash mappings and are fast when hashes are unsalted, but ineffective against salted password storage.

Exam Focus

- Recognize when brute force, dictionary, or rainbow tables are most effective.
- Understand rainbow tables fail when hashes are salted.
- Know Hydra and John the Ripper are common password-cracking tools.

Credential Attacks (OBJ 4.3)

Password Spraying

Password spraying tests one or a few common passwords across many accounts to avoid lockout policies, often automated with CrackMapExec.

Password Masking

Masking attacks generate structured guesses based on password complexity rules, making Hashcat highly effective against predictable formats.

Credential Stuffing and MFA Fatigue

Credential stuffing uses breached username-password pairs with tools like Hydra, while MFA fatigue overwhelms users with push prompts until one is approved.

Exam Focus

- Know spraying targets many accounts with minimal attempts per user.
- Understand mask attacks exploit predictable complexity requirements.
- MFA fatigue is a user-targeted bypass rather than a password crack.

Credential Passing Attacks (OBJ 4.3)

Pass-the-Hash (PtH)

PtH attacks reuse NTLM hashes extracted from LSASS memory, allowing authentication without knowing the plaintext password.

Pass-the-Ticket (PtT)

PtT attacks replay stolen Kerberos tickets such as TGTs or TGS tickets to authenticate within Active Directory environments.

Pass-the-Token

Pass-the-Token attacks reuse stolen JWTs or session tokens in web and cloud services, bypassing password prompts entirely.

Exam Focus

- Credential passing relies on artifacts, not password guessing.
- PtH is tied to NTLM in Windows environments.
- PtT and Pass-the-Token are common in AD and cloud-integrated applications.

Directory Service Attacks (OBJ 4.2, 4.3)

Kerberos-Based Attacks

Kerberoasting cracks service account TGS tickets offline, while Silver and Golden Tickets forge authentication tickets for service or domain-wide compromise.

LDAP Injection

LDAP injection manipulates directory queries through unsanitized input, altering filters to return unauthorized user or group objects.

CrackMapExec Automation

CME automates credential testing, enumeration, and remote execution across many Active Directory hosts, enabling scalable post-exploitation.

Exam Focus

- Golden Tickets require krbtgt hash compromise and persist until rotated twice.
- LDAP injection is conceptually similar to SQL injection but targets directory filters.
- CME is a key automation tool for AD-focused engagements.

CrackMapExec (CME) (OBJ 4.3)

CME Core Purpose

CrackMapExec is a post-exploitation framework that automates authentication validation and host interaction across SMB, WinRM, and LDAP.

Post-Exploitation Actions

CME supports command execution, SAM dumping, LSASS extraction, user enumeration, session discovery, and Kerberos authentication workflows.

Lateral Movement Efficiency

By testing where credentials or hashes work across subnets, CME quickly reveals pivot points and privilege escalation paths.

Exam Focus

- Know CME as the “Swiss Army knife” of Windows domain post-exploitation.
- Understand CME integrates directly with Pass-the-Hash and Pass-the-Ticket workflows.
- Expect CME-related scenarios in authentication attack questions.

SAML Attacks (OBJ 4.3)

SAML Workflow

SAML is an XML-based SSO framework where an IdP issues signed assertions that an SP validates to grant access across multiple applications.

Token Manipulation and Replay

Manipulation alters assertion attributes when signature validation is weak, while replay reuses captured assertions when expiration is not enforced.

SAML Injection

Injection occurs when unsanitized input is used to build SAML XML messages, allowing attackers to alter authentication logic.

Exam Focus

- Signature validation is the primary trust mechanism in SAML.
- Replay attacks succeed when assertions remain valid and reusable.
- Injection exploits insecure XML construction in SSO workflows.

OpenID Connect (OIDC) Attacks (OBJ 4.3)

OIDC Token Workflow

OIDC extends OAuth 2.0 with authentication using JWT-based ID tokens issued by an OP and validated by a relying party.

ID Token Manipulation and Replay

Manipulation changes claims if signature validation is weak, while replay reuses captured bearer tokens when session binding is missing.

Injection, WebFinger, and SSRF

Injection exploits insecure token construction, while WebFinger enumeration and SSRF via dynamic client registration expose additional attack surfaces.

Exam Focus

- OIDC attacks often focus on JWT signature and claim validation failures.
- Replay is effective when token expiration and reuse checks are weak.
- WebFinger and dynamic registration can introduce recon and SSRF vectors.

Hash Attacks (OBJ 4.5)

Baseline Hash Weaknesses

Fast hashes like MD5 or SHA-1 are vulnerable to offline cracking once obtained through database compromise or memory extraction.

Salting and Key Stretching

Salts break rainbow tables, while stretching algorithms like bcrypt or Argon2 slow brute-force attempts but weak configurations remain crackable.

Collision and Birthday Attacks

Collision attacks generate different inputs producing the same hash, and birthday attacks show collisions occur sooner than expected, undermining integrity checks.

Exam Focus

- Know why fast hashes are trivial to crack with GPUs.
- Understand salting prevents rainbow table reuse but does not fix weak passwords.
- Collision resistance is critical when hashes are used for integrity validation.

Section 19. Host Attacks

11 Lessons | Exam Objectives Covered: 4.4 | Does not include demonstrations

Host Attacks (OBJ 4.4)

Host attacks exploit weaknesses inside individual systems to gain unauthorized access, escalate privileges, or execute malicious actions by abusing misconfigurations, unpatched vulnerabilities, and insecure local controls.

Post-exploitation progression

Understanding host-level abuse is essential because it enables attackers to pivot, maintain persistence, and expand control after establishing an initial foothold.

Key host-based themes

This section includes privilege escalation, credential harvesting, endpoint misconfigurations, unquoted service paths, disabling defenses, payload obfuscation, restricted environment escapes, log tampering, and living off the land techniques.

Exam Focus

- Host attacks exploit weaknesses inside a compromised host to escalate privileges and expand access.

Privilege Escalation (OBJ 4.4)

Privilege escalation occurs when a tester gains higher-level permissions by exploiting trust relationships, weaknesses, and misconfigurations in operating systems and applications.

Vertical vs horizontal escalation

Vertical escalation moves from a standard user to administrator or SYSTEM, while horizontal escalation uses one user's access to reach another user's resources at the same privilege level.

Windows escalation tooling

Mimikatz extracts plaintext passwords, hashes, and Kerberos tickets from LSASS memory, while Seatbelt and PowerShell ISE automate enumeration of insecure services, registry permissions, and scheduled tasks.

Exam Focus

- Vertical escalation raises privilege levels, horizontal escalation abuses peer-level access.

- Mimikatz and Seatbelt are key Windows post-exploitation tools for escalation workflows.

Credential Harvesting (OBJ 4.4)

Credential harvesting collects usernames and passwords through external interaction such as phishing or deceptive login portals, while credential dumping extracts stored authentication material after compromise.

Post-exploitation credential extraction

Credential dumping targets plaintext passwords, NTLM hashes, Kerberos tickets, and PINs stored in LSASS memory, enabling lateral movement and escalation.

Key tools: Mimikatz and Rubius

Mimikatz supports pass-the-hash through SAM extraction, while Rubius manipulates Kerberos tickets through renewal, injection, and pass-the-ticket attacks.

Exam Focus

- Harvesting supports initial access, dumping expands access after compromise.
- Expect exam scenarios involving Mimikatz hashes and Kerberos ticket abuse with Rubius.

Misconfigured Endpoints (OBJ 4.4)

Misconfigured endpoints contain insecure or overly permissive settings such as weak authentication, open administrative shares, or unnecessary elevated services.

PowerShell privilege escalation paths

Unrestricted PowerShell policies and weak directory permissions allow attackers to enumerate services and execute malicious scripts through SYSTEM-level scheduled tasks or service loaders.

PsExec lateral movement

PsExec enables authenticated remote command execution, allowing attackers to pivot across hosts when credentials and remote execution settings are improperly secured.

Exam Focus

- Misconfigurations are common escalation and pivot points in Windows networks.
- PowerShell and PsExec frequently appear in endpoint exploitation scenarios.

Unquoted Service Paths (OBJ 4.4)

Unquoted service paths occur when Windows service executable paths contain spaces but lack quotation marks, causing Windows to interpret and execute path segments incorrectly.

PowerShell detection workflow

WMI queries can enumerate auto-start services whose PathName fields contain spaces without quotes, identifying privilege escalation opportunities.

PsExec exploitation

Attackers can place a malicious Program.exe earlier in the interpreted path so the service executes attacker code with elevated privileges when it starts.

Exam Focus

- Unquoted paths are a reliable Windows privilege escalation vector.
- Know detection via PowerShell WMI and exploitation via malicious executable placement.

Disabling Security Software (OBJ 4.4)

Attackers may attempt to disable Defender services with commands like Stop-Service WinDefend, but tamper protection often blocks direct service modification.

Registry-based bypass techniques

Registry policy keys can sometimes weaken protections even when service-level tampering is denied, such as disabling real-time monitoring through HKLM policy edits.

Remote disabling with PsExec

PsExec enables remote execution of Defender-disabling scripts, firewall shutdown commands, or UAC registry modifications across multiple hosts.

Exam Focus

- Modern Windows blocks obvious Defender disable attempts, but registry tampering may still work.
- PsExec extends security-disabling actions to remote systems post-compromise.

Payload Obfuscation (OBJ 4.4)

Payload obfuscation disguises malicious code through encoding, encryption, compression, or code manipulation to evade signature-based detection.

PowerShell Base64 encoding

PowerShell can convert malicious commands into Base64 strings, allowing execution with -EncodedCommand while hiding suspicious strings from security tools.

Remote delivery tools

Psexec and Evil-WinRM execute obfuscated payloads remotely using authenticated sessions, often running under SYSTEM privileges or encrypted WinRM channels.

Exam Focus

- Obfuscation changes payload appearance without changing functionality.
- Expect questions about Base64 PowerShell delivery through Psexec or Evil-WinRM.

User-Controlled Access Bypass (OBJ 4.4)

Misconfigured certificate templates in Active Directory Certificate Services allow attackers to request certificates for privileged authentication without passwords.

Kerberos manipulation with Rubius

Rubius supports ticket extraction, replay, renewal, and Golden Ticket forgery using the krbtgt hash, enabling long-term domain persistence.

Evil-WinRM post-exploitation channel

Evil-WinRM provides an encrypted PowerShell remote shell over WinRM for executing commands, uploading files, and expanding access across domain systems.

Exam Focus

- Certify and Rubius represent high-impact Active Directory escalation techniques.
- Golden Ticket persistence remains until krbtgt rotation occurs twice.

Shell and Kiosk Escapes (OBJ 4.4)

Shell escapes break out of restricted command environments by abusing allowed applications such as vim to spawn shells with commands like !sh.

PowerShell escape vectors

Invoke-Expression can execute cmd.exe from within restricted PowerShell sessions, bypassing intended lockdown controls.

Kiosk escape pathways

Kiosk escapes misuse allowed applications, PowerShell access, or WMI process control to restore full desktop environments from locked interfaces.

Exam Focus

- Restricted environments often fail due to overlooked application functionality.
- Know common escape vectors involving editors, Invoke-Expression, and WMI.

Log Tampering (OBJ 4.4)

Attackers manipulate or destroy logs to hide evidence of compromise, disrupting monitoring, auditing, and forensic reconstruction.

PowerShell log clearing

Clear-EventLog can wipe Security, Application, and System logs when executed with administrative or SYSTEM-level privileges.

Remote tampering with PsExec

PsExec enables clearing logs across remote hosts during lateral movement, erasing evidence without interactive access.

Exam Focus

- Clearing event logs demonstrates high privilege and strong attacker control.
- PsExec enables log tampering across multiple compromised systems.

Living Off the Land (OBJ 4.4)

Living off the land attacks abuse legitimate built-in tools such as PowerShell to perform malicious actions while blending into trusted system activity.

Execution and persistence utilities

Tools like regsvr32.exe execute remote scripts, while schtasks.exe creates scheduled tasks that run payloads with SYSTEM-level privileges.

Data movement with trusted binaries

bitsadmin.exe downloads payloads through BITS, while certutil.exe encodes and transfers data for stealthy exfiltration without external malware.

Exam Focus

- LOLBins reduce detection by relying on pre-installed trusted utilities.
- Expect scenarios involving regsvr32, schtasks, bitsadmin, and certutil abuse.

Section 20. Web Application Vulnerabilities

9 Lessons | Exam Objectives Covered: 4.3, 4.5 | Does not include demonstrations

Web Application Vulnerabilities (OBJ 4.3, 4.5)

Web application vulnerabilities are weaknesses in application design, implementation, or configuration that attackers exploit to bypass authentication, hijack sessions, access unauthorized data, or modify application behavior.

Offensive exploitation impact

These flaws represent direct entry points during penetration tests because exploitation can lead from account compromise to full system impact depending on severity and context.

Exam Focus

- Web vulnerabilities enable bypassing identity controls, session takeover, and unauthorized data access.

Race Conditions (OBJ 4.5)

Race conditions occur when computation outcomes depend on timing or event ordering, allowing attackers to force unintended execution sequences between threads or processes.

Memory-based race exploitation

Memory races can involve pointer dereferencing timing windows, with Dirty COW demonstrating privilege escalation through copy-on-write manipulation.

TOCTOU and locking behavior

TOCTOU flaws arise when resources change between validation and use, while incomplete locks or mutex handling can expose synchronization weaknesses.

Exam Focus

- Race conditions exploit timing gaps that are difficult to detect because they occur between system calls.
- Expect exam focus on TOCTOU validation bypass and Dirty COW style privilege escalation.

Buffer Overflows (OBJ 4.5)

Buffer overflows occur when programs write data beyond allocated memory boundaries, overwriting adjacent structures such as variables or return addresses.

Stack smashing and NOP slides

Attackers exploit overflows by corrupting stack return addresses to redirect execution, often using NOP slides to increase payload reliability.

Integer overflow support and mitigations

Integer overflows can cause incorrect buffer sizing, while defenses include patching, input validation, ASLR, and DEP.

Exam Focus

- Buffer overflows are low-level memory exploits that can lead directly to remote code execution.
- Know stack smashing, NOP slides, and mitigations like ASLR and DEP.

Authentication Flaws and Insecure References (OBJ 4.5)

Broken authentication occurs when login mechanisms are weak, enabling brute force, weak reset logic, or bypass through exposed credential data.

Weak session management risks

Predictable or insecure session identifiers allow attackers to hijack sessions without knowing credentials, especially when tokens appear in URLs or cookies.

IDOR exploitation

Insecure direct object references expose backend identifiers without authorization checks, allowing attackers to access other users' resources by changing parameters.

Exam Focus

- Broken authentication, weak sessions, and IDOR are high-impact web flaws leading to account compromise.
- Expect scenario questions involving brute force, session replay, and parameter manipulation.

Improper Error Handling (OBJ 4.5)

Error handlers catch unexpected conditions, and failures in handling can lead to crashes, unpredictable behavior, or unsafe processing paths.

Verbose error leakage

Stack traces, file paths, SQL queries, and class names revealed in errors provide attackers with reconnaissance for targeted exploitation.

Authentication enumeration paths

Distinct login error messages enable username enumeration, while timing differences can confirm valid accounts even without explicit disclosure.

Exam Focus

- Improper errors leak internal details that accelerate exploitation and credential-based attacks.
- Know how verbose login errors enable enumeration and brute force focusing.

Improper Headers (OBJ 4.5)

Headers like HSTS prevent HTTPS downgrade attacks, while HPKP pins public keys to reduce certificate spoofing risks.

Client-side protection headers

X-Frame-Options mitigates clickjacking, X-XSS-Protection and CSP reduce XSS risk, and X-Content-Type-Options prevents MIME-sniffing abuse.

Cross-domain and feature control

Referrer-Policy limits URL leakage, Expect-CT enforces certificate transparency, and Permissions-Policy restricts browser feature access.

Exam Focus

- Missing HTTP security headers expand attack surface for XSS, CSRF, clickjacking, and session interception.
- Expect exam scenarios involving header misconfiguration and browser-side exploitation.

Code Signing (OBJ 4.5)

Code signing uses cryptographic hashing and private-key signatures to prove software authenticity and detect post-signing modification.

Trust pipeline attack surface

If build systems or signing keys are compromised, attackers can insert malicious code that is signed as legitimate and bypasses reputation defenses.

Signed malware limitations

Code signing only verifies signer control of the key and file integrity, not whether the code is safe, as shown in the SolarWinds compromise.

Exam Focus

- Signed code can still be malicious if the signing environment is compromised.
- Expect exam focus on authenticity vs safety and supply-chain signing abuse.

Vulnerable Components (OBJ 4.5)

Client-side logic is exposed and modifiable, while server-side processing reduces direct tampering but requires backend enumeration.

REST, JSON, and SOAP risks

REST APIs commonly use JSON and are vulnerable to injection and authorization flaws, while SOAP's XML complexity introduces additional exposure.

Browser-side execution models

Extensions, JavaScript, HTML5, and AJAX expand attack vectors through exposed scripts, insecure APIs, and interceptable requests.

Exam Focus

- Component choice shapes attack surface across client logic, APIs, and browser execution environments.
- Expect questions on client-side trust, API injection, and exposed AJAX endpoints.

Software Composition (OBJ 4.5)

Applications inherit vulnerabilities from open-source libraries and frameworks, as shown by breaches like Equifax exploiting Apache Struts.

SCA tool visibility

Tools such as OWASP Dependency-Check and Dependency-Track enumerate components and map versions to known CVEs.

Additional software-level weaknesses

Unsafe functions, weak cryptography, misconfigurations, EoL software, information disclosure, and regression issues broaden exploitation paths.

Exam Focus

- Software composition vulnerabilities often originate from outdated or untracked dependencies.
- Expect exam focus on SCA workflows and supply-chain attack surface analysis

Section 21. Web Application Attacks

10 Lessons | Exam Objectives Covered: 4.5 | Does not include demonstrations

Web Application Attacks (OBJ 4.5)

Web application attacks focus on identifying and exploiting weaknesses in how a web application is built, configured, or executed. These attacks target flaws in code, logic, authentication flows, session handling, and supporting components to gain unauthorized access, extract sensitive data, or disrupt functionality. From an offensive standpoint, understanding the full attack surface, including client-side behaviors, server-side logic, APIs, and third-party dependencies, is essential for finding exploitable conditions.

Key Attack Topics Covered

- Directory Traversals, exploiting insufficient validation to access files outside the intended directory.
- Cross-Site Scripting (XSS), injecting malicious scripts into otherwise trusted content.
- Request Forgeries, including Cross-Site Request Forgery (CSRF) and Server-Side Request Forgery (SSRF).
- SQL Injections, manipulating database queries through unsanitized user input.
- Injection Attacks broadly, including XML injection, command injection, and server-side template injection.
- File Inclusions, local and remote, forcing a server to load unintended files.
- Arbitrary Code Execution, executing attacker-controlled code remotely on the server.
- Web Application Session Hijacking, exploiting weak session controls to impersonate users.
- OWASP ZAP, intercepting and testing traffic for vulnerabilities during development and assessment.

Exam Focus

- Expect scenarios that require identifying which web attack applies based on user input handling and server responses.
- Know how attacks like traversal, injection, and session hijacking connect directly to unauthorized access and exploitation outcomes.
- Be able to recognize how tools such as OWASP ZAP support vulnerability discovery and validation.

Directory Traversals (OBJ 4.5)

Directory traversal attacks exploit insufficient input validation to access files outside the intended web document root, often exposing sensitive operating system files or application data. These attacks commonly serve as early footholds that testers expand into deeper compromise.

Document root escape mechanics

Attackers manipulate file paths using sequences like `../` to climb the directory structure and escape the web document root. By chaining these traversal sequences, penetration testers can reach sensitive locations such as credential stores, configuration files, or system binaries.

File inclusion escalation

Traversal vulnerabilities frequently lead to local or remote file inclusion when applications dynamically load files based on user input. This escalation allows testers to force execution of files or scripts that were never intended to be accessible through the web interface.

Offensive impact

Successful directory traversal often reveals password hashes, API keys, or application secrets that enable further attacks. In many assessments, traversal is the stepping stone to privilege escalation or remote code execution.

Exam Focus

- Directory traversal abuses path handling to escape the web root.
- Expect traversal to be linked with LFI or RFI in exam scenarios.

Cross-Site Scripting (XSS) (OBJ 4.5)

Cross-site scripting is a client-side injection vulnerability that allows attackers to execute malicious scripts in a victim's browser by abusing improper input validation. XSS directly undermines the browser's trust model and enables user-level compromise.

XSS execution model

Injected scripts execute within the victim's browser with the same permissions as the trusted site. This allows attackers to read cookies, modify page content, and act on behalf of the user.

Attack sequence

Attackers identify input validation flaws and craft malicious payloads delivered through URLs, forms, or stored content. When the victim loads the page, the browser executes the injected JavaScript automatically.

Major XSS types

Reflected XSS executes immediately through crafted input, persistent XSS stores payloads in backend databases, and DOM-based XSS executes entirely within the client browser. Each type presents different exploitation paths but similar impact.

Exam Focus

- XSS enables session theft and user impersonation.
- Know reflected, stored, and DOM-based distinctions.

Request Forgeries (OBJ 4.5)

Request forgery attacks manipulate trust relationships to force systems into performing unintended actions. These attacks succeed without stealing credentials by abusing assumptions about request legitimacy.

CSRF abuse

Cross-Site Request Forgery tricks an authenticated browser into submitting unauthorized requests. Because cookies are automatically included, the server believes the action was legitimate.

SSRF abuse

Server-Side Request Forgery manipulates the application server into making outbound requests on the attacker's behalf. This often exposes internal services that are unreachable from the internet.

Trust exploitation

Both CSRF and SSRF exploit misplaced trust in request origin rather than breaking authentication. Penetration testers look for workflows that fail to validate intent or destination.

Exam Focus

- CSRF abuses browser trust while SSRF abuses server trust.
- Expect internal resource access scenarios with SSRF.

SQL Injections (OBJ 4.5)

SQL injection occurs when unvalidated user input alters the structure of database queries. This allows attackers to bypass authentication, extract sensitive data, or modify backend records.

SQL query manipulation

Applications often build SQL statements dynamically using user-supplied input. When this input is not sanitized, attackers can inject additional logic into SELECT, INSERT, UPDATE, or DELETE queries.

Boolean-based bypass

Boolean payloads such as ' OR 1=1 force database conditions to evaluate as true. This commonly results in authentication bypass during login workflows.

Mitigation awareness

Prepared statements and strict input validation prevent user input from altering SQL logic. Understanding these controls helps testers recognize where developers made incorrect trust assumptions.

Exam Focus

- SQL injection enables authentication bypass and data extraction.
- Know Boolean-based payload logic.

Injection Attacks (OBJ 4.5)

Injection attacks occur when applications mistakenly interpret user input as executable instructions. These vulnerabilities often lead to high-impact outcomes such as command execution or data exposure.

Command injection

Command injection passes user input directly into system shell commands without validation. Successful exploitation allows attackers to execute operating system commands beyond intended application behavior.

Server-side template injection (SSTI)

SSTI occurs when template engines evaluate attacker-supplied expressions instead of treating them as data. This can escalate from simple expression evaluation to full arbitrary code execution.

XML injection

XML injection abuses insecure XML parsing logic, often through external entity expansion. These flaws allow attackers to read files, access internal services, or exhaust system resources.

Exam Focus

- Injection flaws span OS, template, and parser layers.
- Expect input-as-code attack scenarios.

File Inclusions (OBJ 4.5)

File inclusion vulnerabilities arise when applications dynamically load files based on untrusted user input. These flaws commonly appear in scripting languages and legacy systems.

Remote file inclusion (RFI)

RFI allows applications to download and execute attacker-hosted files from external sources. This often results in immediate remote code execution on the target server.

Local file inclusion (LFI)

LFI enables attackers to read or execute files already present on the server. Penetration testers use traversal techniques to access configuration files, logs, or uploaded content.

RCE escalation

LFI frequently escalates to remote code execution (RCE) through file upload abuse or log poisoning. These techniques allow injected code to be interpreted by the application runtime.

Exam Focus

- RFI enables direct execution, LFI enables discovery and escalation.
- Know how LFI chains to RCE.

Arbitrary Code Execution (OBJ 4.5)

Arbitrary code execution allows attackers to run their own instructions on a target system. This outcome represents a full compromise of application-level trust.

Execution outcome

Once code execution is achieved, attackers gain control over system resources and processes. This enables credential theft, lateral movement, and persistence.

Deserialization abuse

Deserialization attacks occur when applications reconstruct objects from untrusted data. Attackers leverage gadget chains to trigger unintended method execution.

Web shell persistence

Web shells provide browser-accessible command execution interfaces. They allow attackers to maintain interactive access after initial exploitation.

Exam Focus

- Arbitrary code execution is a critical exploitation outcome.
- Expect deserialization and web shell scenarios.

Web Application Session Hijacking (OBJ 4.5)

Session hijacking attacks target session identifiers used to maintain authenticated user state. These attacks exploit weaknesses in how sessions are generated, transmitted, or protected.

Session fixation

Session fixation forces a victim to authenticate using a session ID known to the attacker. If the application fails to regenerate the ID after login, takeover is immediate.

Session replay

Session replay captures valid session tokens and reuses them to impersonate users. This is common in applications using weak encryption or unprotected HTTP.

XSS-based theft

XSS-based session hijacking uses injected scripts to steal cookies. Missing HttpOnly flags and weak CSP controls make this attack trivial.

Exam Focus

- Session hijacking abuses stolen or controlled session IDs.
- Know fixation, replay, and XSS-based theft.

OWASP ZAP (OBJ 4.5)

OWASP ZAP is a web application penetration testing proxy used to intercept and analyze traffic. It supports both automated discovery and manual exploitation workflows.

Proxy visibility

ZAP intercepts all HTTP requests and responses between the browser and server. This visibility exposes cookies, headers, tokens, and hidden parameters.

Automated and manual scanning

Automated scans identify common vulnerabilities quickly, while manual exploration captures complex authenticated workflows. Both approaches are necessary during real engagements.

HUD and enumeration

The Heads-Up Display injects testing controls directly into the browser. This accelerates discovery and validation of vulnerabilities during navigation.

Exam Focus

- OWASP ZAP is a core proxy and scanning tool.
- Expect questions on spiders, active scans, and proxy workflows.

Section 22. Cloud Attacks

12 Lessons | Exam Objectives Covered: 4.6 | Does not include demonstrations

Cloud Attacks (OBJ 4.6)

Cloud attacks focus on exploiting weaknesses that arise from the unique architecture, shared-responsibility model, and automation inherent to cloud environments. These weaknesses commonly stem from misconfigurations, overly permissive access controls, insecure APIs, and trust relationships between services and third parties. Because cloud platforms are widely adopted and deeply integrated into modern business operations, exploitation of these flaws can expose entire environments rather than individual systems.

Exam Focus

- Recognize cloud-specific attack scenarios, particularly those involving misconfigured storage, overly permissive IAM roles, exposed APIs, or abused trust relationships within the shared-responsibility model.

Identity and Access Management (IAM) Misconfigurations (OBJ 4.6)

IAM misconfigurations represent one of the most critical attack paths in cloud environments because they directly control who can access resources and what actions they can perform.

IAM structure and purpose

IAM defines how users, services, and applications authenticate and what permissions they are granted within a cloud environment. From an offensive perspective, weak IAM design often provides the fastest route to privilege escalation.

Common IAM misconfigurations

Overly permissive policies, wildcard permissions, improperly scoped roles, and default credentials allow attackers to inherit excessive privileges when an account is compromised. These misconfigurations frequently expose administrative capabilities.

Credential exploitation

Exposed access keys allow attackers to authenticate directly to cloud provider APIs. Once obtained, attackers can enumerate resources, modify IAM policies, and establish persistence.

Exam Focus

- IAM misconfigurations commonly lead directly to privilege escalation.
- Expect scenarios involving exposed access keys and overly permissive roles.

Resource Misconfigurations (OBJ 4.6)

Resource misconfigurations weaken the boundaries that should isolate systems and services in cloud environments.

Network segmentation failures

Poor segmentation allows attackers to move laterally once initial access is gained. Flat networks expose sensitive systems to compromise from low-privilege entry points.

Misconfigured network controls

Overly permissive firewall rules and ACLs expose internal services to unnecessary access. These misconfigurations provide direct attack paths into protected systems.

Publicly exposed assets

Storage buckets and services configured for public access often leak sensitive data. Penetration testers frequently discover critical data exposure through these assets.

Exam Focus

- Resource misconfigurations enable lateral movement and data exposure.
- Expect exam questions involving public cloud storage and permissive firewall rules.

Logging Information Exposures (OBJ 4.6)

Logging information exposures occur when logs reveal sensitive data or are improperly protected.

Sensitive data in logs

Logs often capture credentials, tokens, and personal data when applications over-log request details. Attackers can reuse this information to authenticate or escalate access.

Insecure log storage

Logs stored in publicly accessible directories or weakly permissioned buckets can be downloaded by attackers. These logs frequently reveal internal paths and system behavior.

Unencrypted log transmission

Logs transmitted without encryption can be intercepted on the network. This allows attackers to capture sensitive operational data passively.

Exam Focus

- Logs frequently contain high-value credentials and tokens.
- Expect scenarios involving exposed or intercepted log data.

Metadata Service Attacks (OBJ 4.6)

Metadata service attacks exploit SSRF vulnerabilities to extract cloud instance credentials.

Metadata service purpose

Cloud metadata services provide configuration details and temporary IAM credentials to instances. This information is intended to be accessible only internally.

SSRF as an attack vector

SSRF allows attackers to force servers to request internal metadata endpoints. This bypasses network restrictions protecting the metadata service.

Credential extraction and pivoting

Retrieved metadata credentials allow direct API access to cloud resources. Attackers can escalate privileges and pivot deeper into the cloud environment.

Exam Focus

- Metadata attacks rely on SSRF to retrieve IAM credentials.
- Expect exam focus on the metadata IP address and credential extraction.

Image and Artifact Tampering (OBJ 4.6)

Image and artifact tampering targets foundational components used to deploy systems and applications.

VM image tampering

Compromised VM images silently propagate backdoors and weakened controls into every deployed instance. Attackers gain persistent access across environments.

Container image tampering

Malicious container images distribute malware or insecure configurations at scale. A single poisoned image can compromise entire clusters.

Software artifact tampering

Compromised libraries and dependencies execute malicious code inside applications. Supply-chain poisoning provides deep, persistent access.

Exam Focus

- Tampered images and artifacts propagate compromise at scale.
- Expect questions on poisoned images and dependency risks.

Supply Chain Attacks (OBJ 4.6)

Supply chain attacks compromise trusted vendors, dependencies, or update mechanisms.

Third-party dependency compromise

Malicious libraries introduce backdoors into applications that trust them. These attacks bypass internal defenses.

Software update pipeline abuse

Compromised updates distribute malicious code to all customers automatically. This creates widespread compromise.

Upstream provider risks

Cloud providers, managed services, and hardware vendors introduce risk through trust relationships. Weak oversight enables broad impact.

Exam Focus

- Supply chain attacks bypass internal security through trusted sources.
- Expect references to compromised updates and libraries.

Container Exploits and Attacks (OBJ 4.6)

Container attacks target applications, runtimes, and orchestration platforms.

Workload runtime attacks

Traditional vulnerabilities inside containers provide initial access. Exploiting the application is often the first step.

Container escapes

Misconfigurations and runtime flaws allow attackers to break isolation. Escaping a container often leads to host compromise.

Container assessment tools

Tools like Kube-Hunter and Docker Bench identify misconfigurations and exposed control-plane components.

Exam Focus

- Container exploits often begin inside the workload.
- Expect exam focus on container escapes and assessment tools.

Trust Relationship Abuse (OBJ 4.6)

Trust relationship abuse exploits authentication links between systems and services.

Lateral movement via trust

Compromised trusted systems can authenticate to others without revalidation. This enables rapid expansion of attacker access.

Kerberos and SPN abuse

Unconstrained delegation and weak service accounts allow impersonation of privileged users. Kerberoasting exposes service account credentials.

Cloud trust abuse

Federated identity and OAuth permissions extend trust across platforms. Overly broad permissions allow privilege escalation.

Exam Focus

- Trust abuse acts as a force multiplier for attackers.
- Expect exam scenarios involving delegation and federation.

Third-party Integration Exploits (OBJ 4.6)

Third-party integrations introduce external attack paths into internal systems.

Insecure APIs

APIs without strong authentication or authorization allow unauthorized access. Attackers exploit these endpoints to retrieve or modify data.

Vulnerable libraries

Unpatched third-party components introduce exploitable flaws. Attackers leverage these for code execution or data theft.

Insecure webhooks

Unauthenticated webhooks allow attackers to send forged requests. This enables manipulation of internal workflows.

Exam Focus

- Third-party integrations often bypass perimeter defenses.
- Expect API and webhook exploitation scenarios.

Cloud Security Testing (OBJ 4.6)

Cloud security testing uses specialized tools to identify and exploit misconfigurations.

ScoutSuite auditing

ScoutSuite enumerates configurations across AWS, Azure, and GCP. It highlights structural weaknesses early in an assessment.

Pacu exploitation

Pacu actively exploits AWS misconfigurations. It demonstrates real-world impact through automated attack chains.

Prowler and native tools

Prowler and cloud-native tools expose gaps in logging, identity, and configuration. These gaps often enable stealthy exploitation.

Exam Focus

- Cloud security testing blends auditing with exploitation.
- Expect questions comparing tools and their use cases.

Conducting Cloud Audits (OBJ 4.6)

Cloud audits systematically evaluate configurations and security posture.

Audit objectives

Audits identify misconfigurations, excessive permissions, and compliance gaps. These findings map directly to exploitable weaknesses.

Configuration review

Reviewing IAM, logging, networking, and storage reveals high-impact risks. Audits often uncover issues missed by manual testing.

Offensive value

Audit results guide exploitation and demonstrate business risk. They provide structured evidence for remediation.

Exam Focus

- Cloud audits reveal systemic misconfigurations.
- Expect exam focus on audit goals and outcomes.

Section 23. Attacking Specialized Systems

9 Lessons | Exam Objectives Covered: 4.9 | Does not include demonstrations

Attacking Specialized Systems (OBJ 4.9)

Attacks on specialized systems focus on exploiting weaknesses in non-traditional computing environments such as mobile devices, wireless authentication technologies, operational technology, and artificial intelligence platforms. These systems are typically designed with priorities like safety, portability, automation, or real-time control rather than traditional IT security models, which introduces unique attack surfaces attackers can exploit.

Exam Focus

- Recognize attack techniques specific to specialized systems, such as mobile exploitation, NFC/RFID abuse, AI manipulation, or ICS/OT protocol weaknesses, and identify how their unique design constraints create distinct attack surfaces.

Mobile Device Attacks (OBJ 4.9)

Mobile device attacks target vulnerabilities in smartphones and tablets to compromise operating systems, applications, communications, or stored data. Because users frequently install apps, connect to untrusted networks, and carry devices everywhere, mobile platforms present a large and constantly changing attack surface.

Root-level compromise

Root-level compromise occurs through jailbreaking on iOS or rooting on Android, both of which grant administrative privileges on the device. Once a device is jailbroken or rooted, built-in protections like sandboxing, code signing, and secure boot are bypassed, making persistent malware installation and privilege abuse far easier.

Application installation risks

Sideloaded and installing unsigned applications bypass official app store security checks and developer code-signing guarantees. These applications may contain hidden malware, credential stealers, or backdoors, and their presence often indicates that the device is already operating outside normal security boundaries.

Mobile security controls

Organizations rely on controls such as MDM profiles, encryption, VPNs, and location-based restrictions to protect mobile devices. When these controls are

misconfigured or abused, attackers can intercept traffic, bypass access restrictions, or exfiltrate sensitive corporate and personal data.

Exam Focus

- Jailbroken or rooted devices are considered compromised by default.
- Expect exam scenarios involving sideloaded apps and misconfigured mobile controls.

Tools for Mobile Device Attacks (OBJ 4.9)

Mobile penetration testing relies on specialized tools that allow testers to analyze application code, manipulate runtime behavior, and interact directly with devices. These tools address attack surfaces unique to mobile platforms, such as app sandboxes and inter-process communication.

MobSF analysis

MobSF performs both static and dynamic analysis of mobile applications to uncover insecure code, exposed secrets, and risky behaviors. It quickly provides insight into permissions, API usage, and potential vulnerabilities that warrant deeper manual testing.

Frida and Objection

Frida enables runtime instrumentation of mobile applications, allowing testers to intercept function calls, bypass security checks, and observe sensitive data before encryption. Objection builds on Frida by automating common exploitation tasks, reducing the effort required to reach application internals.

Drozer and ADB

Drozer targets Android component misconfigurations by exploiting insecure IPC mechanisms, such as exported activities or content providers. Android Debug Bridge provides low-level device interaction, enabling file-system access, traffic forwarding, and direct inspection of application data.

Exam Focus

- Know which tools support static analysis versus runtime manipulation.
- Expect questions mapping mobile tools to specific attack objectives.

Bluetooth Attacks (OBJ 4.9)

Bluetooth attacks exploit short-range wireless communication that many users leave enabled by default. These attacks help assess wireless hygiene and user susceptibility to unsolicited interactions.

Bluejacking

Bluejacking involves sending unsolicited messages to Bluetooth-enabled devices operating in discoverable mode. While it does not grant system access, it demonstrates how easily attackers can interact with misconfigured devices and potentially support social engineering.

Bluetooth spamming

Bluetooth spamming repeatedly sends messages or files to overwhelm a target device. When file transfers are involved, users who accept them may unknowingly install malicious payloads that exploit device vulnerabilities.

Discoverable mode risks

Devices in discoverable mode actively broadcast their presence to nearby attackers. This configuration dramatically increases exposure and provides an immediate attack path during assessments.

Exam Focus

- Bluetooth attacks depend on discoverable mode.
- Expect distinctions between Bluejacking and Bluetooth spamming.

NFC and RFID Attacks (OBJ 4.9)

NFC and RFID attacks target wireless authentication systems that operate over the air rather than through direct physical contact. These technologies are common in access badges and mobile authentication systems.

RFID cloning

RFID tags transmit identifiers using radio waves and can often be intercepted and cloned without user awareness. Older, unencrypted RFID standards are especially vulnerable to replay and cloning attacks.

NFC exploitation

NFC operates at close range but is widely supported by smartphones, making attacks accessible without specialized hardware. Attackers can read, modify, or emulate NFC tags, sometimes extending range slightly through amplification.

Wireless authentication risks

Both RFID and NFC rely heavily on possession-based authentication. Captured signals can be replayed later, allowing unauthorized access without compromising the original device.

Exam Focus

- Legacy RFID implementations are easily cloned.
- Expect exam questions comparing RFID and NFC characteristics.

AI Attacks (OBJ 4.9)

AI attacks exploit how models interpret input and learn from data, introducing new attack surfaces beyond traditional software vulnerabilities. As AI systems become embedded in business workflows, these attacks are increasingly relevant.

Prompt injection

Prompt injection occurs when attackers craft malicious inputs that cause AI systems to ignore restrictions or reveal sensitive information. This attack is similar to injection flaws in traditional applications but targets language-based reasoning.

Model manipulation

Model manipulation targets the AI system itself by altering training data, parameters, or the deployment environment. These changes can subtly influence outputs or create hidden backdoors that are difficult to detect.

Operational impact

Compromised AI systems may misclassify data, leak information, or behave unpredictably. Because manipulation can be subtle, attackers may maintain long-term influence without immediate detection.

Exam Focus

- Prompt injection targets input, model manipulation targets internals.
- Expect AI-specific terminology on the exam.

Operational Technology (OT) (OBJ 4.9)

Operational Technology controls physical processes rather than digital information. These systems are critical to industries like manufacturing, energy, and transportation.

OT characteristics

OT environments prioritize availability and safety over confidentiality. Failures can cause physical damage, environmental harm, or loss of life.

ICS components

Industrial Control Systems use PLCs, HMIs, and control loops to automate machinery. Manipulating logic directly affects physical behavior.

SCADA environments

SCADA systems manage geographically distributed infrastructure. Connectivity increases attack surface and potential impact.

Exam Focus

- OT security priorities differ from IT systems.
- Expect distinctions between ICS and SCADA.

OT Attacks (OBJ 4.9)

OT attacks exploit insecure industrial communication protocols that control real-world machinery. These protocols often lack modern security protections.

CAN bus attacks

CAN bus messaging lacks authentication and trusts all commands. Injected messages can directly control vehicles or machinery.

Modbus exploitation

Modbus accepts unauthenticated read and write commands. Improper handling can disrupt or damage industrial processes.

Advanced OT protocols

DDS and SIS introduce additional risks when misconfigured. Safety systems require extreme caution during testing.

Exam Focus

- OT protocols often lack authentication and encryption.
- Expect protocol-specific OT attack scenarios.

Testing OT Systems (OBJ 4.9)

Testing OT systems requires specialized methods to avoid disrupting physical processes. Traditional IT tools can introduce significant risk.

Wireshark risks

Packet capture can disrupt timing-sensitive control loops. Misinterpreted protocol data can mislead testers.

Tcpdump limitations

Even lightweight capture introduces latency. Embedded devices may fail under inspection overhead.

Scapy dangers

Packet injection can unintentionally trigger real-world actions. Use only in lab or simulation environments.

OT-safe tools

Passive monitoring platforms and digital twins enable safer assessment. These tools decode protocols without disruption.

Exam Focus

- Traditional IT tools can destabilize OT systems.
- Expect questions on safe OT testing practices.

Section 24. Automated Attacks

8 Lessons | Exam Objectives Covered: 4.10 | Does not include demonstrations

Automated Attacks (OBJ 4.10)

Automated attacks use scripts, frameworks, and specialized tools to perform reconnaissance, exploitation, and post-exploitation tasks at scale with minimal human effort. By automating workflows, attackers can rapidly probe thousands of hosts, weaponize vulnerabilities quickly, and execute repeatable attack chains that would be impossible to perform manually.

Exam Focus

- Recognize how scripting and automation tools are used to scale offensive tasks, such as rapid enumeration, automated exploitation, and repeatable post-exploitation workflows.

Empire/PowerSploit (OBJ 4.10)

Empire and PowerSploit are powerful PowerShell-based offensive frameworks used to automate post-exploitation tasks after initial access. Together they support credential harvesting, privilege escalation, persistence, and lateral movement inside Windows enterprise environments.

Empire command-and-control automation

Empire provides a post-exploitation command-and-control platform that manages compromised hosts through in-memory PowerShell agents. By avoiding direct PowerShell.exe execution, Empire helps bypass many defensive detections while maintaining reliable remote control.

Listener and agent workflow

Empire uses listeners as callback endpoints that allow payloads to connect back and establish agents. Once an agent is active, penetration testers can execute modules like Mimikatz for credential dumping and quickly pivot deeper into the network.

PowerSploit privilege escalation and execution

PowerSploit is a collection of offensive PowerShell scripts that automate reconnaissance, exploitation, and privilege escalation checks. Functions like Invoke-AllChecks rapidly identify misconfigurations, while Invoke-Shellcode enables in-memory payload execution without writing files to disk.

Exam Focus

- Empire focuses on agent-based post-exploitation control, while PowerSploit provides modular offensive scripts.
- Expect exam scenarios involving privilege escalation checks and in-memory execution techniques.

PowerView (OBJ 4.10)

PowerView is a PowerShell reconnaissance tool used to automate enumeration inside Active Directory environments. It helps penetration testers map users, groups, systems, and trust relationships to support escalation and lateral movement planning.

Domain user and group enumeration

PowerView can list all domain users and groups, giving attackers immediate visibility into account structure and privilege organization. Identifying high-value groups like Domain Admins helps prioritize credential targeting and escalation efforts.

Computer and trust discovery

PowerView enumerates domain-joined computers and reveals trust relationships between domains. These insights allow attackers to identify pivot opportunities into connected environments or higher-privilege domains.

Session awareness for targeting

PowerView can detect active sessions on remote hosts, showing where privileged users are logged in. This reconnaissance helps attackers choose the most valuable machines for credential theft or token-based attacks.

Exam Focus

- PowerView is essential for Active Directory reconnaissance and attack path mapping.
- Expect questions about enumeration commands and identifying privileged targets.

PowerUpSQL (OBJ 4.10)

PowerUpSQL is a PowerShell toolkit designed to discover and exploit Microsoft SQL Server environments. SQL Servers are high-value targets because they often store sensitive data and provide escalation paths into Windows domains.

SQL Server instance discovery

PowerUpSQL automates locating SQL Server instances across local systems and domains. Identifying where SQL services run is the first step in building an attack chain against database infrastructure.

Configuration and credential weaknesses

PowerUpSQL enumerates server details such as authentication mode, roles, and version information. It can also check for default or weak credentials that frequently remain unchanged in enterprise deployments.

Privilege escalation and lateral movement

PowerUpSQL automates privilege escalation attempts inside SQL Server and identifies accounts trusted for delegation. Linked SQL Servers act as lateral movement bridges, allowing attackers to pivot across database systems.

Exam Focus

- Default credentials and unsafe stored procedures are common SQL Server attack vectors.
- Expect exam focus on SQL escalation and linked server movement.

AD Search (OBJ 4.10)

Automating Active Directory searches with PowerShell allows penetration testers to gather reconnaissance data quickly and consistently. Efficient AD searching reveals privilege relationships, stale accounts, and high-value targets for escalation.

Active Directory cmdlet automation

PowerShell includes AD-specific cmdlets that allow scripted enumeration of users, groups, and computers. This automation accelerates reconnaissance and reduces manual effort in large environments.

Filtering for high-value targets

Commands like Get-ADUser can be combined with filters to isolate managers, dormant accounts, or users with specific attributes. These filtered searches help attackers focus on accounts most likely to provide escalation opportunities.

Reporting and OU-wide scripting

Pipelines can export privileged group membership into structured reports for targeting and engagement tracking. Looping through organizational units enables large-scale searches for disabled or misconfigured accounts across the directory.

Exam Focus

- PowerShell AD automation supports rapid reconnaissance and privilege targeting.
- Expect questions about filtering and reporting privileged accounts.

Caldera (OBJ 4.10)

Caldera is an open-source adversary emulation platform built on the MITRE ATT&CK framework. It automates realistic attack behaviors to test how well an environment detects and resists offensive techniques.

Breach and attack simulation (BAS)

Caldera supports BAS by replicating attacker behavior in a controlled environment. This helps validate whether theoretical attack paths are actually exploitable and detectable in practice.

Abilities and attack chain automation

Caldera uses modular abilities mapped to ATT&CK techniques, allowing testers to build customized operations. These operations can simulate full kill chains such as credential dumping followed by lateral movement.

Agents and reporting

Lightweight agents execute assigned abilities on target systems and report results back to the server. Caldera's structured logs highlight which techniques succeeded, failed, or triggered defensive responses.

Exam Focus

- Caldera aligns simulations directly to MITRE ATT&CK techniques.
- Expect exam scenarios involving BAS automation and repeatable attack validation.

Infection Monkey (OBJ 4.10)

Infection Monkey is an open-source breach and attack simulation tool that mimics malware propagation to reveal weaknesses in segmentation, patching, and credential hygiene. It helps penetration testers understand how attackers might spread internally.

Controlled malware-like simulation

Infection Monkey emulates worm behavior without causing real damage, allowing safe observation of attacker movement. It demonstrates how quickly weak credentials and misconfigurations enable propagation.

Agent and Monkey Island architecture

The Agent performs simulated exploitation and lateral movement attempts, while Monkey Island acts as the command-and-control server. This structure provides real-time monitoring of attack progression.

Actionable security insights

Reports show which hosts were compromised, which techniques succeeded, and what defenses failed. Repeatable simulations allow validation of remediation and improved resilience over time.

Exam Focus

- Infection Monkey focuses on lateral movement and internal propagation testing.
- Expect questions about simulation components and reporting value.

Atomic Red Team (OBJ 4.10)

Atomic Red Team is an open-source tool that enables small, modular tests of individual MITRE ATT&CK techniques. It provides fast validation of whether specific attacker behaviors are detected or blocked.

Atomic tests and ATT&CK mapping

Atomic Red Team breaks ATT&CK techniques into minimal self-contained simulations called atomic tests. This allows controlled execution of one technique at a time without full attack infrastructure.

PowerShell execution workflow

Tests are executed using simple Invoke-AtomicTest commands that simulate behaviors like credential dumping or persistence. This makes the tool accessible and repeatable for rapid validation.

Detection gap identification

Running atomic tests reveals which attacker actions trigger alerts and which go unnoticed. This helps penetration testers pinpoint monitoring gaps and refine offensive workflows.

Exam Focus

- Atomic Red Team validates individual ATT&CK techniques quickly.
- Expect exam focus on atomic tests and detection validation use cases.

Section 25. Persistence

8 Lessons | Exam Objectives Covered: 5.1 | Does not include demonstrations

Persistence (OBJ 5.1)

This section focuses on persistence: the techniques an attacker uses to maintain long-term, reliable access to a compromised environment after initial compromise. Persistence matters because it is how an attacker survives reboots, credential resets, patching, and cleanup efforts, allowing them to return, monitor, and continue post-exploitation activities throughout an engagement. In Domain 5 (Post-exploitation and Lateral Movement), Objective 5.1 expects you to perform tasks that establish and maintain persistence in realistic scenarios.

Across the lessons, you will study how command and control (C2) frameworks manage compromised hosts, how persistence can be automated so access reappears after disruptions, and how remote shells and backdoors enable continued command execution. You will also examine credential-based persistence through account creation, browser-based persistence through extensions and stored session artifacts, and security control tampering that disables defensive barriers and logging so persistence becomes harder to detect.

Exam Focus

- Recognize and implement persistence mechanisms, such as C2 callbacks, account creation, scheduled tasks, service modification, or backdoors, that allow continued access after system reboots or credential changes.

Command and Control (OBJ 5.1)

Command and control (C2) frameworks are the remote management layer used after a host is compromised, letting a penetration tester control agents, run post-exploitation tasks, and coordinate activity across multiple machines. In practice, C2 turns a single foothold into an operational platform for credential harvesting, persistence, and lateral movement, while also helping you keep actions organized and repeatable during an engagement.

Empire

Empire is a post-exploitation framework built around PowerShell and Python agents, and it is known for running PowerShell in memory instead of relying on powershell.exe. That design choice matters because many defensive detections focus on powershell.exe execution, so in-memory execution can reduce obvious indicators while

still enabling actions like keylogging, credential dumping, and lateral movement modules.

Covenant

Covenant is a .NET-based C2 framework whose agents (“Grunts”) are designed to run well in enterprise environments that heavily use the .NET ecosystem. Its web-based interface provides a centralized dashboard for agents, listeners, and tasking history, which helps you track privilege level, host context, and what actions were run on each machine during post-exploitation.

Mythic

Mythic is a modular, cross-platform C2 framework that emphasizes flexible payload selection instead of a single built-in agent type. Because payloads can be tailored to the operating system and engagement needs (including macOS-focused payloads), Mythic supports more customized operations and easier integration with automation scripts and other tooling through its API-driven design.

Exam Focus

- Know what a C2 framework provides after compromise: agent management, tasking, payload deployment, credential access, and coordinated post-exploitation workflows.
- Be able to compare Empire, Covenant, and Mythic at a high level: Empire’s in-memory PowerShell focus, Covenant’s .NET Grunts and web UI, and Mythic’s modular payload architecture.
- Recognize why C2 is valuable in scenarios: it enables controlled operations at scale and supports persistence and lateral movement from a centralized operator view.

Automating Persistence (OBJ 5.1)

Automating persistence means using operating-system features to ensure access returns even after reboots, logouts, or process termination, so you do not lose a foothold after the initial compromise. From an offensive perspective, automation reduces the need for constant manual re-entry and makes persistence more resilient by re-launching payloads on a schedule, at boot, or at user login.

Scheduled Tasks and Cron Jobs

Scheduled tasks (Windows) and cron jobs (Unix-like systems) are built-in schedulers that can repeatedly execute a chosen payload at fixed times or intervals. This is powerful because the operating system itself becomes responsible for re-running the payload, so even if a defender kills the current session, the scheduled execution can re-establish access later under the same user context.

Service Creation

Services are long-running processes that commonly start at boot and often run with elevated privileges, making them a reliable persistence layer when you can create or modify them. A malicious service blends into expected background activity and persists through reboots, which is why service-based persistence can outlast simpler mechanisms like a single user-level startup entry.

Registry Run Keys

Windows startup registry keys control what programs execute at user login or system startup, so adding a payload reference can create consistent re-launch behavior. Using user-level keys like HKCU can work without full administrative privileges, while system-wide keys like HKLM typically require elevated access but can provide broader persistence across all users.

Exam Focus

- Understand the purpose of automation in persistence: reliable re-execution after reboot, logout, or process termination.
- Know common persistence mechanisms by category: schedulers (schtasks/cron), services (sc/service managers), and Windows startup registry keys (Run entries).
- Be ready to explain layered persistence in scenarios: combining multiple mechanisms increases resilience if one method is removed.

Remote Shells (OBJ 5.1)

Remote shells provide interactive command execution on a compromised system from a distance, making them foundational for maintaining control during post-exploitation. They matter because they let you continue enumeration, privilege escalation, data access, and tool execution without needing repeated exploitation of the original entry point.

Reverse Shells

A reverse shell is created when the compromised host initiates an outbound connection back to the attacker's listener, which often helps bypass inbound firewall rules. Because outbound traffic is commonly allowed, reverse shells can blend into normal egress patterns, and once connected they provide a live channel for running commands as if you were at the target's terminal.

Bind Shells

A bind shell works in the opposite direction: the compromised host opens a listening port and waits for the attacker to connect inbound to that port. This can be useful when

outbound connections are heavily restricted, but it also depends on firewall rules allowing inbound access to the chosen port, which may be less common in tightly controlled environments.

Why Shell Direction Matters

Choosing between reverse and bind shells is largely about network controls, including which direction of traffic is permitted and how monitoring is configured. In scenario questions, you should connect the shell type to the constraint being described, such as restrictive inbound filtering favoring reverse shells or restrictive outbound filtering favoring bind shells.

Exam Focus

- Be able to distinguish reverse shells vs. bind shells based on who initiates the connection and which firewall direction they are intended to bypass.
- Understand why remote shells support persistence: they provide continuing interactive control for post-exploitation workflows.
- In scenarios, tie shell choice to network constraints (inbound vs. outbound filtering) and operational requirements.

Backdoors (OBJ 5.1)

Backdoors provide a method to regain access to a compromised system without reusing the original vulnerability, which makes them valuable for long-term engagements. They often emphasize stealth and durability, giving continued command execution and re-entry even if the initial access vector is patched or monitored.

Trojans

A trojan is malicious code disguised as legitimate software and typically requires user execution, which makes it closely tied to social engineering and user behavior. Once executed, a trojan can install a hidden access mechanism that calls back to an attacker-controlled system, allowing persistence while appearing like normal application activity.

Web Shells

A web shell is server-side script code uploaded to a web server that provides remote administration through a web interface, commonly after a file-upload weakness or insecure configuration is found. Because web shells can blend into normal application directories and execute on demand, they can persist even after the original vulnerability is fixed, enabling continued access and internal pivoting from the server.

Rootkits

Rootkits are designed to hide processes, files, and activity while maintaining privileged access, often operating at the kernel level where they can intercept system behavior. That depth makes rootkits difficult to detect and remove, and they demonstrate how persistence can evolve from simple re-entry mechanisms into long-term, stealthy control of the operating system itself.

Exam Focus

- Know the purpose of backdoors: regain access without repeating the initial exploit, supporting long-term post-exploitation.
- Be able to differentiate trojans, web shells, and rootkits by where they operate and how they provide access (user-executed disguise, server-side script control, kernel-level stealth).
- In scenarios, connect the backdoor type to the environment: web app/server context favors web shells, deep stealth favors rootkits, and user-targeted delivery favors trojans.

Account Credentials (OBJ 5.1)

Credential-based persistence focuses on obtaining valid authentication material and using it to maintain access even if the original compromise method is closed. From an offensive standpoint, using legitimate credentials and accounts blends into routine administrative behavior and can provide durable access across systems without noisy exploit traffic.

Credential Acquisition Methods

Attackers acquire credentials through techniques like credential dumping from memory, capturing keystrokes with a keylogger, or harvesting logins through phishing workflows. These approaches matter because they provide legitimate usernames, passwords, hashes, or tickets that can be reused for access and lateral movement, often reducing reliance on exploit-based re-entry.

Account Creation on Windows and Unix-like Systems

Creating new accounts provides an access path independent of the originally compromised user, so account resets or disablement do not necessarily remove the attacker's foothold. Built-in utilities can create accounts and assign elevated group membership, which is attractive in post-exploitation because the actions resemble standard system management rather than obvious malware deployment.

Hiding and Blending Accounts

Long-term persistence often includes hiding or masking accounts so they do not appear in obvious places like login screens or routine user lists. Techniques include adjusting

Windows settings that control which users are displayed and modifying authentication configurations on Unix-like systems so access can remain available while appearing less visible during casual review.

Exam Focus

- Understand why valid credentials are persistent: they keep working even after a vulnerability is patched if accounts and permissions remain active.
- Know common credential acquisition approaches described in the lesson: dumping, keylogging, and phishing.
- Be able to explain why account creation and concealment can be effective persistence: it blends into normal administrative activity and survives many cleanup actions.

Browser-Based Persistence (OBJ 5.1)

Browser-based persistence uses the web browser as a long-lived access point because browsers are constantly used and often trusted by users. By embedding persistence in extensions, stored session artifacts, or browser settings, an attacker can maintain ongoing control or data access in a way that blends into normal browsing behavior.

Malicious Browser Extensions

Extensions can request powerful permissions such as reading and modifying website content, which can enable credential theft, data collection, or traffic manipulation. If a user is tricked into installing a malicious extension, the attacker gains a durable foothold that persists as long as the extension remains installed and active within the browser.

Cookies and Local Storage Abuse

Cookies and local storage hold session and application data that can be abused to maintain access to accounts, especially when session tokens are stolen or manipulated. When attackers can inject or store malicious content through weaknesses like cross-site scripting, that stored data can re-trigger malicious behavior whenever the user revisits the affected site or web application.

Browser Settings and Vulnerability Exploitation

Attackers may modify settings such as the default homepage, search engine, or proxy configuration to redirect traffic and enable interception or forced exposure to malicious content. In higher-impact situations, browser vulnerabilities can enable code execution and the installation of persistent scripts or tooling that runs when the browser launches, extending persistence beyond a single site or session.

Exam Focus

- Be able to describe browser-based persistence categories: extensions, session/storage artifacts (cookies/local storage), and configuration or vulnerability-based persistence.
- Understand why browsers are attractive persistence targets: high user interaction, trust, and frequent reuse across sessions.
- In scenarios, connect the persistence mechanism to how it survives: installed extension longevity, stored token reuse, or persistent configuration changes like proxy settings.

Security Control Tampering (OBJ 5.1)

Security control tampering supports persistence by weakening or disabling defenses that would otherwise block access, detect tooling, or preserve evidence. From an attacker's viewpoint, removing guardrails like firewalls, endpoint protections, and logging increases the reliability of C2 callbacks, remote shells, and implanted persistence mechanisms across reboots and cleanup attempts.

Disabling Firewalls and Endpoint Protections

Disabling a firewall reduces network restrictions that can interrupt reverse shells or C2 communications, making remote access more reliable once a foothold exists. Similarly, disabling antivirus or other endpoint tools prevents them from quarantining payloads and scripts, allowing persistence artifacts to remain in place long enough to be reused.

Policy, Registry, and Logging Manipulation

Weakening system security policies can reduce friction for account creation, privilege changes, and repeated access, especially when restrictions like lockout policies or prompts are relaxed. Disabling or reducing logging is a major force-multiplier because it limits the evidence defenders rely on for detection and investigation, increasing the chance that persistence remains unnoticed during routine review.

Startup and Scheduled Execution as a Fallback Layer

Scheduled tasks and startup scripts can be used as a persistence safety net so payloads re-launch at boot or on a defined interval even if other mechanisms fail. In real scenarios, layered tampering plus automated execution can create durable persistence, because even if one artifact is removed, another component re-establishes the attacker's access path.

Exam Focus

- Know the major tampering targets: firewalls, antivirus/security tools, security policies, registry startup locations, and logging services.

- Understand why tampering supports persistence: it reduces blocking and detection so access mechanisms (C2/shells/payloads) remain functional.
- Be ready to identify layered persistence in scenarios: combine control tampering with scheduled/startup execution to survive reboots and cleanup

Section 26. Lateral Movement

9 Lessons | Exam Objectives Covered: 5.2 | Does not include demonstrations

Lateral Movement (OBJ 5.2)

Lateral movement refers to the techniques attackers use to move from one compromised system to others within the same network after gaining initial access. Rather than remaining on the first compromised host, attackers expand their reach to identify high-value systems, escalate privileges, and establish broader control across the environment.

Mastering lateral movement techniques allows penetration testers to realistically simulate adversary behavior. Real attackers rarely stop at initial access; instead, they leverage trust relationships, weak configurations, exposed services, and credential reuse to move deeper into the network.

Exam Focus

- Lateral movement expands control beyond the initial foothold.
- Pivoting and relays are foundational techniques.
- Enumeration identifies new systems, services, and credentials.
- Attackers blend lateral movement into normal administrative activity.

Pivoting and Relaying (OBJ 5.2)

Pivoting and relay attacks allow attackers to expand access beyond the initially compromised system. Pivoting routes traffic through a compromised host, while relay attacks forward captured authentication attempts to exploit trust relationships.

Network Pivoting

Network pivoting involves routing attacker traffic through a compromised system to access internal resources that are otherwise unreachable. This is commonly achieved through SSH tunneling or local port forwarding, effectively turning the compromised host into a gateway.

Port forwarding enables access to internal web servers, file shares, or databases directly from the attacker's machine. This allows deeper exploration of segmented or restricted networks.

Application-Level Pivoting

Application-level pivoting leverages exploitation frameworks like Metasploit to route traffic through an active session. Meterpreter sessions can add routes that allow internal subnet scanning and exploitation.

This simplifies multi-stage attacks because scanning and exploitation modules automatically use the pivoted route. It allows structured and efficient lateral expansion across the network.

Authentication Relay Attacks

Relay attacks intercept authentication attempts and forward them to another system without cracking the password. This commonly exploits NTLM authentication in Windows environments.

Tools like ntlmrelayx can relay SMB or LDAP authentication to target systems. If the target accepts the relayed credentials, the attacker gains authenticated access.

Exam Focus

- Network pivoting uses tunneling or port forwarding.
- Application-level pivoting leverages exploitation frameworks.
- NTLM relay attacks forward authentication to other systems.
- SMB and LDAP are common relay targets.
- Pivoting and relays expand attacker footholds.

Enumerating for Lateral Movement (OBJ 5.2)

Enumeration identifies systems, credentials, services, and communication patterns that support lateral movement. After initial compromise, attackers gather intelligence to determine where to move next.

Service Discovery

Service discovery identifies open ports and active services on internal hosts. Tools like Nmap reveal services such as SMB, RDP, and HTTP that expand the attack surface.

Each exposed service presents potential exploitation or credential-based access opportunities. Understanding available services helps prioritize lateral targets.

Network Traffic Discovery

Traffic analysis reveals communication patterns between hosts. Tools like Wireshark and tcpdump capture protocol usage and authentication flows.

Cleartext protocols or repeated authentication attempts may expose credentials. This intelligence helps identify high-value targets and pivot paths.

Credential Capturing and Dumping

Credential capturing intercepts authentication material during network communication. Tools like Responder poison LLMNR and capture NTLM hashes.

Credential dumping extracts stored credentials from compromised systems using tools such as Mimikatz. Recovered credentials frequently enable rapid lateral expansion.

Exam Focus

- Enumeration guides lateral movement decisions.
- Service discovery reveals exposed attack surfaces.
- Credential capturing intercepts authentication in transit.
- Credential dumping extracts stored authentication material.
- Enumeration supports deeper post-exploitation.

Service Discovery (OBJ 5.2)

Service discovery focuses on Windows-based services commonly abused for lateral movement. SMB, RPC, and DCOM expose remote access and execution functionality.

Server Message Block (SMB)

SMB enables file and printer sharing over port 445. Enumerating shares may reveal sensitive files or anonymously accessible resources.

SMB credential testing can enable unauthorized access. SMB is frequently used in relay attacks and privilege escalation scenarios.

Remote Procedure Call (RPC)

RPC allows systems to request remote services across the network. Enumerating RPC endpoints reveals exposed services and potential vulnerabilities.

RPC-based vulnerabilities historically enabled large-scale compromises. Compromised RPC services support remote code execution.

Distributed Component Object Model (DCOM)

DCOM enables remote interaction between Windows components. Attackers can leverage DCOM for legitimate-looking remote execution.

Because DCOM resembles administrative activity, it often evades detection. This makes it effective for stealthy lateral movement.

Exam Focus

- SMB reveals shared resources and credential opportunities.
- RPC enumeration exposes remote service endpoints.
- DCOM enables remote execution using built-in mechanisms.
- Service discovery is critical in Windows environments.
- Expect scenario-based lateral movement questions.

Protocol Discovery (OBJ 5.2)

Protocol discovery identifies cleartext protocols that expose sensitive information. Unencrypted traffic allows attackers to capture credentials and data directly.

Telnet

Telnet provides plaintext remote command-line access. Captured traffic reveals usernames and passwords directly.

This makes Telnet vulnerable to interception attacks. Legacy systems often expose Telnet internally.

FTP

FTP transmits credentials and file data without encryption. Intercepted sessions may reveal sensitive files and login information.

Captured FTP credentials can support lateral movement. Unencrypted transfers expose valuable internal data.

LDAP

LDAP manages directory services such as Active Directory. Unencrypted LDAP exposes authentication and directory queries.

Captured LDAP credentials may provide domain-wide access. This significantly enhances lateral movement capabilities.

Exam Focus

- Cleartext protocols expose credentials in transit.
- Telnet and FTP transmit authentication without encryption.
- Unencrypted LDAP exposes directory data.
- Protocol discovery identifies insecure communications.
- Expect credential interception scenarios.

Remote Access Discovery (OBJ 5.2)

Remote access discovery identifies services that enable remote system control. RDP, VNC, SSH, and WMI frequently serve as lateral movement pathways.

Remote Desktop Protocol (RDP)

RDP provides graphical desktop access over port 3389. Weak or reused credentials commonly enable unauthorized RDP access.

Successful RDP access grants full system interaction. This enables privilege escalation and tool deployment.

Virtual Network Computing (VNC)

VNC provides cross-platform graphical remote access. Many deployments rely on weak or default passwords.

Compromised VNC sessions enable interactive control. This supports lateral expansion and privilege escalation.

Secure Shell (SSH)

SSH provides encrypted remote command-line access. Stolen credentials or private keys enable unauthorized authentication.

SSH tunneling supports stealthy pivoting. This allows attackers to bypass firewall restrictions.

Windows Management Instrumentation (WMI)

WMI enables remote command execution across Windows systems. With administrative credentials, attackers can execute scripts remotely.

WMI blends into normal administrative behavior. This makes it a stealthy lateral movement technique.

Exam Focus

- RDP provides full graphical control.
- VNC often suffers from weak credential exposure.
- SSH supports encrypted remote access and tunneling.
- WMI enables remote command execution.
- Remote access technologies facilitate lateral movement.

Printer Discovery (OBJ 5.2)

Printers are networked devices often overlooked in security monitoring. Misconfigured printer services can provide footholds for lateral movement.

Line Printer Daemon (LPD)

LPD accepts remote print jobs in Unix-based environments. Unauthenticated configurations may allow malicious job submission.

Malicious print jobs can execute embedded scripts. Compromised printers may scan networks or capture data.

JetDirect and SNMP

JetDirect enables direct network printing over TCP port 9100. Default SNMP community strings frequently remain unchanged.

Unauthenticated access allows configuration manipulation. Compromised printers can serve as pivot points.

Exam Focus

- Printers are frequently misconfigured.
- LPD may allow unauthenticated job submission.
- JetDirect exposes TCP 9100 and SNMP weaknesses.
- Compromised printers support lateral movement.
- Expect printer-based exploitation scenarios.

Discovering Internal Websites (OBJ 5.2)

Internal web applications often expose administrative functionality within trusted networks. Discovering and exploiting these services supports lateral movement.

Default Credentials and Weak Authentication

Many internal web interfaces use default credentials. Failure to change defaults grants immediate administrative access.

Administrative access enables configuration changes and data extraction. Default credentials remain a high-impact weakness.

Outdated Software and Session Hijacking

Outdated internal applications often contain known vulnerabilities. SQL injection and XSS flaws can expose credentials and databases.

Session hijacking exploits insecure cookie transmission. Captured session tokens enable impersonation.

API Abuse and Request Smuggling

Internal APIs may accept unsafe parameters. Improper validation allows privilege escalation or data exposure.

HTTP request smuggling manipulates server request parsing. This can bypass authentication controls.

Exam Focus

- Internal web portals frequently expose admin functionality.
- Default credentials provide immediate compromise.
- Outdated software introduces exploitable vulnerabilities.
- Session hijacking exploits insecure cookies.
- API abuse enables advanced lateral movement.

Living Off the Land Tools (OBJ 5.2)

Living off the land tools are legitimate Windows binaries abused for lateral movement. Because these tools are trusted and signed, they often evade detection.

Network Discovery Tools

Built-in utilities such as netstat and net use reveal connections and shared resources. These tools provide visibility into the network structure.

Enumerating users and groups assists privilege escalation planning. Native commands allow intelligence gathering without introducing malware.

Code Execution Binaries

cmd.exe, rundll32.exe, and msbuild.exe execute commands or embedded code. These binaries blend into legitimate administrative activity.

Using LOLBins reduces detection risk. They enable stealthy payload execution.

Reconnaissance Utilities

strings.exe and findstr.exe search files for sensitive data. These tools uncover credentials, API keys, or configuration secrets.

Because they are native tools, their use appears legitimate. LOLBins form a comprehensive stealth toolkit for post-exploitation.

Exam Focus

- LOLBins are legitimate system binaries.
- Built-in tools support stealthy lateral movement.
- Network utilities reveal internal structure.
- Code execution binaries avoid dropping malware.
- Expect scenario questions involving living off the land.

Section 27. Exfiltration

4 Lessons | Exam Objectives Covered: 5.3 | Does not include demonstrations

Exfiltration (OBJ 5.3)

In this section, we focus on exfiltration, which involves the techniques and tools attackers use to extract data covertly from compromised systems. Exfiltration represents the stage of an attack where value is realized, because sensitive corporate documents, credentials, intellectual property, or personally identifiable information are removed from the environment while attempting to avoid detection.

Understanding Exfiltration in Post-Exploitation

Exfiltration is the process of moving data from a compromised system to attacker-controlled infrastructure in a way that minimizes detection. Attackers deliberately stage, package, and disguise data transfers so they blend into legitimate traffic patterns and avoid triggering monitoring systems.

Covert Channels as Exfiltration Mechanisms

Covert channels enable attackers to send data through mechanisms not originally intended for hidden communication. These methods disguise data inside trusted protocols such as DNS, ICMP, HTTPS, or media files, allowing information to leave the environment unnoticed.

Alternate Data Streams and Hidden Storage

Alternate Data Streams (ADS) allow attackers to hide data inside NTFS file attributes without altering visible file contents. This enables staging of stolen information locally before it is quietly transferred out of the system.

Common Data Exfiltration Strategies

Attackers frequently rely on trusted workflows such as email, cloud storage synchronization, virtual disk files, or cross-account cloud permissions to move data externally. Because these methods resemble legitimate business activity, they are difficult to distinguish from normal operations without deep inspection.

Exam Focus

- Exfiltration is part of Domain 5 and aligns to Objective 5.3 (staging and exfiltration concepts).
- Attackers stage, disguise, and transfer data using trusted protocols and workflows.
- Covert channels hide communication inside legitimate traffic.
- Alternate Data Streams allow hidden storage prior to data removal.
- Trusted services such as email and cloud storage are commonly abused.

Covert Channels (OBJ 5.3)

In this lesson, we examine covert channels and how attackers use them to move data or maintain command-and-control communication without detection. A covert channel transfers information using mechanisms not originally intended for hidden communication, allowing attackers to blend malicious traffic into normal activity.

Steganography as a Covert Channel

Steganography hides information inside benign files such as images, audio, or video by modifying insignificant bits of data. Because these changes are visually undetectable, attackers can distribute payloads or instructions through trusted media files without raising suspicion.

DNS Covert Channels

DNS covert channels encode data inside subdomain queries and responses, taking advantage of the fact that DNS traffic is almost always permitted through firewalls. By splitting sensitive information into small encoded requests, attackers can reconstruct the data on attacker-controlled DNS servers outside the network.

ICMP Covert Channels

ICMP covert channels embed data inside ICMP echo request and reply payloads, which are typically used for diagnostics such as ping. Since ICMP traffic is often trusted and lightly inspected, it provides a stealthy method for transmitting commands or exfiltrating data.

HTTPS Covert Channels

HTTPS covert channels hide malicious communication within encrypted web traffic that appears legitimate. Because the data is encrypted with TLS, security devices cannot easily inspect the content, allowing command-and-control or data exfiltration to pass unnoticed.

Exam Focus

- Covert channels hide communication inside trusted formats or protocols.
- Steganography conceals data inside media files.
- DNS tunnels encode data inside DNS queries.
- ICMP payloads can carry hidden data.
- HTTPS encryption provides highly resilient covert communication.

Alternate Data Streams (ADS) (OBJ 5.3)

In this lesson, we focus on Alternate Data Streams (ADS) and how they can be abused during offensive engagements to hide staged data. ADS are a feature of NTFS that allows additional hidden content to be attached to files without altering visible file contents or size.

Understanding NTFS Alternate Data Streams

NTFS supports multiple data streams for a single filename, allowing hidden metadata or supplemental content to exist alongside the primary file data. Most file browsing tools display only the primary stream, meaning additional streams remain invisible during routine inspection.

Hiding Collected Information in ADS

Attackers can attach sensitive information to an existing file by creating a hidden stream using native Windows commands. The file continues to behave normally, which makes this an effective method for staging stolen data locally without drawing attention.

Exfiltrating Hidden Streams

Once data is stored in an alternate stream, the file can be copied or transferred as a normal file, carrying the hidden content with it on NTFS volumes. The hidden stream can later be extracted using built-in commands, allowing attackers to retrieve staged data discreetly.

Detection Considerations

Standard file inspection tools do not reveal alternate data streams, which is why attackers favor this technique. Specialized tools such as Sysinternals Streams can enumerate ADS, and monitoring solutions may detect unusual file behavior.

Exam Focus

- ADS allow hidden content to be attached to NTFS files.
- Hidden streams do not change visible file size or content.
- ADS can be staged locally before exfiltration.
- Specialized tools are required to detect ADS.

Exfiltrating Data (OBJ 5.3)

In this lesson, we examine practical techniques attackers use to move collected data out of a compromised environment. Successful exfiltration depends on blending malicious transfers into trusted systems and workflows so that activity appears legitimate.

Virtual Drive Mounting

Attackers can create virtual hard disk (VHD) files on compromised systems and store stolen data inside them. Because disk image files are often treated as benign large files, security controls may not inspect their internal contents closely.

Email-Based Exfiltration

Compromised email accounts can be used to send attachments or embed sensitive information in outbound messages. Since email is routine business traffic and often encrypted, it can quietly transport data outside the organization.

Cross-Account Cloud Abuse

In cloud environments, trust relationships between accounts can allow attackers to copy data into attacker-controlled accounts. Misconfigured permissions or shared storage resources can enable large-scale data movement without triggering immediate alerts.

Cloud Storage and Public Text Sites

Cloud storage services such as Google Drive or Dropbox can be used to upload stolen data in a way that resembles normal file synchronization. Public text storage sites like Pastebin allow encoded data to be stored and retrieved later without maintaining direct connections to the compromised system.

Exam Focus

- Exfiltration leverages trusted systems and workflows.
- Virtual disk files can stage large data sets.
- Email and cloud storage are common exfiltration channels.
- Cross-account cloud abuse enables large-scale data movement.
- Public text storage sites provide lightweight retrieval options.

Section 28. Cleanup and Restoration

8 Lessons | Exam Objectives Covered: 5.4 | Does not include demonstrations

Cleanup and Restoration (OBJ 5.4)

Cleanup and restoration represent the final phase of a penetration test or breach response, focusing on returning systems to a stable and secure operating state. This phase ensures that no residual artifacts, tools, accounts, or configuration changes remain that could introduce unintended risk after testing concludes.

Purpose of Cleanup and Restoration

Cleanup removes tools, artifacts, accounts, and configurations introduced during testing or exploitation. Restoration validates that systems function as intended after remediation and that no unintended weaknesses remain.

Domain and Objective Alignment

This section aligns to Domain 5, Post-exploitation and Lateral Movement. Objective 5.4 requires explaining cleanup and restoration activities performed after testing.

Core Cleanup Activities

Cleanup activities include persistence removal, reverting configuration changes, removing created credentials, eliminating testing tools, decommissioning infrastructure, preserving artifacts, and securely destroying sensitive data. Each activity ensures the environment returns to its original operational baseline without residual exposure.

Exam Focus

- No tester-introduced access or configuration should remain after engagement.
- Restoration validates system stability and security control functionality.

Persistence Removal (OBJ 5.4)

Persistence removal focuses on eliminating mechanisms that allow continued access after an initial compromise. If persistence mechanisms are not fully removed, attackers can quietly re-enter the environment even after remediation appears complete.

Scheduled Tasks and Startup Automation

Attackers abuse Windows Task Scheduler or Linux cron to automatically execute malicious scripts. Removal requires identifying and deleting malicious scheduled entries using `schtasks`, Task Scheduler, or `crontab -e`.

Registry-Based Persistence

Windows registry keys in HKLM or HKCU may be modified to launch malicious programs at startup. Cleanup requires precise removal of malicious values while documenting all registry changes.

Hidden Files and Unauthorized Accounts

Attackers may hide binaries or create unauthorized local or domain accounts for continued access. Removal must occur at the correct authority level, especially in Active Directory environments.

Remote Access Tools and Backdoors

Payloads, keyloggers, or backdoors may remain installed after exploitation. Specialized tools may be required to detect and fully remove these artifacts.

Exam Focus

- Persistence mechanisms must be fully identified and removed.
- Scheduled tasks, registry keys, hidden files, and accounts are common techniques.
- Documentation of all changes enables accurate cleanup.

Revert Configuration Changes (OBJ 5.4)

Reverting configuration changes ensures all temporary testing modifications are returned to their original secure state. Failure to revert changes can leave unintended vulnerabilities or operational disruptions.

Network and Firewall Rules

Temporary firewall rules opened during testing must be documented and removed. Snapshots can assist with restoring systems to their original configuration.

System and Application Security Settings

Testing may involve weakening encryption or authentication controls to validate impact. These settings must be restored to enforce proper security mechanisms.

Database, Logging, and DNS Settings

Database permissions modified for testing must be reverted to restrictive defaults. Logging and DNS configurations must also be restored to ensure visibility and normal operations.

Exam Focus

- All configuration changes must be documented and reversed.

- Firewall, database, logging, and DNS settings are common targets.
- Proper rollback prevents post-engagement security gaps.

Created Credentials Removal (OBJ 5.4)

Tester-created credentials must be removed at the conclusion of an engagement. Leaving accounts or tokens active creates unintended long-term access paths.

User Account Removal

Local accounts can be deleted directly on standalone systems. In centralized systems such as Active Directory, deletion must occur at the domain controller.

Authentication Authority Awareness

Cleanup depends on where authentication authority resides. Removing an account locally does not remove it from centralized directory services.

Revoking API Keys and Tokens

API keys and tokens created during testing must be explicitly revoked in the issuing service. Deleting scripts referencing keys does not invalidate the credential itself.

Exam Focus

- Remove credentials at the correct authentication authority level.
- Local and domain accounts require different cleanup approaches.
- API keys must be revoked, not just deleted locally.

Removal of Testing Tools (OBJ 5.4)

Testing tools deployed during exploitation must be fully removed to restore the operational baseline. Both in-memory and persistent tools require verification.

In-Memory vs Persistent Tools

In-memory payloads disappear on reboot but should still be verified. Persistent executables, scripts, and agents must be deliberately deleted.

Secure Deletion of Artifacts

Logs, configuration files, and captured data may remain after tool removal. Utilities such as shred or secure deletion tools prevent forensic recovery.

Removal of Automation Tasks

Cron jobs or scheduled tasks created to execute tools must be deleted. Failure to remove automation can reintroduce access after engagement.

Exam Focus

- Remove executables, logs, agents, and supporting files.
- Use secure deletion for sensitive artifacts.
- Eliminate scheduled automation linked to testing tools.

Decommission Testing Infrastructure (OBJ 5.4)

Testing infrastructure must be intentionally terminated after engagement completion. Leaving infrastructure active creates financial, operational, and security risks.

Identifying Created Resources

All virtual machines, containers, databases, storage volumes, and networking components must be inventoried. Cloud resources must be reviewed across all regions used.

Terminating Compute and Storage

Instances must be terminated rather than merely stopped. Storage volumes, snapshots, and backups containing sensitive data must be securely handled or deleted.

Networking and Log Cleanup

Temporary VPCs, security groups, and routing rules must be removed. Logs generated during testing must be securely stored or destroyed per engagement requirements.

Exam Focus

- Terminate all compute, storage, and networking resources.
- Review all cloud regions for overlooked assets.
- Handle backups and logs securely to prevent residual exposure.

Artifact Preservation (OBJ 5.4)

Artifact preservation ensures evidence generated during testing is securely collected and stored. Artifacts provide proof of impact and support defensible reporting.

Types of Artifacts

Artifacts include logs, screenshots, scripts, configuration files, scan results, and notes. These records demonstrate what was tested and how access was achieved.

Secure Storage and Documentation

Artifacts must be stored securely with access controls and backups. Documentation and labeled screenshots ensure repeatability and clarity.

Handling Sensitive Data

Sensitive data encountered during testing must be protected and retained only as long as necessary. Data should be anonymized or securely deleted when no longer required.

Exam Focus

- Artifacts provide proof of work and must be secured.
- Use encryption and access controls for stored evidence.
- Protect and limit retention of sensitive data.

Secure Data Destruction (OBJ 5.4)

Secure data destruction ensures all testing artifacts are permanently removed and unrecoverable. Simple file deletion is insufficient because data remains recoverable until overwritten.

Why Deletion Is Insufficient

Deleting a file removes its pointer but not its data. File carving techniques can recover deleted artifacts if they are not overwritten.

Shredding and Overwriting

Shredding overwrites files with random data or zeros multiple times to prevent reconstruction. Tools such as shred automate this process on traditional drives.

Storage Type Considerations

HDDs support effective overwriting due to predictable storage patterns. SSDs require manufacturer-supported secure erase commands because wear leveling prevents reliable overwriting.

Exam Focus

- Deletion alone does not remove data permanently.
- Use shredding or overwrite techniques for HDDs.
- Use secure erase commands for SSDs.

Section 29. Remediation Recommendations

11 Lessons | Exam Objectives Covered: 1.5 | Does not include demonstrations

Remediation Recommendations (OBJ 1.5)

Remediation recommendations are the actionable guidance provided to address vulnerabilities identified during a penetration test. They bridge the gap between discovery and improvement by turning technical findings into practical steps that reduce risk. Rather than simply identifying what is broken, effective remediation explains what must change, why it matters, and how to prevent recurrence. This section focuses on analyzing findings and recommending appropriate remediation within a report, covering system hardening, input sanitization, network controls, authentication, encryption, patch management, process changes, administrative controls, physical controls, and operational policies.

Exam Focus

- Remediation recommendations translate technical findings into actionable security improvements.
- Effective remediation reduces risk and prevents recurrence of vulnerabilities.
- Remediation spans technical, administrative, physical, and operational controls.

System Hardening (OBJ 1.5)

System hardening is the process of reducing a system's attack surface by removing unnecessary services, interfaces, and default configurations. From a remediation perspective, these actions directly reduce how easily a penetration tester or attacker can gain or maintain access after compromise.

System Hardening Fundamentals

System hardening begins with attack surface reduction, meaning any unnecessary service, interface, or feature should be removed or disabled. Using structured hardening checklists and ensuring regular patch installation reduces exposure to publicly known exploits and lowers compromise likelihood.

Configuration and Access Controls

Configuration controls include closing unused ports, uninstalling unnecessary services, and enforcing access control lists to limit who can access resources. Applying least privilege, reviewing administrative accounts, disabling unused accounts, and ensuring anti-malware updates directly restrict attacker escalation paths.

Patch Management with Risk Handling

Patch management involves identifying, testing, prioritizing, and deploying updates while balancing operational stability and security. Where patches are unavailable, compensating controls such as segmentation and restricted access reduce exploitability until permanent fixes are possible.

Exam Focus

- System hardening reduces attack surface by removing unnecessary services and configurations.
- Least privilege and secure account management prevent privilege escalation.
- Patch management is ongoing and requires prioritization and testing.
- Compensating controls protect legacy or unpatchable systems.

User Input Sanitization (OBJ 1.5)

User input sanitization remediation focuses on eliminating root causes of injection vulnerabilities rather than masking symptoms. From a penetration testing perspective, all input must be treated as untrusted and validated before processing.

Trust Boundaries and Input Validation

All user input from browsers, APIs, headers, and cookies must be treated as attacker-controlled and validated accordingly. Approved listing techniques that strictly define acceptable input are stronger than blocklisting and reduce bypass opportunities.

Root-Cause Injection Remediation

Injection vulnerabilities occur when code and data are not separated, allowing user input to alter application logic. Parameterized queries and prepared statements eliminate SQL injection by ensuring input is treated strictly as data rather than executable code.

Effective Remediation Communication

Remediation must identify all input points and enforce validation, parameterization, and output encoding. Follow-up testing and developer education ensure vulnerabilities do not reappear and secure coding becomes standard practice.

Exam Focus

- All input must be treated as untrusted by default.
- Approved listing is stronger than blocklisting.
- Parameterized queries eliminate SQL injection root causes.
- Remediation requires validation, output encoding, and follow-up testing.

Network and Infrastructure Controls (OBJ 1.5)

Network and infrastructure remediation focuses on reducing attacker movement and limiting impact after initial compromise. Recommendations should directly address weaknesses that enabled lateral movement, escalation, or persistence during testing.

Network Segmentation and Isolation

Network segmentation divides environments into isolated segments to contain compromise impact. Effective remediation requires enforced firewall rules and restrictive access control lists rather than relying solely on VLAN definitions.

Infrastructure Security Controls

Infrastructure controls strengthen authentication, encryption, patching, and monitoring to protect confidentiality, integrity, and availability. Recommendations must directly map to how attackers succeeded, closing privilege escalation and detection gaps.

Clear Remediation Communication

Remediation guidance should clearly define architectural redesign, segmentation enforcement, and monitoring validation steps. Training and documentation ensure secure configurations are maintained long-term.

Exam Focus

- Segmentation limits attacker movement and scope of compromise.
- Infrastructure controls must address authentication, encryption, and patching gaps.
- Monitoring must be properly tuned and actively reviewed.
- Remediation must be clearly communicated and enforceable.

Authentication Recommendations (OBJ 1.5)

Authentication remediation addresses credential abuse, authentication bypass, and trust failures. Recommendations must eliminate root causes rather than simply adding inconvenience.

Secrets Management

Secrets management governs how passwords, API keys, and tokens are generated, stored, and rotated. Password managers reduce reuse and weak passwords, directly raising attacker cost.

Multifactor Authentication (MFA)

Multifactor authentication requires independent verification factors, significantly disrupting credential-based attacks. Remediation must enforce MFA consistently and remove insecure fallback mechanisms.

Cryptographic Trust Management

Certificate lifecycle management and key rotation prevent trust abuse and limit compromise impact. Automation and secure key storage ensure long-term enforcement rather than policy-only controls.

Effective Remediation Communication

Remediation should prioritize high-risk systems such as remote access and cloud management. Clear, prioritized guidance ensures measurable reduction in credential attack success.

Exam Focus

- Password managers reduce reuse and weak credential exposure.
- MFA significantly disrupts credential-based attacks.
- Certificate management and key rotation limit trust abuse.
- Remediation must prioritize critical authentication systems.

Encryption Recommendations (OBJ 1.5)

Encryption remediation ensures data confidentiality at rest and in transit. Missing or weak encryption allows limited access to escalate into full data compromise.

Encryption Gaps and Risk

Encryption converts plaintext into ciphertext to prevent unauthorized reading of sensitive data. Unencrypted data at rest or in transit allows attackers immediate access once systems are compromised.

Strong Algorithms and Deployment

Modern algorithms such as AES and RSA must replace deprecated cryptography. Encryption must be consistently applied across wireless networks, remote administration, and authentication exchanges.

Key Management and Enforcement

Secure key storage, lifecycle management, and regular rotation prevent long-term compromise impact. Eliminating hard-coded keys and enforcing TLS ensures encryption is not undermined by misconfiguration.

Exam Focus

- Encryption protects confidentiality at rest and in transit.
- Modern algorithms must replace deprecated cryptography.
- Secure key management is essential to effective encryption.
- Proper implementation limits blast radius after compromise.

Patch Management Recommendations (OBJ 1.5)

Patch management remediation reduces exposure to known vulnerabilities while balancing operational stability. Recommendations must improve the overall patch process, not just address individual exploited systems.

Patch Management Fundamentals

Patch management identifies, tests, and deploys updates to fix vulnerabilities. Missing patches are frequently exploited because they represent publicly documented weaknesses.

Operational Constraints and Risk Handling

Legacy systems and stability concerns often delay patching, requiring testing and formal risk acceptance. Risk-based prioritization using CVSS aligns remediation with measurable severity levels.

Compensating Controls and Documentation

When patching is not possible, segmentation, monitoring, and restricted access reduce exploitability. Clear documentation ensures visibility and accountability for unpatched systems.

Exam Focus

- Patch management reduces exposure to known exploits.
- Risk-based prioritization aligns remediation with severity.
- Testing minimizes disruption before deployment.
- Compensating controls reduce risk when patching is delayed.

Process Level Remediation (OBJ 1.5)

Process-level remediation changes workflows and operational practices when technical fixes are not immediately possible. Attackers frequently exploit unsafe operational habits rather than broken systems.

Process-Level Fundamentals

Process-level remediation modifies how work is performed to reduce risk without changing underlying systems. This approach disrupts low-effort attack paths while preserving operational continuity.

Common Operational Weaknesses

Insecure practices such as Telnet usage, weak password rotation, and unencrypted traffic flows create exploitable conditions. Changing workflows, such as enforcing SSH or VPN routing, reduces exposure without system replacement.

Effective Remediation Communication

Recommendations must directly tie exploited weaknesses to specific process changes. Training ensures new workflows are understood and consistently followed.

Exam Focus

- Process-level remediation changes behavior rather than technology.
- Operational weaknesses often enable exploitation.
- Workflow adjustments reduce attacker success.
- Training reinforces long-term effectiveness.

Administrative Controls (OBJ 1.5)

Administrative controls are governance-level practices that shape access management and development security. Weak governance frequently enables technical vulnerabilities discovered during penetration tests.

Role-Based Access Control

RBAC assigns permissions based on job roles rather than individuals, enforcing least privilege when properly managed. Regular access reviews prevent role creep and excessive privilege accumulation.

Password Policies and Credential Practices

Modern guidance emphasizes long, memorable passwords and discourages outdated forced complexity rules. Credential managers reduce reuse and increase resistance to spraying and cracking attacks.

Policies and Secure SDLC

Policies define expectations while procedures define implementation steps, and weak policies allow vulnerabilities to persist. A structured SDLC ensures security is integrated from planning through maintenance, reducing recurring flaws.

Exam Focus

- RBAC enforces least privilege when properly managed.
- Modern password guidance favors length over complexity.
- Credential managers reduce reuse and weak passwords.
- Secure SDLC prevents recurring vulnerabilities.

Physical Controls (OBJ 1.5)

Physical remediation strengthens access restrictions and monitoring to prevent unauthorized facility access. Penetration tests frequently demonstrate that physical access is easier than remote compromise.

Physical Access Controls

Badge enforcement, biometrics, and access control vestibules reduce tailgating and impersonation. Multi-factor physical authentication significantly lowers risk from stolen or cloned credentials.

Protection of Facilities and Assets

Locked racks, controlled key management, and personal device restrictions limit impact after entry. These controls reduce data exfiltration and unauthorized equipment access.

Monitoring and Awareness

Recurring training improves resistance to social engineering and reinforces badge verification practices. Proper camera placement and retention policies increase accountability and deterrence.

Exam Focus

- Physical access controls reduce unauthorized entry.
- Layered protections limit impact after entry.
- Training increases social engineering resistance.
- Surveillance enhances detection and deterrence.

Operational Controls and Policies (OBJ 1.5)

Operational controls influence how people work, when they access systems, and how misuse is detected. These controls raise attacker cost without relying solely on technical safeguards.

Security Awareness and Training

Structured user education reduces phishing success and social engineering risk. Simulated training and role-based awareness measurably increase resistance to human-based attacks.

Personnel-Based Fraud Prevention

Job rotation and mandatory vacations reduce unchecked, long-term access and expose misuse. These practices disrupt insider threats and credential abuse persistence.

Time-Based Access Restrictions

Time-of-day restrictions limit when users can authenticate, reducing after-hours exploitation opportunities. Constraining access windows increases anomaly detection when credentials are misused.

Exam Focus

- Security awareness reduces social engineering success.
- Job rotation and vacations disrupt long-term misuse.
- Time-based restrictions limit credential abuse flexibility.
- Operational controls reduce attacker dwell time.

Section 30. Penetration Test Reporting

8 Lessons | Exam Objectives Covered: 1.2, 1.4 | Does not include demonstrations

Penetration Test Reporting (OBJ 1.4)

Penetration test reporting is the final phase of the testing process and is often the most important because it translates technical findings into actionable insight. A well-written report documents discovered vulnerabilities, explains their root causes and associated risks, and clearly outlines practical remediation recommendations so the organization can reduce risk and improve its security posture.

Purpose and Value of Reporting

The report serves as the roadmap for reducing risk and improving the organization's security posture. Rather than simply listing issues, it explains impact, likelihood, and priority so informed decisions can be made by both technical and non-technical stakeholders.

Engagement Management Objectives

This section aligns with Domain 1, Engagement Management, specifically Objectives 1.2 and 1.4. These objectives require explaining collaboration, communication activities, and the components of a penetration test report.

Reporting Lifecycle Overview

Penetration test reporting includes executive summaries, root cause analysis, structured report components, documentation standards, risk scoring, definitions, scope limitations, and special considerations. Together, these elements ensure findings are clearly communicated and actionable.

Exam Focus

- Penetration test reporting translates technical findings into actionable insight.
- Reports document vulnerabilities, root causes, risks, and remediation recommendations.
- The report acts as a roadmap for reducing organizational risk.

Executive Summary Process (OBJ 1.4)

The executive summary is a high-level overview of the penetration test and is often the only section read by senior stakeholders. Its purpose is to communicate risk, impact, and urgency clearly so leadership can make informed decisions.

Defining Purpose and Scope

The executive summary must explain what was tested, why the testing was performed, and the objectives of the engagement. Clearly stating scope ensures stakeholders understand the limits of the test and interpret findings appropriately.

Summarizing Methods and Key Findings

This section briefly explains the testing approach and highlights major findings in non-technical language. Vulnerabilities are translated into meaningful business impact rather than technical jargon.

Overall Security Posture and Conclusion

The executive summary concludes with a clear assessment of overall security posture. This provides leadership with a concise answer to how secure the organization is and what must be prioritized.

Exam Focus

- Executive summaries communicate risk and urgency to leadership.
- Scope and objectives must be clearly defined.
- Findings should be translated into non-technical business language.
- A clear conclusion must state overall security posture.

Root Cause Analysis (OBJ 1.4)

Root cause analysis explains why vulnerabilities exist rather than only documenting what was exploited. It identifies underlying conditions across people, processes, and technology that enabled successful attacks.

Purpose and Value

Root cause analysis transforms isolated findings into actionable insight by identifying systemic weaknesses. Addressing root causes prevents repeated exploitation of similar vulnerabilities.

Common Organizational and Technical Causes

Common causes include weak physical security, poor patch management, lack of operating system hardening, outdated protocols, obsolete cryptography, and insufficient cybersecurity training. These recurring weaknesses often enable multiple attack paths.

Linking Findings to Causes

Each vulnerability should be mapped back to its root cause so remediation addresses the true problem. This ensures organizations fix underlying weaknesses instead of repeatedly treating symptoms.

Exam Focus

- Root cause analysis explains why vulnerabilities exist.
- Common causes include weak training, poor patching, and outdated protocols.
- Mapping findings to root causes improves long-term security.

Report Components (OBJ 1.2, 1.4)

A complete penetration test report follows a structured format that ensures findings are clear, credible, and actionable. These components translate exploitation activity into useful information for stakeholders.

Core Report Sections

Core sections include the Executive Summary and Methodology, which establish context and credibility. These sections explain what was tested and how the engagement was conducted.

Technical Findings and Attack Narratives

Detailed Findings document vulnerabilities, severity, and exploitability. Attack Narratives describe step-by-step exploitation to demonstrate real-world attacker impact.

Recommendations and Professional Presentation

Recommendations address root causes and long-term improvements. Professional formatting, version control, and confidentiality notices enhance credibility and usability.

Exam Focus

- Executive Summary and Methodology establish credibility.
- Detailed Findings and Attack Narratives document exploitation.
- Recommendations address root causes.
- Professional presentation improves clarity and impact.

Risk Scoring and Prioritization (OBJ 1.4)

Risk scoring and prioritization convert exploitation results into measurable risk levels. This process determines which vulnerabilities require immediate remediation.

Risk Scoring Fundamentals

Risk scoring evaluates likelihood and impact using standardized models such as the Common Vulnerability Scoring System (CVSS). Scores map vulnerabilities to severity levels such as low, medium, high, or critical.

Articulating Severity and Impact

Severity must be explained in technical and business terms. Impact may include data loss, financial damage, reputational harm, or regulatory consequences.

Prioritization for Remediation

Prioritization focuses remediation efforts on vulnerabilities that provide rapid or deep compromise. This reflects real-world attacker behavior.

Exam Focus

- CVSS provides standardized severity scoring.
- Severity must be translated into business impact.
- Remediation should focus on the most exploitable issues first.

Definitions in the Report (OBJ 1.4)

The definitions section ensures all readers understand key terminology used in the report. This makes findings accessible to both technical and non-technical stakeholders.

Purpose of Definitions

Definitions clarify important terms such as vulnerability, exploit, risk, threat, mitigation, and remediation. Clear explanations prevent misunderstanding and improve report usability.

Common Terms Explained

Commonly defined terms include penetration testing, zero-day, encryption, firewall, and attack vector. Plain-language definitions ensure consistent interpretation of findings.

Exam Focus

- Definitions ensure clarity across audiences.
- Key terms such as vulnerability and exploit must be clearly explained.
- Plain-language definitions improve decision-making.

Limits and Assumptions (OBJ 1.2, 1.4)

Limits and assumptions define what the assessment can realistically discover and how results should be interpreted. They clarify that penetration testing is a controlled simulation, not complete coverage.

Penetration Test Limitations

Testing is limited to systems within the defined scope. Out-of-scope systems may contain vulnerabilities that are not identified.

Constraints and Assumptions

Time, resource, and access limitations affect test depth. Assumptions about environment stability and authorized access influence interpretation of results.

Exam Focus

- Scope boundaries limit assessment coverage.
- Time and resource constraints affect testing depth.
- Assumptions shape interpretation of findings.

Special Considerations (OBJ 1.2, 1.4)

Special considerations address how reports are communicated, protected, validated, and followed up after testing concludes. Because reports document compromise details, they must be securely handled.

Reporting and Secure Handling

Reports must be structured clearly and distributed securely using encrypted methods. Access controls and version tracking protect confidentiality, integrity, and availability.

Peer Review, Acceptance, and Retesting

Peer review improves report accuracy. Client acceptance and attestation confirm understanding, and retesting validates remediation effectiveness.

Exam Focus

- Reports must be securely distributed and protected.
- Peer review strengthens accuracy and credibility.
- Retesting confirms remediation effectiveness.

Section 31. Conclusion

2 Lessons

Conclusion

This final lesson serves as a comprehensive wrap-up of the entire CompTIA PenTest+ preparation journey. It reinforces the structure of the five exam domains, clarifies how the material was intentionally organized for effective learning, and provides practical guidance for successfully scheduling and passing the certification exam. Rather than introducing new technical content, this lesson consolidates everything learned and transitions you from student to certification candidate.

Review of the Five Exam Domains

The course covered all five domains of the CompTIA PenTest+ certification exam, even though they were not taught in the exact order listed in the official exam objectives. This intentional reorganization was designed to help you learn penetration testing in a more chronological and logical progression that mirrors how an actual engagement unfolds.

Domain One, Engagement Management (13%), focuses on planning, scoping, regulations, stakeholder communication, frameworks, and reporting. Domain Two, Reconnaissance and Enumeration (21%), centers on gathering information through active and passive techniques, including open-source intelligence.

Domain Three, Vulnerability Discovery and Analysis (17%), emphasizes identifying and assessing weaknesses across environments. Domain Four, Attacks and Exploits (35%), represents the largest portion of the exam and covers executing attacks and applying tools strategically.

Domain Five, Post-exploitation and Lateral Movement (14%), addresses actions taken after gaining access, including persistence and lateral movement. Together, these domains represent the complete lifecycle of a penetration test, which this course walked through in a structured and practical way.

Navigating the Exam Objectives

Although CompTIA publishes exam objectives in a fixed order, they are not necessarily arranged in the most intuitive teaching sequence. This course covered every objective and each sub-bullet, even if presented in a different order to enhance comprehension.

If you need to review specific objectives, each lesson title clearly includes the relevant objective number in parentheses. This makes it easy to return to particular exam objectives and reinforce any areas that require additional study.

Scheduling and Taking the Exam

The CompTIA PenTest+ exam can be scheduled through PearsonVue, either at a local testing center or remotely using the OnVue online proctoring system. Online testing requires a private room, a reliable internet connection, and a webcam so a live proctor can monitor the session.

To schedule the exam, you must first purchase an exam voucher. Vouchers can be purchased directly from PearsonVue or the CompTIA store at full retail price, or through Dion Training's voucher program at diontraining.com/vouchers, where eligible students may receive discounted pricing.

Five Exam Day Success Strategies

The first recommended strategy is to use the whiteboard provided during the exam to perform a brain dump of critical information such as ports or key terms. Spending the first one to three minutes writing this information down can provide a valuable reference later in the exam.

The second strategy is to skip performance-based simulations initially and complete all multiple-choice questions first. Students who return to simulations after building confidence with multiple-choice questions often see measurable score improvements.

The third strategy is to guess strategically when uncertain, since there is no penalty for incorrect answers. Eliminating clearly incorrect options significantly increases the probability of selecting the correct answer.

The fourth strategy is to schedule the exam at your optimal time of day and arrive early to reduce stress. If testing online, remember that restroom breaks are not permitted once the exam begins.

The fifth and final strategy is confidence. Use practice exams to identify weak areas, review detailed explanations, and continue practicing until you are confident—not memorizing answers, but understanding concepts and reasoning.

Using Practice Exams Effectively

The purpose of practice exams is not memorization but conceptual mastery. CompTIA writes the official questions, so understanding why answers are correct or incorrect is far more important than memorizing answer keys.

Review detailed explanations after each practice attempt and continue practicing until you feel fully prepared. Confidence should be based on consistent performance and understanding, not guesswork.

BONUS: What Comes Next?

This bonus lesson focuses on continued professional growth after completing the course. It emphasizes ongoing learning, additional certifications, and expanded training opportunities through Dion Training.

Continuing Certification Paths

Dion Training offers additional certification preparation courses including CySA+, Linux+, SecurityX, and ITIL4. These courses are available both on Udemy and directly through diontraining.com.

The website versions often include complete study solutions such as video training, quizzes, study guides, full-length practice exams, and in many cases, bundled exam vouchers.

Website Course Benefits

Courses purchased directly through diontraining.com include a 100% pass guarantee for eligible certifications. If a student completes the course requirements and does not pass on the first attempt, the cost of the retake is covered.

For CompTIA certifications, website courses may also include integrated hands-on labs providing approximately 40 hours of guided practical experience. These labs strengthen real-world, hands-on keyboard skills beyond exam preparation.

Exam Voucher and Take-Two Options

Exam vouchers can also be purchased directly through diontraining.com by selecting the certification and country of testing. Some certifications include a “take two” option, allowing a discounted retake if necessary.

Staying Connected and Advancing

Students are encouraged to remain engaged with the Dion Training community through social media and online groups. Continued learning, networking, and professional development are essential for long-term cybersecurity success.

The conclusion reinforces that completing the course demonstrates commitment and discipline. Whether advancing within a current role or transitioning into a new cybersecurity career, continued growth and education remain key to success.